

Mol: The Category of Stateful Transformations.
An Algebraic Theory of Zero-Allocation Dataplane Construction

XENOT Corporation

2026

Abstract

Network dataplane construction faces two opposing pressures. On one side, latency and throughput targets are met only by eliminating allocation from the hot path: no heap traffic, no per-packet allocation, no copy that can be fused away. On the other, a demand for modular construction: small components (parsers, filters, encoders, ring buffers, transports) composed into larger pipelines without surrendering the performance guarantee. The usual answer to compositionality, the abstraction layer, is itself the most common source of allocation; a dataplane that must be allocation-free by construction has to be optimized by composition, not in spite of it.

This thesis develops the algebraic theory behind it. It defines **Mol**, the category of stateful transformations, whose objects are types and whose morphisms are deterministic Mealy machines: equivalence classes of pairs $(S, step)$ with $step : S \times A \rightarrow B \times S$, quotiented by the state-space bijection condition so that only observable behavior identifies a machine. Mol is symmetric monoidal under the tuple product, unit $()$; sequential composition is $\circ$ and parallel composition is $\otimes$. Its pure subcategory is equivalent to the category of types and total functions, and its effectful subcategory is isomorphic to the Kleisli category of the state monad; every hybrid molecule decomposes uniquely as $q \circ e \circ p$.

The thesis statement is that Mol is a complete algebraic foundation for zero-allocation dataplane construction: every dataplane artifact is a molecule, and its engineering properties are theorems. The theorem catalog, each with a complete proof, covers: the structure of Mol; equational theories for rings, parsers, and transports and their rewrite theories; free molecules and normal forms as a precomputable lookup table; cost enrichment with global optimality by shortest paths; Kleisli decomposition and traced feedback with Banach and Dobrushin bounds; operadic batching with syscall amortization; cut elimination as pipeline fusion, linearity as a zero-allocation guarantee, and affine typing; situations as finite, decidable optimization problems; and the optimization theory of the kernel boundary, the affine crossover, and the lattice method. Classical results (coherence, the Kleisli construction, Nerode–Hopcroft minimization, contraction theory, traced structure) are restated and proved inside Mol. Two optimality theorems anchor the model itself: the pure, effectful, and hybrid classes are the unique congruence-compatible extension of the atom classification, and the free construction is initial, so every application expressible as a stateful transformation is expressible as a molecule without rigidity.

The upshot is a theory in which normalization is sound and cost-preserving, minimal machines are computable, loops are provably bounded, and zero allocation is a consequence of typing rather than a performance accident. Computational claims are verified by executable Rust tools; no conjecture is presented as a theorem. A companion software standard turns the same theorems into enforceable engineering policy.

Contents

1	Introduction	6
1.1	Motivation	6
1.2	Thesis statement	6
1.3	Contributions and theorem catalog	7
1.4	Scope	7
1.5	Roadmap	8
2	Preliminaries	9
2.1	Categories	9
2.2	Functors	10
2.3	Natural Transformations	10
2.4	Monoidal and Symmetric Monoidal Categories	10
2.5	Traced Monoidal Categories	11
2.6	Monads, Algebras over Monads, and Kleisli Categories	11
2.7	Coalgebras	12
2.8	Lawvere Theories	12
2.9	Operads, Multicategories, and Clubs	13
2.10	Effectus Theory	13
2.11	Linear Logic and Geometry of Interaction	14
2.12	Accessible and Locally Presentable Categories	14
2.13	Universal Algebra and Clones	14
2.14	Notation and acronyms	15
3	The Category Mol	17
3.1	Machines and Morphisms	17
3.2	Symmetric Monoidal Structure	19
3.3	Pure Molecules	20
3.4	Effectful Molecules and the Kleisli Category	21
3.5	Hybrid Molecules	22
3.6	Behavioral Equivalence	22
3.7	Atoms and Determinism	24
3.8	Discussion	24
3.9	Optimality of the model	24
4	Lawvere Theories for Dataplane Components	27
4.1	The Ring-Buffer Theory	27
4.2	The Parser Theory	29

4.3	The Transport Theory	30
4.4	Models in Mol and the Licensed Rewrite Rules	30
5	The Rewrite Engine	32
5.1	Extending the Engine: Modular Completion	36
6	Free Molecules and Canonical Normal Forms	39
6.1	Types, atoms, and signatures	39
6.2	The free category on a signature	40
6.3	Existence and the universal property	40
6.4	Finiteness of hom-sets	41
6.5	Unique normal forms up to coherence	41
6.6	The precomputed lookup table	43
7	Enrichments and Quantitative Semantics	45
7.1	The cost model	45
7.2	Global optimality as a shortest-path problem	46
7.3	The fusion bound	47
7.4	Refinement and fast-path specialization	48
7.5	Complexity closure	50
7.6	Descriptive complexity	50
7.7	Projection reduction	51
7.8	Resource-Sensitive Cost Enrichment	52
7.9	Summary	54
8	Kleisli Adjunction and Pure/Effectful Decomposition	55
9	Coalgebraic Minimization	59
10	Traced Monoidal Structure, Feedback, and Fixed-Iteration Loops	63
10.1	The Reactor Loop	63
10.2	Contraction and Convergence	64
10.3	Contraction Rates under Composition	65
10.4	Stochastic State Updates and the Dobrushin Coefficient	66
10.5	Information Geometry as Future Work	67
10.6	Summary	67
11	Operadic Composition and Batching	68
12	Linear Logic, Cut Elimination, and Zero Allocation	71
12.1	The Affine Type System and the No-Allocation Theorem	73
13	Situations and Applications	77
14	Testing Strategies Derived from the Framework	80
14.1	Equational Law Testing (Theorem 4.3–Theorem 4.8)	80
14.2	Bisimulation and Equivalence Checking (Theorem 3.23)	80
14.3	Model-Based Testing	81
14.4	Conformance to Lawvere Theories	81

14.5	Fuzzing for Memory Safety	81
14.6	Differential Testing	81
14.7	Cost-Annotated Regression (Theorem 7.3–Theorem 7.8)	82
14.8	Property-Based Testing of Rewrites (Theorem 5.1–Theorem 5.7)	82
14.9	State-Minimization Verification (Theorem 9.6–Theorem 9.9)	82
14.10	Type-Level Guarantees	83
15	Related Work	84
15.1	Sequential Machines: Mealy and Moore	84
15.2	Coalgebra and Automata Theory	84
15.3	Traced Monoidal Categories	85
15.4	Lawvere Theories	85
15.5	Operads, Multicategories, and Clubs	85
15.6	Linear Logic and Geometry of Interaction	85
15.7	Effectus Theory and Markov Categories	85
15.8	Bisimulation as Congruence	86
15.9	Implicit and Descriptive Complexity	86
15.10	High-Performance Dataplanes	86
16	Conclusion	87
16.1	The Theorem Catalog	87
16.2	Limitations	88
16.3	Future Work	93
17	Optimization Discovery: Kernel-Boundary Cost, Threshold Theorems, and the Lattice Method	94
17.1	The kernel-boundary enrichment	94
17.2	The batch fusion law	95
17.3	Threshold theorems: realization choice as a crossover	96
17.4	The optimization lattice	96
17.5	Datapath bisimulation and bypass as retraction	97
17.6	Cache soundness as bounded staleness	97
17.7	The scheduler estimate is a band contraction	98
17.8	Zero-boundary loops: the hot-path effect discipline	98
17.9	Route matching as a deterministic automaton	99
17.10	Realization choice is a measured threshold	100
17.11	The zero-allocation claim is machine-checked	100
17.12	Startup lattice autotuning	100
17.13	Measured case studies: before and after	101
17.13.1	Case study 1: the input-output molecule	101
17.13.2	Case study 2: the pure compute molecule	101
17.13.3	How the optimizations stack	102
17.14	Verification and summary	103
A	Proof-Checking Methodology	105
A.1	The Verification Discipline	105
A.2	The Verification Tools	105
A.3	Reproducibility	105

A.4	Scope of the Machine Checks	107
B	Optimization and Testing Catalogs	108
B.1	Theorem-to-Optimization Catalog	108
B.2	Testing-Strategy Catalog	108
B.3	How to Read the Catalogs	108

Chapter 1

Introduction

1.1 Motivation

A network dataplane is the part of a protocol stack that moves bytes on the hot path: parsing, validating, filtering, routing, buffering, and exchanging data with the transport. In the construction of network dataplanes, two notions organize the work. An *atom* is the smallest useful component: a pure or effectful transformation with no hidden state, total on its declared domain. A *molecule* is a composition of atoms (sequential or tensorial) with bounded internal state; a ring buffer, a parser, and the whole receive path are molecules. The engineering target is concrete: *zero allocation on the hot path*: no heap traffic, no per-packet allocation, no copy that can be fused away.

The target is hard because the requirement is global (an allocation anywhere in a composed pipeline defeats the guarantee everywhere) and because the usual answers to compositionality, abstraction layers, dynamic dispatch, managed containers, are themselves the most common sources of allocation. A dataplane that must be allocation-free by construction must therefore be *optimized by composition*: the object of study is a family of stateful transformations together with the rules for joining them, and the performance of a pipeline is a property of how its parts are wired.

Sequential machines are the natural model: Mealy machines [Mealy, 1955] compute deterministically from a fixed state space, and Moore’s analysis [Moore, 1956] established the basic shape of minimization. What is missing is a compositional, algebraic treatment: a category whose objects are types and whose morphisms are machines, closed under the two composition schemes a dataplane uses (sequential ($\circ$) and parallel ($\otimes$)) and in which engineering questions have theorem-shaped answers. Does normalization preserve behavior and cost? Is the minimal state space computable? When does a loop provably converge? When is a pipeline provably allocation-free? This thesis answers them by developing the category **Mol** of stateful transformations and proving the structure theorems that make it a foundation.

1.2 Thesis statement

Thesis. The category **Mol** of stateful transformations (deterministic Mealy machines (S, step) with $\text{step} : S \times A \rightarrow B \times S$, quotiented by the state-space bijection condition) is a complete algebraic foundation for the construction of zero-allocation network dataplanes: every dataplane artifact is a molecule, molecules compose sequentially and in parallel within a symmetric monoidal category, and the engineering properties of dataplanes (normalization, minimization, loop convergence, batching, zero allocation, solvability of design problems) are theorems of **Mol**.

The claim rests on three supports. *Compositionality*: **Mol** is symmetric monoidal [Mac Lane,

1971], with tensor the tuple product, unit $()$, and coherence from Mac Lane’s theorem; results about pipelines are results about morphisms. *Completeness*: the catalog Theorem 3.7–Theorem 3.29 covers the category structure (Theorem 3.7–Theorem 3.27, Theorem 3.28, Theorem 3.29), equational theories and rewrite systems (Theorem 4.3–Theorem 5.7, Theorem 5.9), free molecules and normal forms (Theorem 6.4–Theorem 6.9), cost enrichment and optimality (Theorem 7.3–Theorem 7.19, Theorem 7.22), pure/effectful decomposition (Theorem 8.1–Theorem 8.5), minimal state spaces (Theorem 9.6–Theorem 9.9), loops and feedback (Theorem 10.5–Theorem 10.10), batching (Theorem 11.1–Theorem 11.3), zero allocation (Theorem 12.1–Theorem 12.2, Theorem 12.6), design decision problems (Theorem 13.1–Theorem 13.2), and kernel-boundary optimization (Theorem 17.2–Theorem 17.20). *Honesty*: every theorem carries a complete proof; [N] statements are original and proved with elementary means, [C] statements restate classical results with provenance cited; computational claims are verified by executable Rust tools whose logs appear in Appendix A; no conjecture is presented as a theorem.

1.3 Contributions and theorem catalog

The theorems of this thesis are grouped by the chapter that states and proves each. Statements without a citation are original to this thesis; classical results are restated and proved inside Mol with the source cited. Around the theorems stand the framework (chapter 3), which recasts Mealy-style sequential machines [Mealy, 1955, Moore, 1956] as morphisms, the verification of computational claims by executable Rust tools (logs in Appendix A), and a companion software standard that turns the theorems into enforceable engineering policy.

chapter 3 proves the category structure: composition, well-definedness of the state-space quotient, symmetric monoidal structure, the pure and effectful subcategories, bisimulation congruence, tensor closure, and determinism. chapter 4 models rings, parsers, and transports as equational theories. chapter 5 proves termination, confluence, and Knuth–Bendix completion for the rewrite engine. chapter 6 constructs free molecules and proves finiteness of normal forms and their completeness as a lookup table. chapter 7 proves cost optimality by shortest paths and the refinement order. chapter 8 proves the decomposition of every hybrid molecule into pure, effectful, and pure factors, and the traced structure of loops. chapter 9 proves minimal state spaces and compositional bounds. chapter 10 proves contraction bounds for fixed-iteration loops. chapter 11 proves the batching laws. chapter 12 proves cut elimination as pipeline fusion and linearity as a zero-allocation guarantee. chapter 13 proves the finiteness and decidability of design decision problems. chapter 17 proves the kernel-boundary laws, the affine crossover, the optimization lattice, the band contraction of the admission estimate, and the route automaton theorems.

1.4 Scope

This thesis is a synthesis-and-application thesis with deliberately bounded scope; the boundaries are part of the argument.

In scope. Mol and its structure theorems Theorem 3.7–Theorem 3.29, each with a complete proof; the application of the theory to dataplane engineering: normalization, dispatch, batching, minimization, loop analysis, zero-copy reasoning, kernel-boundary cost, and design decision problems; and the verification of every computational claim by executable Rust tools, with PASS logs in Appendix A and catalogs in Appendix B.

Out of scope. This thesis does not construct a transport engine. A companion implementation of TCP, UDP, and SCTP lives in the same repository, in the `Code/` tree, and is not a substitute

for the proofs. Machine-specific performance techniques (socket options, polling strategies, SIMD, syscall catalogs) appear only in the companion standard’s non-normative appendix, not as theorems. A complete machine-checked formalization of the catalog is future work and is not claimed.

Methodological filter. No conjecture is presented as a theorem; any item that cannot be proved as stated is weakened, stated precisely, or moved to future work. [C] statements restate classical results proved inside Mol, with the original cited; [N] statements are original and proved with elementary machinery. Every computational claim is discharged by an executable Rust tool; no argument relies on unverified computation.

1.5 Roadmap

Part I: Foundations (chapter 2–chapter 8). chapter 2 fixes the categorical machinery: monoidal, symmetric monoidal, and traced monoidal categories [Mac Lane, 1971, Joyal et al., 1996]; monads and Kleisli; coalgebras; Lawvere theories [Lawvere, 1963]; operads and clubs [Leinster, 2004, Kelly, 1974]; effectus theory; linear logic [Girard, 1987]; locally presentable categories [Adámek and Rosický, 1994]. chapter 3 defines Mol and its subcategories (Theorem 3.7–Theorem 3.27); chapter 4 models rings, parsers, and transports as Lawvere theories (Theorem 4.3–Theorem 4.8); chapter 5 develops the rewrite engine, with Knuth–Bendix completion [Knuth and Bendix, 1970] and Newman confluence [Newman, 1942] (Theorem 5.1–Theorem 5.7, Theorem 5.9); chapter 6 constructs free molecules and normal forms (Theorem 6.4–Theorem 6.9); chapter 7 proves cost optimality and refinement (Theorem 7.3–Theorem 7.19, Theorem 7.22); chapter 8 proves the decomposition $M = q \circ e \circ p$ (Theorem 8.1–Theorem 8.5).

Part II: Structure theorems (chapter 9–chapter 12). chapter 9: minimal realization and compositional bounds [Nerode, 1958, Hopcroft, 1971] (Theorem 9.6–Theorem 9.9). chapter 10: feedback and fixed-iteration loops, with Banach–Dobrushin contraction bounds [Banach, 1922, Dobrushin, 1956] (Theorem 10.5–Theorem 10.10). chapter 11: operadic composition and batching (Theorem 11.1–Theorem 11.3). chapter 12: cut elimination as fusion, linearity as zero allocation (Theorem 12.1–Theorem 12.2, Theorem 12.6).

Part III: Applications (chapter 13–chapter 16). chapter 13: a situation is a triple $(A \rightarrow B, \Phi, \bar{c})$, a solution a molecule M with $M \vdash \Phi$ and $\text{cost}(M) \leq \bar{c}$ (Theorem 13.1–Theorem 13.2). chapter 14: law tests, bisimulation [Park, 1981], model-based testing, fuzzing, cost regression. chapter 15: related work. chapter 16: summary, limitations, future work. Appendix A: proof-checking scripts and PASS logs; Appendix B: optimization and testing catalogs.

Chapter 2

Preliminaries

This chapter fixes the categorical and algebraic vocabulary used throughout the thesis. Everything stated here is standard: definitions are given, the sources of record are cited, and no proofs are included. The presentation is self-contained in the sense that every notion appealed to in later chapters is defined here or directly cited.

2.1 Categories

Definition 2.1 (Category). A *category* $\mathcal{C}$ consists of

- a collection $\text{Ob}(\mathcal{C})$ of *objects*;
- for every pair of objects A, B , a set $\mathcal{C}(A, B)$ of *morphisms*, the *hom-set*, with $f \in \mathcal{C}(A, B)$ written $f: A \rightarrow B$;
- for every object A , an *identity* morphism $1_A \in \mathcal{C}(A, A)$;
- for every triple of objects A, B, C , a *composition* function

$$\circ: \mathcal{C}(B, C) \times \mathcal{C}(A, B) \rightarrow \mathcal{C}(A, C), \quad (g, f) \mapsto g \circ f,$$

such that composition is *associative*, $(h \circ g) \circ f = h \circ (g \circ f)$, and the *identity laws* hold, $1_B \circ f = f = f \circ 1_A$ Mac Lane [1971].

Definition 2.2 (Isomorphism). A morphism $f: A \rightarrow B$ in a category is an *isomorphism* if there exists a morphism $g: B \rightarrow A$ with $g \circ f = 1_A$ and $f \circ g = 1_B$; the objects A and B are then *isomorphic*, written $A \cong B$ Mac Lane [1971].

Remark 2.3. A category is *small* if its objects and morphisms form sets and *locally small* if every hom-set $\mathcal{C}(A, B)$ is a set; all categories in this thesis are locally small, and the distinction matters only for §2.12 Mac Lane [1971].

Example 2.4. The category **Set** of sets and total functions is the running example. Reading the objects of a typed language as types, the total functions between them form a category; this is the discrete approximation of the category **PureMol** of chapter 3.

2.2 Functors

Definition 2.5 (Functor). A *functor* $F: \mathcal{C} \rightarrow \mathcal{D}$ assigns to each object A of $\mathcal{C}$ an object FA of $\mathcal{D}$ and to each morphism $f: A \rightarrow B$ a morphism $Ff: FA \rightarrow FB$, preserving identities and composition: $F1_A = 1_{FA}$ and $F(g \circ f) = Fg \circ Ff$ Mac Lane [1971].

Definition 2.6 (Full, faithful, essentially surjective). A functor $F: \mathcal{C} \rightarrow \mathcal{D}$ is *full* if $\mathcal{C}(A, B) \rightarrow \mathcal{D}(FA, FB)$ is surjective for all A, B , *faithful* if it is injective for all A, B , and *essentially surjective on objects* if every object of $\mathcal{D}$ is isomorphic to FA for some object A of $\mathcal{C}$ Mac Lane [1971].

Definition 2.7 (Equivalence of categories). An *equivalence* of categories is a functor that is full, faithful, and essentially surjective on objects; we write $\mathcal{C} \simeq \mathcal{D}$ Mac Lane [1971].

2.3 Natural Transformations

Definition 2.8 (Natural transformation). Let $F, G: \mathcal{C} \rightarrow \mathcal{D}$ be functors. A *natural transformation* $\alpha: F \Rightarrow G$ is a family of morphisms $\alpha_A: FA \rightarrow GA$, one for each object A of $\mathcal{C}$, such that for every morphism $f: A \rightarrow B$ the *naturality condition*

$$Gf \circ \alpha_A = \alpha_B \circ Ff$$

holds, i.e. the square built from α_A , α_B , Ff , and Gf commutes Mac Lane [1971].

Definition 2.9 (Natural isomorphism). A natural transformation $\alpha: F \Rightarrow G$ is a *natural isomorphism* if every component α_A is an isomorphism; the functors are then *naturally isomorphic*, $F \cong G$ Mac Lane [1971].

Remark 2.10. Functors and natural transformations organize (small) categories into a category **Cat**. Its isomorphisms are the invertible functors, while the more useful notion of sameness is the equivalence of categories: a functor F is an equivalence exactly when there is a functor G with $G \circ F \cong 1_{\mathcal{C}}$ and $F \circ G \cong 1_{\mathcal{D}}$ (natural isomorphisms). This is the relation asserted between **PureMol** and the category of types and total functions in chapter 3.

2.4 Monoidal and Symmetric Monoidal Categories

Definition 2.11 (Monoidal category). A *monoidal category* is a category $\mathcal{C}$ equipped with

- a *tensor product* bifunctor $\otimes: \mathcal{C} \times \mathcal{C} \rightarrow \mathcal{C}$, written $(A, B) \mapsto A \otimes B$;
- a *unit object* I ;
- natural isomorphisms $\alpha_{A,B,C}: (A \otimes B) \otimes C \rightarrow A \otimes (B \otimes C)$ (the *associator*) and $\lambda_A: I \otimes A \rightarrow A$, $\rho_A: A \otimes I \rightarrow A$ (the *left* and *right unitors*),

such that the *pentagon* and *triangle* coherence diagrams commute; e.g. the pentagon states the equality of the two morphisms $((A \otimes B) \otimes C) \otimes D \rightarrow A \otimes (B \otimes (C \otimes D))$ built from α and the tensor of α with identities Mac Lane [1971].

Definition 2.12 (Strict monoidal category). A monoidal category is *strict* if α , λ , and ρ are identity natural transformations, so that $(A \otimes B) \otimes C = A \otimes (B \otimes C)$ and $I \otimes A = A = A \otimes I$ on the nose.

Definition 2.13 (Symmetric monoidal category). A *symmetric monoidal category* is a monoidal category with a natural isomorphism $\sigma_{A,B}: A \otimes B \rightarrow B \otimes A$ (the *symmetry*) such that $\sigma_{B,A} \circ \sigma_{A,B} = 1_{A \otimes B}$ and σ is coherent with the associator and the unitors Mac Lane [1971].

Remark 2.14 (Coherence). Mac Lane’s coherence theorem states that every diagram whose edges are built from α , λ , ρ , and σ commutes; hence no information is lost in treating $A \otimes (B \otimes C)$ and $(A \otimes B) \otimes C$ as interchangeable. Consequently every monoidal category is monoidally equivalent to a strict one (Mac Lane’s strictification), a fact used implicitly throughout Mac Lane [1971]. The tensor product of chapter 3 is tuple formation with unit $()$, which is strict and symmetric.

2.5 Traced Monoidal Categories

Definition 2.15 (Traced monoidal category). Let $\mathcal{C}$ be a symmetric monoidal category. A *trace* is a family of operations

$$\text{Tr}_{A,B}^U: \mathcal{C}(A \otimes U, B \otimes U) \rightarrow \mathcal{C}(A, B),$$

natural in A and B and dinatural in the *feedback object* U , satisfying the following axioms Joyal et al. [1996], Hu and Vicary [2021]:

- *Vanishing*: $\text{Tr}_{A,B}^I(f) = f$ for $f: A \otimes I \rightarrow B \otimes I$, and $\text{Tr}_{A,B}^{U \otimes V} = \text{Tr}_{A,B}^U \circ \text{Tr}_{A \otimes U, B \otimes U}^V$;
- *Superposing*: for $f: A \otimes U \rightarrow B \otimes U$ and $g: C \rightarrow D$,

$$\text{Tr}_{A \otimes C, B \otimes D}^U(f \otimes g) = \text{Tr}_{A,B}^U(f) \otimes g;$$

- *Yanking*: $\text{Tr}_{U,U}^U(\sigma_{U,U}) = 1_U$.

A *traced monoidal category* is a symmetric monoidal category equipped with a trace. The trace internalizes feedback: a morphism $f: A \otimes U \rightarrow B \otimes U$ computes a transformation $A \rightarrow B$ while threading the feedback object U through the loop. This is the structure underlying the fixed-iteration loops of chapter 10.

2.6 Monads, Algebras over Monads, and Kleisli Categories

Definition 2.16 (Monad). A *monad* on a category $\mathcal{C}$ is a triple (T, η, μ) where $T: \mathcal{C} \rightarrow \mathcal{C}$ is a functor and $\eta: 1_{\mathcal{C}} \Rightarrow T$ (the *unit*) and $\mu: T^2 \Rightarrow T$ (the *multiplication*) are natural transformations satisfying the *unit laws* $\mu \circ T\eta = 1_T = \mu \circ \eta T$ and the *associativity law* $\mu \circ T\mu = \mu \circ \mu T$ Mac Lane [1971], Métayer [2004].

Definition 2.17 (Algebra over a monad). Let (T, η, μ) be a monad on $\mathcal{C}$. A *T-algebra* (an *Eilenberg–Moore algebra*) is a pair (A, a) consisting of an object A and a morphism $a: TA \rightarrow A$ such that $a \circ \eta_A = 1_A$ and $a \circ \mu_A = a \circ Ta$. A *homomorphism* of T -algebras $(A, a) \rightarrow (B, b)$ is a morphism $f: A \rightarrow B$ with $f \circ a = b \circ Tf$. T -algebras and their homomorphisms form the category $\mathcal{C}^T$ Mac Lane [1971].

Definition 2.18 (Kleisli category). The *Kleisli category* $\mathcal{C}_T$ of a monad (T, η, μ) on $\mathcal{C}$ has the same objects as $\mathcal{C}$; a morphism $A \rightarrow B$ in $\mathcal{C}_T$ is a morphism $f: A \rightarrow TB$ in $\mathcal{C}$. The identity on A is η_A , and the composition of $f: A \rightarrow TB$ with $g: B \rightarrow TC$ is

$$g \circ_T f = \mu_C \circ Tg \circ f: A \rightarrow TC,$$

the *bind* operation of the monad Mac Lane [1971], Métayer [2004].

Example 2.19 (State monad). Let $\mathcal{C}$ have finite products and fix an object S of *states*. The *state monad* is the monad with $TX = (X \times S)^S$, unit $\eta_X: x \mapsto \lambda s. (x, s)$, and multiplication given by threading the state through two successive computations. A Kleisli morphism $A \rightarrow B$ is then a morphism $A \times S \rightarrow B \times S$, exactly the transition function of a deterministic Mealy machine with state space S Mac Lane [1971], Métayer [2004], Mealy [1955], Moore [1956], Bonchi et al. [2024]. The Kleisli category of this monad models **EffMol**(Ctx) in chapter 3, and the pure/effectful decomposition of chapter 8 is phrased in this language.

Remark 2.20. Every monad arises from an adjunction; the Kleisli category is the initial and $\mathcal{C}^T$ the terminal resolution of the monad as a pair of adjoint functors Mac Lane [1971], Métayer [2004].

2.7 Coalgebras

Definition 2.21 (Coalgebra for an endofunctor). Let $F: \mathcal{C} \rightarrow \mathcal{C}$ be an endofunctor. An *F-coalgebra* is a pair (S, s) consisting of an object S and a morphism $s: S \rightarrow FS$. A *homomorphism* of *F-coalgebras* $(S, s) \rightarrow (T, t)$ is a morphism $f: S \rightarrow T$ with $t \circ f = Ff \circ s$. Coalgebras and their homomorphisms form the category **Coalg**(F) Colcombet and Petrişan [2017].

Definition 2.22 (Final coalgebra). A *final* (terminal) coalgebra is a terminal object of **Coalg**(F): a coalgebra (Z, ζ) such that for every coalgebra (S, s) there is a unique homomorphism $(S, s) \rightarrow (Z, \zeta)$ Colcombet and Petrişan [2017].

Definition 2.23 (Behavioral equivalence). Two states $s_1 \in S_1$, $s_2 \in S_2$ of *F-coalgebras* are *behaviorally equivalent* if there is a coalgebra (T, t) and homomorphisms from (S_1, s_1) and (S_2, s_2) into it identifying them; when F preserves weak pullbacks this coincides with *bisimilarity* in the sense of Park [1981], Colcombet and Petrişan [2017].

Example 2.24 (Mealy machines as coalgebras). Fix sets A and B of inputs and outputs. A deterministic Mealy machine (S, step) with $\text{step}: S \times A \rightarrow B \times S$ is exactly a coalgebra for the functor $FX = (B \times X)^A$ Mealy [1955], Moore [1956], Bonchi et al. [2024]. Behavioral equivalence is then language equivalence, and the quotient constructions of chapter 9 refine the classical Nerode and Hopcroft minimization algorithms Nerode [1958], Hopcroft [1971], Colcombet and Petrişan [2017].

2.8 Lawvere Theories

Definition 2.25 (Lawvere theory). Let **Fin**^{op} be the category whose objects are the natural numbers and whose morphisms $m \rightarrow n$ are the functions $n \rightarrow m$. A *Lawvere theory* is a category $\mathcal{L}$ with finite products together with an identity-on-objects, strict, finite-product-preserving functor **Fin**^{op} $\rightarrow \mathcal{L}$. The object 1 is the *generic object*: every object of $\mathcal{L}$ is a finite product of copies of 1 Lawvere [1963], Lack and Rosický [2008].

Definition 2.26 (Model of a Lawvere theory). A *model* of a Lawvere theory $\mathcal{L}$ in a category $\mathcal{C}$ with finite products is a finite-product-preserving functor $\mathcal{L} \rightarrow \mathcal{C}$. Models for $\mathcal{C} = \mathbf{Set}$ are the *algebras* of the theory, and models and natural transformations form a category **Mod**($\mathcal{L}, \mathcal{C}$) Lawvere [1963], Lack and Rosický [2008].

Remark 2.27. The *free algebra* on n generators is the model represented by object n : the finite-product-preserving functor $\mathcal{L}(n, -)$ with values in **Set**, whose underlying set is the set of terms in

n variables modulo the equations of the theory Lawvere [1963]. A finitary equational theory in the sense of universal algebra (Section 2.13) is the same data as a Lawvere theory, and its variety of algebras is $\mathbf{Mod}(\mathcal{L}, \mathbf{Set})$. The equations of the dataplane component theories of chapter 4 (ring, parser, transport) are Lawvere theories whose models are the corresponding molecules.

2.9 Operads, Multicategories, and Clubs

Definition 2.28 (Symmetric operad). A *symmetric operad* P (in $\mathbf{Set}$) consists of a set $P(n)$ of n -ary operations for each $n \geq 0$, an action of the symmetric group Σ_n on $P(n)$, a distinguished element $1 \in P(1)$ (the *identity operation*), and composition maps

$$P(k) \times P(n_1) \times \cdots \times P(n_k) \rightarrow P(n_1 + \cdots + n_k),$$

that are associative, unital with respect to 1, and equivariant with respect to the symmetric group actions Leinster [2004], Weber [2014].

Definition 2.29 (Algebra over an operad). An *algebra* over a symmetric operad P is a set A together with, for each $p \in P(n)$, an operation $A^n \rightarrow A$, respecting the identity, the composition maps, and the symmetric group action Leinster [2004], Weber [2014].

Definition 2.30 (Multicategory). A *multicategory* is a category-like structure in which a morphism has a finite *sequence* of objects as domain: a morphism $(A_1, \dots, A_n) \rightarrow B$, with identity morphisms $1_A: (A) \rightarrow A$ and an associative *substitution* composition that is unital. A *symmetric* multicategory carries a compatible action of Σ_n on morphisms with n -ary domain Leinster [2004], Weber [2014].

Remark 2.31. An operad is precisely a one-object symmetric multicategory: its morphisms whose domain is a list of copies of the single object are the operations, and substitution is operadic composition. Kelly’s *clubs* generalize both: a club is a category over the category of finite sets (or of permutations) whose substitution structure presents the free algebras of a 2-monad, covering many doctrines of categories with structure, including the symmetric monoidal ones at work in chapter 11 and chapter 5 Kelly [1974], Leinster [2004].

2.10 Effectus Theory

Effectus theory formalizes partiality, probabilistic choice, and measurement in one categorical framework, axiomatically oriented around the *effects* (observables) of a system.

Definition 2.32 (Effect algebra). An *effect algebra* is a set E with a partial binary operation $\oplus$ and distinguished elements $0 \neq 1$, such that $\oplus$ is commutative and associative wherever defined, $a \oplus 0 = a$ for all a , every element a has a unique *orthosupplement* $a^\perp$ with $a \oplus a^\perp = 1$, and $a \oplus 1$ is defined only for $a = 0$ Cho et al. [2015].

Definition 2.33 (Effectus). An *effectus* is a category with finite coproducts and a terminal object 1, admitting the pushouts of coproduct injections that are needed to compose *states* and *effects*, such that for every object X the hom-set of *effects* $X \rightarrow 1$ carries an effect algebra structure and composition induces effect-algebra homomorphisms Cho et al. [2015].

Remark 2.34. Effects generalize predicates: on a measurable space the effects are the $[0, 1]$ -valued measurable functions with $\oplus$ truncated addition. The framework supplies one axiomatics for partiality and probabilistic choice, relevant to the error and context channels of the effectful molecules of chapter 3; a closely related categorical treatment of probability is developed in the theory of Markov categories Cho et al. [2015], Fritz [2019].

2.11 Linear Logic and Geometry of Interaction

Definition 2.35 (Linear logic). *Linear logic* is a substructural logic in which every hypothesis is used exactly once. It is presented by sequent rules for the multiplicative connectives $\otimes$ (tensor) and $\multimap$ (linear implication), the additive connectives $\&$ and $\oplus$, and the *exponentials* $!$ and $?$ that govern contraction and weakening, i.e. unlimited use of a hypothesis Girard [1987].

Remark 2.36 (Cut elimination and geometry of interaction). The *cut rule* composes proofs, and *cut elimination* rewrites a proof to a cut-free normal form, which is deterministic and is the computational content of the proof. The *geometry of interaction* (GoI) interprets proofs as morphisms of a traced monoidal category and cut elimination as the trace: the *Int* construction embeds every traced monoidal category into a compact closed one, where the trace becomes genuine composition Girard [1987], Joyal et al. [1996], Seiller [2015]. Deforestation and the zero-allocation fusion of chapter 12 are the thesis’s instance of this correspondence.

2.12 Accessible and Locally Presentable Categories

Definition 2.37 (Presentable objects, accessible categories). Let κ be a regular cardinal. An object A of a category $\mathcal{C}$ is κ -*presentable* if the hom-functor $\mathcal{C}(A, -)$ preserves κ -filtered colimits. The category $\mathcal{C}$ is κ -*accessible* if it has κ -filtered colimits and there is a set of κ -presentable objects such that every object of $\mathcal{C}$ is a κ -filtered colimit of objects from that set; $\mathcal{C}$ is *accessible* if it is κ -accessible for some regular cardinal κ Adámek and Rosický [1994].

Definition 2.38 (Locally presentable categories). A category is *locally κ -presentable* if it is κ -accessible and cocomplete, and *locally presentable* if it is locally κ -presentable for some regular cardinal κ Adámek and Rosický [1994].

Remark 2.39. Every locally presentable category is complete and cocomplete, and the categories of models of Lawvere theories (Section 2.8) and the categories of algebras of many signatures are locally presentable. Local presentability is the standard tool for proving the existence of free structures, such as the free symmetric monoidal category $F(\Sigma)$ whose existence chapter 6 invokes Adámek and Rosický [1994], Lawvere [1963].

2.13 Universal Algebra and Clones

Definition 2.40 (Signature and algebra). A *signature* is a set Σ of operation symbols together with an assignment of a finite *arity* to each symbol. A Σ -*algebra* is a set A together with, for each n -ary symbol $\sigma \in \Sigma$, an operation $\sigma_A: A^n \rightarrow A$ Knuth and Bendix [1970].

Definition 2.41 (Terms, equations, varieties). Given a signature Σ and a set X of variables, the *terms* over X are formed from variables and operation symbols. An *equation* is a pair of terms $s \approx t$; a Σ -algebra *satisfies* the equation if the two terms denote equal operations under every assignment of values to variables. A class of algebras closed under satisfaction of a set of equations is a *variety* Lawvere [1963], Knuth and Bendix [1970].

Definition 2.42 (Clone). A *clone* on a set A is a set C of finitary operations on A that contains all projections and is closed under composition. The clone of a Σ -algebra is the set of its *term operations* Lawvere [1963].

Remark 2.43. A clone on A is the concrete shadow of a Lawvere theory: it is exactly the data of a model of an algebraic theory with carrier A (Section 2.8). The *word problem* for a variety asks whether two terms denote the same operation in the free algebra; the Knuth–Bendix completion procedure decides it whenever it terminates, by producing a terminating and confluent rewriting system, and Newman’s lemma reduces confluence to local confluence Knuth and Bendix [1970], Newman [1942], Epstein and Sanders [1998]. By the classical theorem of Birkhoff [Birkhoff, 1935], the varieties are precisely the classes of algebras closed under homomorphic images, subalgebras, and products. The equational theory of the runtime is treated by these means in chapter 5.

Notion	Defined in	Applied in
Categories, functors, natural transformations	§2.1	chapter 3
Monoidal and symmetric monoidal categories	§2.4	chapter 3, chapter 6
Traced monoidal categories	§2.5	chapter 10
Monads, algebras, Kleisli categories	§2.6	chapter 8, chapter 3
Coalgebras	§2.7	chapter 9
Lawvere theories	§2.8	chapter 4
Operads, multicategories, clubs	§2.9	chapter 11, chapter 5
Effectus theory	§2.10	chapter 3
Linear logic and geometry of interaction	§2.11	chapter 12
Accessible and locally presentable categories	§2.12	chapter 6
Universal algebra and clones	§2.13	chapter 5

Table 2.1: The categorical tools of this chapter and where the thesis applies them.

2.14 Notation and acronyms

All acronyms are defined here and at their first use.

Acronyms.

- **SMC**: symmetric monoidal category (section 2.4).
- **DFA**: deterministic finite automaton.
- **EMA**: exponentially weighted moving average (the scheduler estimate of section 17.7).
- **IPC**: instructions per cycle, a processor counter.
- **TLB**: translation lookaside buffer; a dTLB is the data TLB.
- **RSS**: resident set size, the physical memory a process occupies.
- **L1, L3**: first- and last-level processor caches.
- **TCP, UDP, SCTP**: the Transmission Control Protocol, the User Datagram Protocol, and the Stream Control Transmission Protocol.
- **HTTP**: the Hypertext Transfer Protocol.
- **NIC**: network interface controller.

- **DNS**: the Domain Name System.
- **FTP**: the File Transfer Protocol.
- **LL(1)**: left-to-right, leftmost-derivation parsing with one token of lookahead.
- **RFC**: Request for Comments, the standards document series of the Internet Engineering Task Force.
- **SQPOLL**: submission-queue polling, an `io_uring` mode in which the kernel polls the submission queue and the application skips the `enter-system-call`.
- **TLS**: Transport Layer Security.
- **QUIC**: the connection-oriented transport protocol used by HTTP/3.
- **SIMD**: single instruction, multiple data.
- **KB**: Knuth–Bendix completion (chapter 5).
- **TTL**: time to live, a cache validity window.
- **AF_XDP**: the kernel’s eXpress Data Path socket family.
- **io_uring**: the kernel’s asynchronous completion-ring interface.
- **MSG_ZEROCOPY**: the socket flag for zero-copy sends.
- **PGO**: profile-guided optimization.

Notation. $\circ$ is sequential composition and $\otimes$ is parallel composition (tensor); id_A is the identity on the object A ; $\simeq$ is behavioral equivalence; $\beta(M)$ is the boundary count of Definition 17.1; $c(M)$ is the cost vector of Definition 7.1; $\text{NF}(M)$ is the normal form of M .

Chapter 3

The Category **Mol**

This chapter builds the algebraic foundation of the thesis. The objects of the category **Mol** are types; the morphisms are *stateful transformations*: deterministic Mealy machines (S, step) with $\text{step} : S \times A \rightarrow B \times S$, quotiented by the condition that two machines are equal exactly when their state spaces are in bijection and the bijection carries one transition structure to the other. State is then an implementation detail: a morphism is an input–output behavior together with its internal state space, not a particular machine. On this we establish the category structure (Theorem 3.7–Theorem 3.8), the symmetric monoidal structure (Theorem 3.12), the subcategories of pure, effectful, and hybrid molecules (Theorem 3.14–Theorem 3.17), behavioral equivalence as a congruence (Theorem 3.23), closure under the tensor (Theorem 3.24), and determinism (Theorem 3.27). Throughout, types are the types of the ambient universe of chapter 2, $A \times B$ is the product type, and $()$ is the unit type.

3.1 Machines and Morphisms

Definition 3.1 (Machine). Let A and B be types. A *machine* from A to B is a pair (S, step) consisting of a type S , the *state space*, and a total function

$$\text{step} : S \times A \rightarrow B \times S.$$

A machine is a deterministic Mealy machine: on input a in state s it produces output b and next state s_1 where $\text{step}(s, a) = (b, s_1)$ [Mealy, 1955, Moore, 1956]. Determinism is built in: step is a function, so from a given state and input exactly one output and one next state result.

Definition 3.2 (State-space isomorphism). Two machines (S, step) and (S', step') from A to B are *state-space isomorphic*, written $(S, \text{step}) \cong (S', \text{step}')$, if there exists a bijection $\varphi : S \rightarrow S'$ such that for all $s \in S$ and $a \in A$,

$$\text{step}(s, a) = (b, s_1) \implies \text{step}'(\varphi(s), a) = (b, \varphi(s_1)).$$

Remark 3.3. State-space isomorphism is an equivalence relation: reflexive with $\varphi = \text{id}_S$, symmetric with φ^{-1} , transitive by composing bijections. Isomorphic machines differ only in the names of their states: for every input they emit the same outputs and traverse matching states. This is the *bijection quotient* announced in chapter 1: state is unobservable in **Mol** except through its type.

Definition 3.4 (The category **Mol**). The category **Mol** has the types as objects. For types A and B , a morphism $f : A \rightarrow B$ of **Mol** is an equivalence class $[S, \text{step}]$ of machines from A to B under

state-space isomorphism. Composition and identity are defined in Theorem 3.5 and Theorem 3.6; that they are well defined on classes and satisfy the category laws is the content of Theorem 3.7 and Theorem 3.8.

Definition 3.5 (Composition). Let $f = [S, \text{step}] : A \rightarrow B$ and $g = [T, \text{tstep}] : B \rightarrow C$ be morphisms of Mol. Their *composite* is the class

$$g \circ f = [T \times S, \text{comp}] : A \rightarrow C,$$

where $\text{comp} : (T \times S) \times A \rightarrow C \times (T \times S)$ is given by

$$\text{comp}((t, s), a) = (c, (t_1, s_1)) \quad \text{whenever} \quad (b, s_1) = \text{step}(s, a), (c, t_1) = \text{tstep}(t, b).$$

The composite machine runs f on the input, feeds f 's output and the T -component of the state to g , and records both next states; the state space of a composite is the product of the state spaces. That the class $g \circ f$ is independent of the chosen representatives is Theorem 3.8.

Definition 3.6 (Identity). For each type A , the *identity* morphism is

$$\text{id}_A = [(), \text{idstep}] : A \rightarrow A, \quad \text{idstep}((), a) = (a, ()).$$

The identity machine has the unit type as state space and copies its input to the output.

Theorem 3.7 (Mol is a category (Theorem 3.7)). *[N]. Composition of (S, step) classes is associative with identity: Mol is a category.*

Proof. Composition on classes is well defined by Theorem 3.8, proved below; given representatives, the composite class is computed as in Theorem 3.5. We verify the category laws on representatives; the quotient inherits them.

Associativity. Let $f : A \rightarrow B$, $g : B \rightarrow C$, $h : C \rightarrow D$ be realized by (S, step) , (T, tstep) , (R, rstep) . By Theorem 3.5, $(h \circ g) \circ f$ and $h \circ (g \circ f)$ are realized by the machines with state spaces $(R \times T) \times S$ and $R \times (T \times S)$. Unwinding the definition, both machines process an input a by running f , then g , then h in that order, emitting h 's output and recording all three next states; the two machines differ only in how the state triple is bracketed. Formally, the canonical bijection

$$\varphi : (R \times T) \times S \rightarrow R \times (T \times S), \quad \varphi((r, t), s) = (r, (t, s)),$$

is a state-space isomorphism: if the first machine takes $x = (r, t), s$ on input a to $(d, x_1) = (d, (r_1, t_1), s_1))$, then the second takes $\varphi(x) = (r, (t, s))$ on a to $(d, (r_1, (t_1, s_1))) = (d, \varphi(x_1))$, because both machines run the same three components in the same order on the same data. Hence $(h \circ g) \circ f = h \circ (g \circ f)$ in Mol.

Identity laws. For $f = [S, \text{step}] : A \rightarrow B$, the composite $\text{id}_B \circ f$ is realized by $(() \times S, \text{comp})$, which runs f and then copies the output, taking $((), s)$ on input a to $(b, ((), s_1))$ when $\text{step}(s, a) = (b, s_1)$. The bijection $\varphi((), s) = s$ is a state-space isomorphism onto f , since $\text{step}(\varphi((), s), a) = \text{step}(s, a) = (b, s_1) = (b, \varphi((), s_1))$; so $\text{id}_B \circ f = f$. Symmetrically, $f \circ \text{id}_A = f$ via $\varphi(s, (()) = s$. Both laws therefore hold on classes. $\square$

Theorem 3.8 (The bijection quotient is well defined (Theorem 3.8)). *[N]. The state-space bijection quotient is well defined: equivalent machines compose to equivalent machines. If $(S, \text{step}) \cong (S_1, \text{step}_1)$ and $(T, \text{tstep}) \cong (T_1, \text{tstep}_1)$, then*

$$(T, \text{tstep}) \circ (S, \text{step}) \cong (T_1, \text{tstep}_1) \circ (S_1, \text{step}_1),$$

so the composite class $[g \circ f]$ is independent of the representatives.

Proof. Let $\varphi : S \rightarrow S_1$ and $\psi : T \rightarrow T_1$ be the state-space isomorphisms, so that $\text{step}_1(\varphi(s), a) = (b, \varphi(s_1))$ whenever $\text{step}(s, a) = (b, s_1)$, and likewise for ψ . The two composites are realized by $(T \times S, \text{comp})$ and $(T_1 \times S_1, \text{comp}_1)$ with the transition law of Theorem 3.5. Define $\Phi : T \times S \rightarrow T_1 \times S_1$ by $\Phi(t, s) = (\psi(t), \varphi(s))$; Φ is a bijection. Suppose $\text{comp}((t, s), a) = (c, (t_1, s_1))$, that is, $(b, s_1) = \text{step}(s, a)$ and $(c, t_1) = \text{tstep}(t, b)$. Then

$$\text{comp}_1(\Phi(t, s), a) = \text{comp}_1(\psi(t), \varphi(s), a) = (c, (\psi(t_1), \varphi(s_1))) = (c, \Phi(t_1, s_1)),$$

where the middle equality applies the isomorphism hypotheses to the two component steps. Hence Φ is a state-space isomorphism of the composites, which is the claim. $\square$

Remark 3.9. With Theorem 3.8 in place, composition is a genuine function on the hom-sets of Mol, and Theorem 3.7 makes Mol a category. The same product-of-isomorphisms argument shows that the tensor product of the next section is well defined on classes.

3.2 Symmetric Monoidal Structure

The dataplane composes pipelines in two ways: sequentially, by $\circ$, and in parallel, by a tensor product that pairs independent subpipelines. For Mol the tensor is the tuple product and the unit is $()$.

Definition 3.10 (Tensor product). On objects, $A \otimes B = A \times B$, and the unit object is $I = ()$. On morphisms, for $f = [S, \text{step}] : A \rightarrow B$ and $g = [T, \text{tstep}] : C \rightarrow D$,

$$f \otimes g = [S \times T, \text{step} \otimes \text{tstep}] : A \times C \rightarrow B \times D,$$

where

$$(\text{step} \otimes \text{tstep})((s, t), (a, c)) = ((b, d), (s_1, t_1)) \quad \text{whenever} \quad (b, s_1) = \text{step}(s, a), (d, t_1) = \text{tstep}(t, c).$$

The two machines run independently, one on each component of the input, and the state space of a tensor is the product of the state spaces.

Definition 3.11 (Braiding). For types A and B , the *braiding* is the pure morphism

$$\sigma_{A,B} = [(), \text{swap}] : A \times B \rightarrow B \times A, \quad \text{swap}((a, b)) = ((b, a), ()).$$

Theorem 3.12 (Mol is symmetric monoidal (Theorem 3.12)). *[C]. Mol is symmetric monoidal: the tensor product of Theorem 3.10 with unit $()$ is a bifunctor; the associator, the unitors, and the braiding of Theorem 3.11 are the canonical pure morphisms; and they satisfy the coherence conditions of Mac Lane [Mac Lane, 1971].*

Proof. Well definedness. If $\varphi : S \rightarrow S_1$ and $\psi : T \rightarrow T_1$ are state-space isomorphisms of f with f_1 and of g with g_1 , then $\Phi(s, t) = (\varphi(s), \psi(t))$ is a state-space isomorphism of $f \otimes g$ with $f_1 \otimes g_1$: the square commutes componentwise by the two isomorphism hypotheses, exactly as in the proof of Theorem 3.8. The tensor therefore descends to classes.

Bifunctoriality. The tensor of identity machines $((), \text{idstep}) \otimes ((), \text{idstep})$ is realized by $() \times ()$ with the copy transition, the class $\text{id}_{A \times B}$. It remains to check the interchange law: for $f : A \rightarrow B$, $f' : B \rightarrow C$, $g : A' \rightarrow B'$, $g' : B' \rightarrow C'$ with realizations (S, step) , (S', step') , (T, tstep) , (T', tstep') ,

$$(f \otimes g) \circ (f' \otimes g') = (f \circ f') \otimes (g \circ g').$$

The left-hand side is realized by the state space $(S' \times T') \times (S \times T)$: it runs f' and g' on the input pair, then f and g on the outputs. The right-hand side is realized by $(S \times S') \times (T \times T')$: it runs f' then f , and g' then g . Both machines apply the four transitions f' , g' , f , g exactly once each, in that order, to the corresponding components, and the canonical bijection

$$\Phi((s', t'), (s, t)) = ((s, s'), (t, t'))$$

carries the state after the four transitions of the first machine to the state after the four transitions of the second; the outputs coincide componentwise. Hence Φ is a state-space isomorphism and the interchange law holds, so $\otimes$ is a bifunctor [Mac Lane, 1971].

Naturality of the braiding. For $f : A \rightarrow A'$ and $g : B \rightarrow B'$ with realizations (S, step) , (T, tstep) , both $(g \otimes f) \circ \sigma_{A,B}$ and $\sigma_{A',B'} \circ (f \otimes g)$ run f on the first component and g on the second, then swap the outputs; their state spaces are $T \times S$ and $S \times T$, matched by the bijection $\Phi(t, s) = (s, t)$, which commutes with the transitions because the swap step is pure. Hence σ is natural in both arguments.

Associator and unitors. The associator $\alpha_{A,B,C} : (A \times B) \times C \rightarrow A \times (B \times C)$ is the pure machine that reassociates the tuple, $\alpha((a, b), c) = (a, (b, c))$; the left and right unitors $\lambda_A : () \times A \rightarrow A$ and $\rho_A : A \times () \rightarrow A$ are the pure projection machines, $\lambda((a), a) = (a, a)$ and $\rho(a, ()) = (a, a)$. Each is an isomorphism in Mol, its inverse being the pure machine running the same tuple manipulation backwards.

Coherence. The pentagon, triangle, and hexagon diagrams commute because both paths through each diagram compute the same function on tuples. In the pentagon, both composite paths reassociate $((A \otimes B) \otimes C) \otimes D$ to $A \otimes (B \otimes (C \otimes D))$, and reassociation of tuples is unique. In the triangle, both paths project $((A \otimes I) \otimes B)$ to $A \otimes B$; in the hexagon, both sides of

$$(1_B \otimes \sigma_{A,C}) \circ (\sigma_{A,B} \otimes 1_C) = \sigma_{A,B \otimes C}$$

map (a, b, c) to (b, c, a) , since the left-hand side first swaps a and b and then swaps a and c , and the right-hand side swaps the single A -component past the pair $B \otimes C$. Similarly $\sigma_{B,A} \circ \sigma_{A,B} = \text{id}_{A \otimes B}$, since two swaps return the tuple. Hence σ is a coherent symmetry. By Mac Lane's coherence theorem every diagram whose edges are built from α , λ , ρ , and σ commutes, so Mol is symmetric monoidal: and, since composition is by Theorem 3.8, the coherence equations hold on classes [Mac Lane, 1971]. This is the structure on which the free constructions of chapter 6 and the traced structure of chapter 10 are built. $\square$

3.3 Pure Molecules

A molecule with no state at all computes a total function. The subcategory these form is the functional fragment of Mol.

Definition 3.13 (PureMol). **PureMol** is the subcategory of Mol with the same objects and with morphisms the classes of machines representable with state space $()$: a morphism $f : A \rightarrow B$ of PureMol is a class $[(\text{step})]$. The quotient is trivial on pure machines: because the only bijection $() \rightarrow ()$ is the identity, two pure classes are equal exactly when their step functions agree pointwise.

Theorem 3.14 (PureMol is the category of functions (Theorem 3.14)). **[N]**. *PureMol is equivalent to the category **Type** of types and total functions between them; in fact the two categories are isomorphic.*

Proof. Define $E : \text{PureMol} \rightarrow \mathbf{Type}$ as the identity on objects and, on morphisms, $E[(\cdot), \text{step}] = f$ where $\text{step}((\cdot), a) = (f(a), (\cdot))$. This is well defined by Theorem 3.2: two pure machines are state-space isomorphic exactly when their step functions agree pointwise. It is a functor: by Theorem 3.5 the composite of the pure machines for f and g runs f then g and hence represents $g \circ f$, and the identity machine represents id_A . E is full: every total function $f : A \rightarrow B$ is realized by the machine $((\cdot), \text{step}_f)$ with $\text{step}_f((\cdot), a) = (f(a), (\cdot))$. E is faithful: if $E[(\cdot), \text{step}] = E[(\cdot), \text{step}']$, the step functions agree, so the classes are equal. And E is bijective on objects. Hence E is an equivalence of categories [Mac Lane, 1971]; since the inverse on hom-sets sends f to $[(\cdot), \text{step}_f]$ and is the identity on objects, the equivalence is an isomorphism of categories. PureMol is the type theoretic shadow of Mol itself: objects as types, morphisms as functions. $\square$

3.4 Effectful Molecules and the Kleisli Category

A dataplane carries a fixed ambient *context* Ctx: the packet, the workspace, the request: through every pipeline stage. An effectful molecule is a machine whose only state is the context; composing effectful molecules threads a single context through both, which is exactly the bind of the state monad.

Definition 3.15 (EffMol). Fix a type Ctx of contexts. **EffMol**(Ctx) is the category with the same objects as Mol whose morphisms $A \rightarrow B$ are the step functions $\text{step} : A \times \text{Ctx} \rightarrow B \times \text{Ctx}$. These are machines with state space Ctx: compared by literal equality of functions. The identity on A is $\text{id}(a, s) = (a, s)$, and the composite of $\text{step} : A \times \text{Ctx} \rightarrow B \times \text{Ctx}$ with $\text{tstep} : B \times \text{Ctx} \rightarrow C \times \text{Ctx}$ is the *context-threading* function

$$(\text{tstep} \circ \text{step})(a, s) = (c, s_2) \quad \text{where} \quad (b, s_1) = \text{step}(a, s), (c, s_2) = \text{tstep}(b, s_1).$$

Both machines see the same single context, threaded from first to second.

Remark 3.16. Every step function of EffMol(Ctx) is a machine with state space Ctx and hence determines a class [Ctx, step] of Mol; but EffMol is not a subcategory of Mol under inherited composition. The Mol-composite of two context machines has state space $\text{Ctx} \times \text{Ctx}$ (the contexts multiply) while the EffMol-composite threads one copy; and the EffMol identity is the class of a machine that is the Mol identity only when Ctx is a singleton. EffMol is the model of the effectful fragment; its relation to Mol is developed in chapter 8.

Theorem 3.17 (EffMol is the Kleisli category of the state monad (Theorem 3.17)). *[N]. EffMol(Ctx) is isomorphic to the Kleisli category of the state monad on Ctx.*

Proof. Recall the state monad of chapter 2: on the category of types, $TX = (X \times \text{Ctx})^{\text{Ctx}}$ with unit $\eta_X(x) = \lambda s. (x, s)$ and multiplication threading two state computations. Both EffMol(Ctx) and the Kleisli category $\text{Kl}(T)$ have the types as objects. A Kleisli morphism $A \rightarrow B$ is a function $A \rightarrow TB = (B \times \text{Ctx})^{\text{Ctx}}$, which by currying is exactly a function $A \times \text{Ctx} \rightarrow B \times \text{Ctx}$, i.e. a morphism of EffMol(Ctx); this bijection F on hom-sets is the identity on objects. The Kleisli identity is η_A , which under currying is the EffMol identity $(a, s) \mapsto (a, s)$. Kleisli composition is the bind $g \circ_T f = \mu_C \circ Tg \circ f$, which uncurries to the context-threading law

$$(g \circ_T f)(a, s) = (c, s_2) \quad \text{where} \quad (b, s_1) = f(a, s), (c, s_2) = g(b, s_1),$$

matching Theorem 3.15 exactly. Hence F preserves identity and composition; the inverse construction sends each step function to its curried form. F is therefore an isomorphism of categories [Mac Lane, 1971, Métayer, 2004]. The parenthetical “algebras over the monad” of the catalog refers to the Eilenberg–Moore category, the other resolution of the same monad; the isomorphism here is with the Kleisli category, which is what the dataplane threads. $\square$

3.5 Hybrid Molecules

Real dataplane stages carry both an explicit pure state: a counter, a checksum register, a ring pointer: and the ambient context. A hybrid molecule is a machine whose state space splits as a product of a pure part and the context.

Definition 3.18 (HybridMol). Fix the context type Ctx . **HybridMol**(Ctx) is the subcategory of Mol with the same objects and with morphisms the classes of machines (S, step) whose state space S is a singleton or is isomorphic to $S_{\text{pure}} \times \text{Ctx}$ for some type S_{pure} . The singleton clause includes the pure identity and with it all pure morphisms, which the identity laws of Theorem 3.7 require. Equivalently, a hybrid molecule is a machine with state space $S_{\text{pure}} \times \text{Ctx}$: a pure state component S_{pure} that the stage owns and a context component Ctx that it shares with the ambient pipeline.

Remark 3.19. HybridMol is closed under composition and tensor. The composite of hybrid machines with pure parts S_{pure} and T_{pure} is realized by

$$(T_{\text{pure}} \times \text{Ctx}) \times (S_{\text{pure}} \times \text{Ctx}) \cong (T_{\text{pure}} \times S_{\text{pure}} \times \text{Ctx}) \times \text{Ctx},$$

again of the form (pure part) $\times$ (context), the canonical isomorphism being a state-space isomorphism by the product case of Theorem 3.8; the tensor closes by the same computation, absorbing the extra context copy into the pure part, as recorded in Theorem 3.24. EffMol(Ctx) sits inside HybridMol as the case $S_{\text{pure}} = ()$. The decomposition of a hybrid molecule as $M = q \circ e \circ p$ with p, q pure and e effectful is proved in chapter 8.

3.6 Behavioral Equivalence

Two machines may differ in their state spaces yet compute the same transformation on input words. Behavioral equivalence identifies exactly those machines, and it is the relation under which Mol is quotiented in the minimization and testing chapters.

Definition 3.20 (Response). For a machine $M = (S, \text{step}) : A \rightarrow B$ and a state $s \in S$, the *response* of s is the function $r_{M,s} : A^* \rightarrow B^*$ defined by $r_{M,s}(\varepsilon) = \varepsilon$ and, for an input symbol a and a word w over A ,

$$r_{M,s}(a \cdot w) = b \cdot r_{M,s_1}(w) \quad \text{wherever} \quad \text{step}(s, a) = (b, s_1),$$

with $\cdot$ word concatenation. The response is the full input–output behavior of the machine started in state s .

Definition 3.21 (Behavioral equivalence). Machines $M, M' : A \rightarrow B$ are *behaviorally equivalent*, written $M \approx M'$, if their sets of realized responses coincide:

$$\{r_{M,s} : s \in S\} = \{r_{M',s'} : s' \in S'\}.$$

Each state of one machine realizes a response realized by some state of the other. For deterministic machines this relation coincides with bisimulation in the sense of Park [Park, 1981].

Remark 3.22. State-space isomorphic machines are behaviorally equivalent: the isomorphism pairs states with equal responses, and the response of the image state is the response of the source state by Theorem 3.2. Hence $\approx$ is well defined on the morphisms of Mol. It is an equivalence relation, being equality of sets, and it is strictly coarser than state-space isomorphism: a machine with an unused state realizes the same responses as the machine without it.

Theorem 3.23 (Behavioral equivalence is a congruence (Theorem 3.23)). *[N]. Behavioral equivalence (bisimulation) is a congruence for $\circ$ and $\otimes$; the quotient category $\text{Mol}/\approx$ is a symmetric monoidal category.*

Proof. Composition is a congruence. First observe that the response of a state determines the responses of all its successors: if $r_{M,s} = r_{M,s'}$, $\text{step}(s, a) = (b, s_1)$, and $\text{step}(s', a) = (b, s'_1)$, then for every word w , $b \cdot r_{M,s_1}(w) = r_{M,s}(a \cdot w) = r_{M,s'}(a \cdot w) = b \cdot r_{M,s'_1}(w)$, so $r_{M,s_1} = r_{M,s'_1}$. Now let f, g be realized by (S, step) , (T, tstep) and write $R_f = \{r_{f,s}\}$, $R_g = \{r_{g,t}\}$. Reading an input word symbol by symbol, the composite machine runs f and threads its outputs into g ; by the observation, the response of the composite at (t, s) is determined by the pair $(r_{g,t}, r_{f,s})$ alone, through the recurrence

$$r_{g \circ f, (t,s)}(a \cdot w) = c \cdot r_{g \circ f, (t_1, s_1)}(w),$$

where $(b, s_1) = \text{step}(s, a)$, $(c, t_1) = \text{tstep}(t, b)$, and the successor responses in the recursion are functions of $(r_{g,t}, r_{f,s})$ by the observation. Hence the response set of $g \circ f$ depends only on R_f and R_g ; if $f \approx f'$ and $g \approx g'$ then $R_f = R_{f'}$ and $R_g = R_{g'}$, so the two composites have the same response set: $g \circ f \approx g' \circ f'$.

Tensor is a congruence. The tensor machine runs f and g independently on the two components, so

$$r_{f \otimes g, (s,t)}(w_1, w_2) = (r_{f,s}(w_1), r_{g,t}(w_2)),$$

and the response set of $f \otimes g$ is $R_f \times R_g$. Equality of the component sets gives $f \otimes g \approx f' \otimes g'$. Hence $\approx$ is a congruence for both operations.

The quotient. A congruence is exactly what is needed for the operations to descend to classes: define $[g]_{\approx} \circ [f]_{\approx} = [g \circ f]_{\approx}$ and $[f]_{\approx} \otimes [g]_{\approx} = [f \otimes g]_{\approx}$, well defined by the two congruences. The category laws and the coherence equations of Theorem 3.12 hold in Mol and are equations between morphisms, so they hold in the quotient; the structural morphisms of Theorem 3.12 remain isomorphisms there, since e.g. $[\sigma_{B,A}]_{\approx} \circ [\sigma_{A,B}]_{\approx} = [\text{id}]_{\approx}$. Hence $\text{Mol}/\approx$ is a symmetric monoidal category. On small finite machines the congruence property is checked exhaustively by the `bisim` tool of Appendix A. $\square$

Theorem 3.24 (Closure under the tensor (Theorem 3.24)). *[N]. PureMol and EffMol are closed under the tensor: the tensor of two pure molecules is pure; the tensor of an effectful molecule of EffMol(Ctx) with a pure molecule, on either side, is effectful; and the tensor of two effectful molecules of EffMol(Ctx) is a molecule of EffMol(Ctx $\times$ Ctx), equivalently a hybrid molecule.*

Proof. The tensor of pure machines $((), \text{step})$ and $((), \text{tstep})$ is realized by the state space $() \times ()$ with the product transition; the canonical bijection $() \times () \rightarrow ()$ is a state-space isomorphism, so the tensor class is pure. For $f = [\text{Ctx}, \text{step}]$ in $\text{EffMol}(\text{Ctx})$ and a pure machine $((), \text{tstep})$, the tensor $f \otimes p$ is realized by $\text{Ctx} \times ()$; the bijection $\Phi(s, ()) = s$ is a state-space isomorphism onto the machine with state space Ctx and transition $\text{step}_1(s, (a, c)) = (b, d), s_1$ where $(b, s_1) = \text{step}(s, a)$ and d is the output of the pure component on c , because the pure component performs no state update. Hence $f \otimes p \in \text{EffMol}(\text{Ctx})$; the case $p \otimes f$ is symmetric. Finally, the tensor of two context machines is realized by $\text{Ctx} \times \text{Ctx}$, which is a morphism of $\text{EffMol}(\text{Ctx} \times \text{Ctx})$ and, being of the form $S_{\text{pure}} \times \text{Ctx}$ with $S_{\text{pure}} = \text{Ctx}$, a hybrid molecule by Theorem 3.18. $\square$

Remark 3.25. The exact boundary of Theorem 3.24 is worth stating: the tensor of two $\text{EffMol}(\text{Ctx})$ molecules has state space $\text{Ctx} \times \text{Ctx}$, and since $\text{Ctx} \times \text{Ctx} \cong \text{Ctx}$ fails in general (for finite Ctx with more than one element), $\text{EffMol}(\text{Ctx})$ is not closed under the tensor with itself in the strict sense. What the tensor always preserves is purity strictly and effectfulness in the graded sense above: contexts add, they do not vanish.

3.7 Atoms and Determinism

Molecules are built from *atoms*: the primitive components of a dataplane, total on their declared domain, with no hidden state beyond their declared one.

Definition 3.26 (Atoms). An *atom* is a molecule that is a generator rather than a composite: formally, a chosen morphism of a signature Σ , and a *molecule over* Σ is any morphism of the subcategory of Mol generated by Σ under $\circ$ and $\otimes$ [Lawvere, 1963]. Concretely, the atoms of a dataplane are the pure atoms: total functions such as *parse*, *checksum*, *push*, *pop*: and the effectful atoms of $\text{EffMol}(\text{Ctx})$, such as reading and writing context fields. Every atom is total on its declared domain and deterministic. The theories of signatures are the subject of chapter 4, and their free molecules and normal forms of chapter 6.

Theorem 3.27 (Determinism (Theorem 3.27)). *[N]. Pure molecules and tensor products of pure molecules are deterministic; behavioral equivalence preserves determinism.*

Proof. A molecule is deterministic by construction of Theorem 3.1: its transition function is a total function, so from any state and input exactly one output and one next state result; no Mol morphism involves choice. In particular every pure molecule is deterministic, and the tensor of pure molecules is pure by Theorem 3.24, hence deterministic. The content of the theorem is that determinism, unlike purity, is a property of behavior: for every machine M and state s , the response $r_{M,s}$ of Theorem 3.20 is a total single-valued function $A^* \rightarrow B^*$. Behavioral equivalence compares exactly these functions (Theorem 3.21), so if $M \approx M'$ then M' realizes precisely the responses of M : a deterministic machine has only deterministic equivalents, and every class of $\text{Mol}/\approx$ consists of deterministic behaviors. Purity, by contrast, is not invariant under $\approx$: a machine with an unused state is behaviorally equivalent to a pure machine without it, yet is not pure. This is why the rewrite and minimization chapters normalize behavior first (chapter 5, chapter 9): determinism is preserved by every equivalence the engine uses, and the same input always yields the same output through the same states. $\square$

3.8 Discussion

Mol realizes the design of chapter 1: a category whose objects are types and whose morphisms are stateful transformations, quotiented so that state is an implementation detail, and closed under the two composition schemes of the dataplane. The category laws (Theorem 3.7–Theorem 3.8) license equational reasoning about pipelines; the symmetric monoidal structure (Theorem 3.12) licenses parallel composition and the tuple manipulations used throughout chapter 4–chapter 5; the subcategories PureMol , $\text{EffMol}(\text{Ctx})$, and $\text{HybridMol}(\text{Ctx})$ (section 3.3–section 3.5) fix the vocabulary in which chapter 8 proves the pure/effectful decomposition; and behavioral equivalence (Theorem 3.23) is the relation under which minimization (chapter 9) and testing (chapter 14) operate, while determinism (Theorem 3.27) is the assumption under which the feedback structure of chapter 10 and the decision problems of chapter 13 are stated. The next chapter adds equational theories: dataplane components are models of Lawvere theories, and the ring, parser, and transport equations become laws that molecules must satisfy [Lawvere, 1963].

3.9 Optimality of the model

This section makes precise the claim that the pure, effectful, and hybrid classes of Mol are forced by the model rather than chosen by taste, and that the model excludes no application. Two theorems

Theorem	Tag	Statement
3.7	[N]	Mol is a category: composition of (S, step) classes is associative with identity.
3.8	[N]	The state-space bijection quotient is well defined: equivalent machines compose to equivalent machines.
3.12	[C]	Mol is symmetric monoidal: tensor the tuple product, unit $()$, coherent braiding [Mac Lane, 1971].
3.14	[N]	PureMol is equivalent to the category of types and total functions.
3.17	[N]	EffMol(Ctx) is isomorphic to the Kleisli category of the state monad on Ctx.
3.23	[N]	Behavioral equivalence is a congruence for $\circ$ and $\otimes$; Mol/ $\approx$ is symmetric monoidal.
3.24	[N]	PureMol and EffMol are closed under the tensor.
3.27	[N]	Pure molecules and tensor products of pure molecules are deterministic; behavioral equivalence preserves determinism.

Table 3.1: The theorems of this chapter; the statements reproduce the catalog of section 1.3. [N] marks statements original to this thesis, proved with elementary means; [C] marks classical results restated and fully proved inside Mol, with the source cited.

establish the claim: the classification is the unique extension of the atom classification that respects the algebraic structure (Theorem 3.28), and the free construction is initial, so every interpretation factors through it and no application is rigidified (Theorem 3.29).

Theorem 3.28 (The classification is forced). *Let $\Sigma = \Sigma_p \uplus \Sigma_e$ be an atom signature partitioned into pure and effectful atoms, and let $F(\Sigma)$ be the free symmetric monoidal category on Σ . There is exactly one assignment $c : \text{Mor}(F(\Sigma)) \rightarrow \{P, E, H\}$ of the three classes that (i) agrees with the partition on atoms, (ii) assigns the structural morphisms (identities, braidings) to P , and (iii) is a congruence: the class of a composite or tensor is a function of the classes of the factors, with the closure rules $P \circ P \subseteq P$, $P \circ E \subseteq E$, $E \circ P \subseteq E$, $E \circ E \subseteq E$ (sequential composition is Kleisli-closed), $P \otimes P \subseteq P$, $P \otimes E \subseteq E$, $E \otimes E \subseteq H$, and $H \otimes \cdot \subseteq H$. The class map is*

$$c(f) = \begin{cases} P & \text{if } f \text{ is pure,} \\ E & \text{if } f \text{ is effectful,} \\ H & \text{otherwise.} \end{cases}$$

Every class of the free model is one of the three; no fourth class is generated, and no two are identified.

Proof. Define c by induction on the syntax of morphisms of $F(\Sigma)$. Atoms: $c(a) = P$ for $a \in \Sigma_p$ and $c(a) = E$ for $a \in \Sigma_e$; structural morphisms have $c = P$. For composites, apply the closure rules; for tensors, apply the tensor closure rules. The assignment is total by induction, because every morphism of the free category is built from atoms, structural morphisms, $\circ$, and $\otimes$. It is well-defined with respect to the equational theory of $F(\Sigma)$: the symmetric monoidal axioms (associativity, unit, interchange, braiding) equate morphisms whose class assignment agrees, because the class of a composite is a function of the classes of its factors and the closure rules are closed under the structural equations (identities and braidings are pure, and composing or tensoring with a pure morphism preserves the class by the closure rules). Uniqueness: any assignment satisfying (i)–(iii) must agree with c on atoms and structural morphisms, and must then agree on every composite

by induction on its syntax, because (iii) forces the class of each composite to be the closure-rule value of the classes of its factors. The “no fourth class” claim is the totality of the three-valued map; the “no identification” claim follows because the map is well-defined, so two morphisms in different classes cannot be equated by the theory. $\square$

Theorem 3.29 (Universality: no application is excluded). *(1) For any symmetric monoidal category $\mathcal{C}$ and any interpretation of the signature Σ in $\mathcal{C}$, there is a unique strict symmetric monoidal functor $F(\Sigma) \rightarrow \mathcal{C}$ extending the interpretation. (2) Every finite-state Mealy transformation is realized by a molecule over a finite signature. (3) Mol is closed under products, sums, feedback (traces), and batching. Consequently every application expressible as a deterministic stateful transformation is expressible as a molecule, and structural equality in the free model is decided by the coherence theorem of Mac Lane [Mac Lane, 1971].*

Proof. (1) is the universal property of the free symmetric monoidal category: $F(\Sigma)$ is the initial object in the category of symmetric monoidal categories equipped with a Σ -indexed family of objects, and the unique functor is given by interpreting generators as generators, composites as composites, and coherence isomorphisms by the coherence theorem. (2) Let (S, step) with $\text{step} : S \times A \rightarrow B \times S$ be a finite-state Mealy machine. Take the signature Σ with one effectful atom $e_s : A \rightarrow B$ for each state $s \in S$ whose step function is $e_s(a) = (b, s')$ where $\text{step}(s, a) = (b, s')$, and one choice atom for the initial state. The molecule $\otimes$ -wiring of the atoms together with the trace construction of chapter 10 realizes the machine: the state is threaded through the trace, and each step is the effectful atom for the current state. The realization is exact because Mealy determinism makes the output a function of the state and input alone. (3) Products and sums are objects of the free category over a signature extended by the corresponding constructors; feedback is the traced structure of chapter 10; batching is the operadic structure of chapter 11. The closure statements are theorems of those chapters. $\square$

Remark 3.30 (On the choice of language). The model uses categories of machines rather than the alternative frameworks for effectful computation. Each alternative is a proper substructure of the same picture: the Kleisli category of the state monad is exactly the effectful subcategory of Mol (section 3.4); Freyd categories and Hughes arrows are abstract machines with a premonoidal tensor, while Mol’s tensor is a true product, which is what licenses the interchange and coherence laws of section 3.2; coalgebraic final semantics underlies the minimization theorems of chapter 9; effectus theory targets probabilistic and quantum effects and is not needed for deterministic stateful transformations; and process calculi introduce nondeterminism, which would invalidate the cost and allocation theorems of chapter 7 and chapter 12. The trichotomy of Theorem 3.28 is finer than the deterministic/probabilistic dichotomy of effectus theory within the deterministic world, and Theorem 3.29 shows that no application is excluded. The choice of Mol is therefore not a heuristic preference; it is the minimal language in which the engineering properties of this thesis are theorems.

Chapter 4

Lawvere Theories for Dataplane Components

A component is a molecule governed by equations. The ring buffer obeys cancellation laws relating *push* and *pop*; the parser obeys the laws of sequence and choice; the transport obeys the laws of batching. This chapter collects the operations and equations of each component into a *Lawvere theory* Lawvere [1963], Lack and Rosický [2008] (§2.8), with three consequences. First, the equations become theorems: an identity of the theory holds in *every* model, so a law verified once holds for every realization of the component in Mol. Second, the theory is the instrument of *completeness*: for the ring theory the free model decides every term identity, so the rewrite engine of chapter 5 is complete on the ring fragment. Third, the equations, read from left to right, are *rewrite rules*: they license the optimizations of a dataplane: ring cancellation (§4.1), parser left factoring (§4.2), and batch regrouping (§4.3): and each rule is sound because the equation is an identity of the behavioral quotient $\text{Mol}/\approx$ (Theorem 3.23). The theories, their models in Mol, and the rules they license are developed in §4.1–§4.4, with Theorem 4.3–Theorem 4.8 stated and proved in place.

Composition convention. The sequential composite of molecules is written in pipeline order: $f;g$ denotes “perform f , then g ”, the composite $g \circ f$ of section 2.1 read in the opposite direction; the catalog statements of chapter 1 write this composite of the unary ring operations with $\circ$. The tensor $f \otimes g$ is the parallel composite of Mol (chapter 3), a symmetric monoidal category (Theorem 3.12). Equations whose two sides realize genuinely different state spaces are asserted in the behavioral quotient $\text{Mol}/\approx$ (Theorem 3.23); where the state spaces are in bijection, they hold in Mol itself.

4.1 The Ring-Buffer Theory

The ring buffer is the workhorse of the dataplane: a preallocated, fixed-capacity array, indexed by bitmask (Theorem 11.3), into which elements are written and from which they are read. Its equational content is exactly what the rewrite engine of chapter 5 needs.

Definition 4.1 (Ring-buffer theory). The *ring-buffer theory* $\mathcal{R}$ is the one-sorted Lawvere theory

over the unary signature $\{push, pop, peek\}$ presented by the equations

$$push; pop = id, \tag{4.1}$$

$$pop; push = id \quad (\text{when the buffer is non-empty}), \tag{4.2}$$

$$peek; pop = pop, \tag{4.3}$$

$$peek; peek = peek, \tag{4.4}$$

with id the identity operation. A *model* of $\mathcal{R}$ in Mol assigns to the generic object a type B of buffer states, realizes each operation as a molecule of the appropriate type, and satisfies the equations as identities of $\text{Mol}/\approx$; the guarded equations hold on the domains on which the operations are defined.

The *buffer-content model* realizes the theory on finite sequences. Fix a type E of elements; write $a :: s$ for the sequence with front element a and remaining content s . On the state space $B = E^*$ of finite buffers the operations are realized by the transformations

$$push(a, s) = a :: s, \quad pop(a :: s) = (a, s), \quad peek(a :: s) = (a, a :: s),$$

the last two defined on the non-empty buffers. A finite ring of capacity $N = 2^k$ realizes the same laws on the invariant states $E^{\leq N-1}$ (Theorem 11.3), with $push$ defined while room remains. This is the ring-buffer molecule of the dataplane in the single-ended discipline: $push$ inserts at the write position, pop removes the element most recently inserted, and the two are behavioral inverses up to the guards. The equations characterize exactly this discipline; under FIFO usage a pop after a push removes a different element, and the laws fail.

Remark 4.2 (The free ring model). The free algebra of the $\{push, pop\}$ fragment on one generator is the *ring* of positions $\mathbb{Z}$: carrier $\mathbb{Z}$, $push(n) = n + 1$, $pop(n) = n - 1$, generator 0. Every cyclic quotient $\mathbb{Z}/2^k$ is a finite model of the fragment, and $\mathbb{Z}$ is the free model for which the theory is complete (Theorem 4.3).

Theorem 4.3 (Ring theory equations (Theorem 4.3) [N].) *The ring theory equations $push \circ pop = id$ and $pop \circ push = id$ (when the buffer is non-empty) hold in every model of $\mathcal{R}$, and the equations are complete for the free ring model.*

Proof. Read in pipeline order, the two equations are (4.1)–(4.2); the equations of a presentation hold in every model, so it remains to verify the intended model and the completeness claim.

The buffer-content model is a model. On $B = E^*$, $push; pop$ sends (a, s) to $a :: s$ and back to (a, s) : the identity transformation on $E \otimes B$. On a non-empty buffer $a :: s$, $pop; push$ sends the state to (a, s) and back to $a :: s$: the identity wherever pop is defined. The peek equations are verified identically: $peek; pop$ and pop both map $a :: s$ to (a, s) , and $peek; peek = peek$ since $peek$ leaves the state unchanged.

Completeness for the free ring model. Orient the two equations as the rewrite rules $push; pop \rightsquigarrow id$ and $pop; push \rightsquigarrow id$, and consider terms in one variable x . Each step deletes one $push$ and one pop , so the *net exponent* $e(t) = \#push - \#pop$ is invariant, and a term with no adjacent pair of distinct operations is a pure run $push^k(x)$ or $pop^k(x)$, of which exactly one exists per value of e . Every term reduces to the unique normal form with its net exponent, so two terms are provably equal exactly when their net exponents agree. In the free ring model $\mathbb{Z}$ of Theorem 4.2, a term of net exponent e evaluates to e , so two terms are equal in the free ring model exactly when their net exponents agree. Provable equality and validity therefore coincide: the equations are complete for the free ring model, and the same argument decides the word problem of the fragment by normal forms, the content of the ring rewrite system of chapter 5. $\square$

4.2 The Parser Theory

Parsers in a dataplane must be deterministic: at every point the machine makes exactly one choice, resolved by the next unconsumed token, the LL(1) discipline. Under this discipline the combinator structure of parsing is a Lawvere theory.

Definition 4.4 (Parser theory). The *parser theory* $\mathcal{P}$ is the Lawvere theory over the binary operations *seq* (sequence) and *choice*, and the nullary operations ε (the empty parser) and *fail* (the failing parser), presented by the equations

$$\text{seq}(\text{seq}(p, q), r) = \text{seq}(p, \text{seq}(q, r)) \quad (\text{sequence associative}), \quad (4.5)$$

$$\text{seq}(\varepsilon, p) = p = \text{seq}(p, \varepsilon) \quad (\text{identity laws}), \quad (4.6)$$

$$\text{choice}(\text{choice}(p, q), r) = \text{choice}(p, \text{choice}(q, r)) \quad (\text{choice associative}), \quad (4.7)$$

$$\text{choice}(p, \text{fail}) = p = \text{choice}(\text{fail}, p) \quad (\text{failure identity}), \quad (4.8)$$

$$\text{seq}(p, \text{choice}(q, r)) = \text{choice}(\text{seq}(p, q), \text{seq}(p, r)) \quad (\text{left distribution}). \quad (4.9)$$

The last equation is *left distribution*: sequence distributes over choice on the left, and read from right to left it is *left factoring*, the extraction of the common prefix p .

A *parser* in Mol is a molecule $P : T^* \rightarrow T^* \otimes R$ over a token type T and a result type R : it consumes a prefix of the input stream and returns the remaining stream with a result, failing (via the effectus structure of chapter 3) outside its domain. Sequence is the Mol composite $p; q$; choice runs p 's lookahead test and takes the matching branch. Each choice is resolved by the next unconsumed token, so the models of $\mathcal{P}$ are the deterministic parsers of the LL(1) discipline, and the equations of Theorem 4.4 are identities of this semantics (Theorem 4.5).

Theorem 4.5 (Parser theory (Theorem 4.5) [N]). *In the parser theory, sequence distributes over choice, choice is associative, the identity laws hold, and left factoring is sound: the rewrite*

$$\text{choice}(\text{seq}(p, q), \text{seq}(p, r)) \rightsquigarrow \text{seq}(p, \text{choice}(q, r))$$

preserves behavior.

Proof. Work in the stream semantics of §4.2: a parser denotes, for every input stream, the pair (remaining stream, result), or failure. Sequence is composition of these partial transformations and choice is the union over the branch selected by the lookahead test, so it suffices to compare the two sides of each equation on every stream.

Associativity of *seq* and of *choice* is that of the underlying composition and of the ternary “first, else” decision. The identity laws hold because ε consumes nothing and always succeeds, while *fail* never succeeds: both $\text{seq}(\varepsilon, p) = p = \text{seq}(p, \varepsilon)$ and $\text{choice}(p, \text{fail}) = p = \text{choice}(\text{fail}, p)$ are immediate. For left distribution, fix p, q, r . On the left, the input stream is first processed by p , and the choice between q and r is resolved by the first unconsumed token. On the right, the same stream is accepted exactly when it is accepted by $\text{seq}(p, q)$ or $\text{seq}(p, r)$, i.e. exactly when p succeeds on it and q or r succeeds on the remainder; and the result is p 's result followed by the branch result, the same on both sides. The two sides therefore accept the same streams and return the same remainder and result, hence are equal in $\text{Mol}/\approx$. Soundness of left factoring is the same equation read right to left: the rewrite replaces a term by an equal one, so behavior is preserved on every input. $\square$

4.3 The Transport Theory

The transport moves messages across a boundary: a socket, a ring, a device: and the fixed cost of each crossing makes batching the central optimization (chapter 11). The transport theory makes precise the equations that batching relies on.

Definition 4.6 (Transport theory). Fix a type M of messages. The *transport theory* $\mathcal{T}$ is the Lawvere theory over the unary operations $recv$ and $send$ and the n -ary operations $batch_n$ ($n \geq 1$), presented by the equations

$$batch_n(recv, \dots, recv) = \underbrace{recv \otimes \dots \otimes recv}_n, \quad batch_n(send, \dots, send) = \underbrace{send \otimes \dots \otimes send}_n,$$

and by the *interchange law*

$$(f \otimes g); (h \otimes k) = (f; h) \otimes (g; k)$$

for all molecules f, g, h, k of matching types. A *model* of $\mathcal{T}$ in Mol realizes $recv$ and $send$ as transport molecules and $batch_n$ as the n -batch construction of chapter 11; the batch equations state that batching is tensoring, which holds for pure operations: the datagram discipline, in which successive receives are independent: and the interchange law holds in every model.

Theorem 4.7 (Transport theory (Theorem 4.7) [N]). */ In the transport theory, $batch_recv = recv \otimes \dots \otimes recv$, and the interchange law $(f \otimes g); (h \otimes k) = (f; h) \otimes (g; k)$ holds.*

Proof. The batch equation is the defining equation of Theorem 4.6, and it holds in the intended model: by the batch construction of chapter 11, the n -batch of a molecule applies its transition n times, once per input, threading the state; a pure molecule has a single-element state space, so there is no state to thread, and its n -fold tensor applies the same transition to each coordinate independently. Both sides denote the same transformation on the tuple A^n , so $batch_n(recv, \dots, recv) = recv \otimes \dots \otimes recv$, and likewise for $send$.

For the interchange law, let $f : A \rightarrow B$, $h : B \rightarrow C$, $g : A' \rightarrow B'$, $k : B' \rightarrow C'$; both sides are morphisms $A \otimes A' \rightarrow C \otimes C'$. On an input (a, a') , the left side runs f then h on the first component and g then k on the second; the right side runs $f; h$ and $g; k$ on the two components. Both traverse the same transitions per component, and the state spaces $(S_f \times S_g) \times (S_h \times S_k)$ and $(S_f \times S_h) \times (S_g \times S_k)$ correspond under the reassociation bijection, so the machines are equal in Mol: the bifactoriality of the tensor of a symmetric monoidal category (Theorem 3.12, Mac Lane [1971]) in pipeline order. $\square$

4.4 Models in Mol and the Licensed Rewrite Rules

Each theory of this chapter is realized by the corresponding molecule: the ring buffer of §4.1, the parser of §4.2, the transport of §4.3: and each equation, read from left to right, is a rewrite rule available to the rewrite engine of chapter 5. Since every equation is an identity in $\text{Mol}/\approx$, every rule preserves behavior, and the termination and confluence results of chapter 5 (Theorem 5.1–Theorem 5.7) turn the rules into a decision procedure. The central ring rule is ring cancellation: a *push* immediately followed by a *pop* is behaviorally invisible and can be deleted.

Theorem 4.8 (Ring cancellation (Theorem 4.8) [N]). */ An adjacent pair $push \circ pop$ is removable from any ring term with behavior preserved: for every context C in the theory, $C[push; pop] = C[id]$ holds in every model.*

Proof. By (4.1), $push; pop = id$ in every model. Contexts are built from the term constructors: composition with a constant and tensor with a constant: and each preserves equality: if $u = v$ then $w; u = w; v$, $u; w = v; w$, and $w \otimes u = w \otimes v$ for every w . Induction on the construction of C gives $C[push; pop] = C[id]$ in every model, and equal morphisms of $\text{Mol}/\approx$ are behaviorally equivalent, so deleting the pair: in particular from a pipeline in which a pushed element is popped immediately: preserves behavior. The guarded equation (4.2) yields the analogous rule: $pop; push$ is removable wherever the buffer is known to be non-empty. $\square$

Table 4.1: The component theories, their laws, and the rewrite rules they license. Every rule preserves behavior, because each law is an identity of $\text{Mol}/\approx$.

Theory	Law	Licensed rewrite rule
Ring $\mathcal{R}$	$push; pop = id$ (4.3, 4.8)	$push; pop \rightsquigarrow id$: ring cancellation.
Ring $\mathcal{R}$	$pop; push = id$, guarded (4.3)	$pop; push \rightsquigarrow id$ wherever the buffer is known non-empty.
Parser $\mathcal{P}$	left distribution (4.5)	$choice(p; q, p; r) \rightsquigarrow p; choice(q, r)$: left factoring.
Parser $\mathcal{P}$	choice associative (4.5)	$choice(choice(p, q), r) \rightsquigarrow choice(p, choice(q, r))$: reassociation.
Transport $\mathcal{T}$	batch = tensor (4.7)	$batch_n(recv) \rightsquigarrow recv \otimes \dots \otimes recv$: flattening.
Transport $\mathcal{T}$	interchange (4.7)	$(f \otimes g); (h \otimes k) \rightsquigarrow (f; h) \otimes (g; k)$: independent branches.

Remark 4.9. The three theories present components as algebraic structures in the sense of functorial semantics Lawvere [1963], Lack and Rosický [2008]: every implementation of a component is a model, and the equational theory of the component is the theory presented here. The conformance tests of the software standard assert the laws of Table 4.1 directly on the molecule (Appendix B).

Chapter 5

The Rewrite Engine

A dataplane is assembled from atoms and then kept in canonical shape by the rewrite engine: the runtime’s equational theories, directed so that the engine applies them in one direction only, replacing redundant or expensive subexpressions by cheaper ones and reducing every expression to a unique normal form. This chapter formalizes that machinery. The equational theories of chapter 4: the ring theory of bounded buffers (Theorem 4.3–Theorem 4.8), the parser theory of sequence and choice (Theorem 4.5), and the transport theory of batched I/O (Theorem 4.7): become term rewrite systems; monoidal coherence is absorbed into the representation by Mac Lane’s strictification, so coherence generates no rewrite rules of its own Mac Lane [1971]. Termination is proved by well-founded orders (lexicographic path orders), confluence follows from critical-pair analysis and Newman’s lemma Newman [1942], and the systems are produced by Knuth–Bendix completion, the term-rewriting analogue of computing a Gröbner basis Knuth and Bendix [1970]. The seven theorems Theorem 5.1–Theorem 5.7, stated and proved in full below, then establish: termination, local confluence, and completion for the ring system (Theorem 5.1–Theorem 5.3); confluence and unique normal forms for the parser and batch rewrites (Theorem 5.4–Theorem 5.5); soundness of normalization for behavior and cost (Theorem 5.6); and decidability of the full runtime equational theory on finite signatures (Theorem 5.7).

Term rewrite systems. Fix a *signature*: a finite set Σ of operation symbols, each with a fixed arity, and a countably infinite set X of variables. *Terms* are built as usual: variables are terms, and for an n -ary symbol f and terms $t_1, \dots, t_n$ the expression $f(t_1, \dots, t_n)$ is a term; the *size* $|t|$ counts its operation symbols. A *substitution* σ maps variables to terms, extended homomorphically; a *rewrite rule* is a pair $l \rightarrow r$ of terms; and a *term rewrite system* (TRS) is a set of rules. A *rewrite step* $C[l\sigma] \rightarrow_R C[r\sigma]$ replaces an instance of a left-hand side inside any context C , and $\rightarrow_R^*$ is its reflexive–transitive closure. A term is a *normal form* when no step applies to it, and $\text{NF}(t)$ is its normal form when one exists and is unique.

Termination, confluence, critical pairs. A TRS is *terminating* if it admits no infinite rewrite sequence. A *reduction order* is a well-founded order $\succ$ on terms, closed under contexts and substitutions, with $l \succ r$ for every rule $l \rightarrow r$; any reduction order proves termination. The order used below is the classical *lexicographic path order* (LPO): fix a *precedence* on the operation symbols and compare $f(s_1, \dots, s_m)$ with $g(t_1, \dots, t_n)$ by (i) $s_i \succeq t$ for some i , or (ii) f higher in precedence than g and the left term exceeds every t_j , or (iii) $f = g$ and the argument tuples compare lexicographically; LPO is a reduction order over finite signatures. A TRS is *confluent* if any two terms reachable from a common term are *joinable* (both reduce to a common term), and *locally*

confluent if the same holds for one-step reachability. A *critical pair* is a pair of one-step reducts produced by overlapping two left-hand sides at a non-variable position and unifying at their most general overlap; the classical Knuth–Bendix criterion says a TRS is locally confluent iff every critical pair is joinable Knuth and Bendix [1970]. Termination plus local confluence yields confluence by Newman’s lemma, proved below Newman [1942].

The three systems. The engine normalizes three theories on disjoint sorts, so rules of different theories never overlap. The *ring system* R has the two cancellation rules

$$\text{push}(\text{pop}(x)) \rightarrow x, \quad \text{pop}(\text{push}(x)) \rightarrow x,$$

which read, in the composition notation of Theorem 4.3, as $\text{push} \circ \text{pop} = \text{id}$ and $\text{pop} \circ \text{push} = \text{id}$. The non-emptiness guard on the second equation of Theorem 4.3 is satisfied by the pattern itself: $\text{pop}(\text{push}(x))$ denotes a buffer that has just received the element pushed by the inner push and is non-empty by construction. The *parser system* P has the single left-factoring rule

$$(x \cdot a) \oplus (x \cdot b) \rightarrow x \cdot (a \oplus b),$$

with $\oplus$ ordered choice and $\cdot$ sequence; the runtime stores every alternative as a sequence, so an atom is a sequence of length one and the rule factors atom prefixes as well. The *batch system* B has the single flattening rule

$$\text{batch}(\text{batch}(x, y), z) \rightarrow \text{batch}(x, \text{batch}(y, z)),$$

the binary restriction of the n -ary batch of Theorem 11.1.

Newman’s lemma. A classical result bridges local to global confluence: a terminating and locally confluent rewrite relation is confluent Newman [1942]. Since $\rightarrow$ is terminating, its strict transitive closure $\rightarrow^+$ is well-founded, and we argue by well-founded induction on t that any two terms reachable from t are joinable. If t is a normal form, the only term reachable from t is t itself. Otherwise let $t \rightarrow^* u$ and $t \rightarrow^* v$; if either sequence is empty, u and v are trivially joinable, so assume both begin with one step, $t \rightarrow t_1 \rightarrow^* u$ and $t \rightarrow t_2 \rightarrow^* v$. By local confluence t_1 and t_2 are joinable: $t_1 \rightarrow^* w$ and $t_2 \rightarrow^* w$. The terms t_1 , t_2 , and w are proper reducts of t , so the induction hypothesis applies to each: u and w are joinable and v and w are joinable, yielding $u \rightarrow^* w_1$, $w \rightarrow^* w_1$, $v \rightarrow^* w_2$, $w \rightarrow^* w_2$; and since w_1 and w_2 are reachable from w , they are joinable, $w_1 \rightarrow^* z$, $w_2 \rightarrow^* z$. Then $u \rightarrow^* z$ and $v \rightarrow^* z$: u and v join, and induction gives confluence.

Theorem 5.1 (The ring rewrite system is terminating (Theorem 5.1)). *[N]. The ring rewrite system $\text{push}(\text{pop}(x)) \rightarrow x$ and $\text{pop}(\text{push}(x)) \rightarrow x$ is terminating: in the lexicographic path order with precedence $\text{push} \succ \text{pop}$ every rule strictly decreases the term, and every rule also strictly decreases the number of operation symbols.*

Proof. We verify the lexicographic path order directly. For the first rule, $\text{push}(\text{pop}(x)) \succ x$: the immediate subterm $\text{pop}(x)$ satisfies $\text{pop}(x) \succ x$, since $\text{pop}(x)$ has the immediate subterm x with $x \succeq x$ (clause (i)), and clause (i) applied to $\text{push}(\text{pop}(x))$ gives the claim. Symmetrically $\text{pop}(\text{push}(x)) \succ x$, its immediate subterm $\text{push}(x)$ dominating x the same way. LPO is a reduction order over the finite signature $\{\text{push}, \text{pop}\}$, so every step decreases a well-founded order and the system is terminating. The size witness is immediate: each left-hand side has two more operation symbols than its right-hand side, so $|\text{push}(\text{pop}(x))| = |x| + 2 > |x|$, and similarly for the second rule. $\square$

Theorem 5.2 (Local confluence of the ring system (Theorem 5.2)). *[N]. The ring rewrite system of Theorem 5.1 is locally confluent: every critical pair is joinable. It is therefore confluent, by Newman’s lemma, since it is terminating Newman [1942].*

Proof. Let $l_1 = \text{push}(\text{pop}(x))$ and $l_2 = \text{pop}(\text{push}(y))$ (variables renamed apart). The non-variable positions of l_1 are the root and the argument of push , holding the subterm $\text{pop}(x)$. At the argument position, $\text{pop}(x)$ unifies with l_2 under $\theta = \{x \mapsto \text{push}(y)\}$, and the two one-step rewrites of the overlap term $\text{push}(\text{pop}(\text{push}(y)))$ both produce $\text{push}(y)$: the root redex substitutes θ into the right-hand side, and the inner redex $\text{pop}(\text{push}(y))$, reduced by l_2 , leaves $\text{push}(y)$ as well. Symmetrically, the subterm $\text{push}(y)$ at the argument position of l_2 unifies with l_1 under $\{y \mapsto \text{pop}(x)\}$, and the two one-step rewrites of $\text{pop}(\text{push}(\text{pop}(x)))$ both produce $\text{pop}(x)$. At the root the left-hand sides do not unify, since $\text{push} \neq \text{pop}$, and the remaining positions hold variables, where redexes never occur. Hence the only critical pairs are the trivial root self-overlaps and the pairs $\langle \text{push}(y), \text{push}(y) \rangle$ and $\langle \text{pop}(x), \text{pop}(x) \rangle$, each joining in a single step, so every critical pair is joinable and the Knuth–Bendix criterion gives local confluence Knuth and Bendix [1970]; with termination (Theorem 5.1), Newman’s lemma gives confluence Newman [1942]. The normal forms are the terms $\text{push}^k(x)$ and $\text{pop}^k(x)$, $k \geq 0$: a term that avoids both patterns $\text{push}(\text{pop}(\cdot))$ and $\text{pop}(\text{push}(\cdot))$ has all its operation symbols equal. $\square$

Theorem 5.3 (Knuth–Bendix completion of the ring theory (Theorem 5.3)). *[N]. Knuth–Bendix completion of the ring theory, with the equations of Theorem 4.3 oriented by the size-lexicographic order, terminates in a complete system: it returns exactly the two cancellation rules of Theorem 5.1, which are terminating and confluent. The computational claim is script-verified: the checker `kb_completion` of Appendix A executes the completion, and its run prints `PASS` Knuth and Bendix [1970].*

Proof. The completion procedure orients every equation by a well-founded order: here the size-lexicographic order (smaller size first, ties broken lexicographically): then computes critical pairs and adds a rule for every pair that fails to join, until all critical pairs join. The equations $\text{push}(\text{pop}(x)) = x$ and $\text{pop}(\text{push}(x)) = x$ are already oriented: each left-hand side has size 2 and the right-hand side size 0, so the initial rules are exactly R_1 and R_2 of Theorem 5.1. By the critical-pair computation of Theorem 5.2, the two nontrivial critical pairs join in one step and the trivial self-overlaps join in zero steps; no critical pair fails to join, so no rule is ever added and the procedure terminates after the first pass. The resulting system is terminating (Theorem 5.1) and confluent (Theorem 5.2), hence complete, and because the rules are exactly the equations, it presents the same theory and decides it: two terms are equal in the ring theory iff they reduce to a common term, iff their normal forms coincide. The script `kb_completion` implements terms, matching, rewriting, normalization, critical-pair computation, and the completion loop for these two equations; it asserts that completion terminates, that every critical pair joins, that the original rules are present, and that the system decides the theory on sample terms, and it prints `PASS`. The run is recorded in Appendix A Knuth and Bendix [1970]. $\square$

Theorem 5.4 (Parser left factoring (Theorem 5.4)). *[N]. The parser rewrite system with the single left-factoring rule $(x \cdot a) \oplus (x \cdot b) \rightarrow x \cdot (a \oplus b)$ is confluent and terminating. Consequently every parser term has a unique left-factored normal form: a term in which no choice node has two branches sharing a common first component.*

Proof. Termination: for any substitution σ , the left-hand side $(x \cdot a) \oplus (x \cdot b)$ contains $2|x\sigma| + |a\sigma| + |b\sigma| + 3$ operation symbols and the right-hand side $x \cdot (a \oplus b)$ contains $|x\sigma| + |a\sigma| + |b\sigma| + 2$; the difference $|x\sigma| + 1 \geq 1$ is positive, so every step decreases the size and no infinite sequence exists.

Local confluence: the left-hand side $l = (x \cdot a) \oplus (x \cdot b)$ has non-variable positions ε , 1, and 2; the subterms at positions 1 and 2 are $\cdot$ -terms, and no $\cdot$ -term unifies with l , whose root is $\oplus$, so the only overlap of l with a renaming of itself is the trivial root overlap, joining in zero steps. With no other rules, every critical pair is joinable, the system is locally confluent, and Newman’s lemma gives confluence; termination then yields a unique normal form for every term: a term with no redex, i.e. no choice node $\oplus$ whose two branches are sequences with identical first component, which is the left-factored form. The rule is sound by Theorem 4.5 (sequence distributes over choice), and since the runtime stores each alternative as a sequence, atom branches are sequences of length one and atom prefixes are factored as well. $\square$

Theorem 5.5 (Batch-flattening rewrites (Theorem 5.5)). *[N]. The batch rewrite system with the single rule $\text{batch}(\text{batch}(x, y), z) \rightarrow \text{batch}(x, \text{batch}(y, z))$ is terminating and confluent on batch trees. Consequently every batch tree has a unique flattened normal form: a right spine $\text{batch}(x_1, \text{batch}(x_2, \dots, \text{batch}(x_{n-1}, \text{batch}(x_n, \dots)))$ identified with the single n -ary batch of Theorem 11.1.*

Proof. Termination: in the lexicographic path order with the single operation symbol batch and lexicographic argument status, the rule decreases, since the first arguments compare $\text{batch}(x, y) \succ x$ (clause (i): the immediate subterm x satisfies $x \succeq x$), and the lexicographic comparison is decided by the first argument. LPO is a reduction order over the finite signature $\{\text{batch}\}$, so the system is terminating. Local confluence: the left-hand side $l = \text{batch}(\text{batch}(x, y), z)$ has non-variable positions ε and 1; the subterm at position 1, $\text{batch}(x, y)$, unifies with a renaming of l under $x \mapsto \text{batch}(x', y')$, $y \mapsto z'$, producing the overlap term $\text{batch}(\text{batch}(\text{batch}(x', y'), z'), z)$. The outer redex rewrites this to $\text{batch}(\text{batch}(x', y'), \text{batch}(z', z))$, and one further step brings it to $\text{batch}(x', \text{batch}(y', \text{batch}(z', z)))$; the inner redex rewrites the overlap term to $\text{batch}(\text{batch}(x', \text{batch}(y', z')), z)$, and two further steps reach the same term. The critical pair joins, and the root self-overlap is trivial; the Knuth–Bendix criterion gives local confluence, and Newman’s lemma gives confluence. The unique normal form is the term with no redex, i.e. a right spine of batch applications, the flattened batch of Theorem 11.1. $\square$

Theorem 5.6 (Normalization soundness (Theorem 5.6)). *[N]. Suppose every rule of the three systems is sound: the equation it encodes holds as behavioral equivalence of molecules: and cost-nonincreasing: $M \rightarrow N$ implies $\text{cost}(N) \leq \text{cost}(M)$, componentwise on the cost vector of chapter 7. Then normalization is sound and cost-monotone: for every molecule M ,*

$$\text{NF}(M) \approx M \quad \text{and} \quad \text{cost}(\text{NF}(M)) \leq \text{cost}(M).$$

Proof. By termination and confluence of the three systems (Theorem 5.1–Theorem 5.5), every term has a unique normal form and NF is a function: it is the common endpoint of all maximal reduction sequences from the input. For soundness, each rule is behavior-preserving by hypothesis: concretely, the ring rules are the equations of Theorem 4.3, the parser rule is the distributive law of Theorem 4.5, and the batch rule is the flattening identity of Theorem 11.1: so a single step satisfies $M \approx N$; behavioral equivalence is an equivalence relation (chapter 3), and an induction on the length of the reduction sequence from M to $\text{NF}(M)$ gives $\text{NF}(M) \approx M$. For cost, a single step satisfies $\text{cost}(N) \leq \text{cost}(M)$ in every component by hypothesis, and chaining the inequalities along the sequence, by induction on its length, gives $\text{cost}(\text{NF}(M)) \leq \text{cost}(M)$ componentwise. The two halves are independent; the concrete rules satisfy both hypotheses: cancellation removes a redundant pair of operations, factoring parses a shared prefix once instead of once per branch, and flattening replaces a nested batch by a flat one with the same I/O count. $\square$

Theorem 5.7 (Decidability of the runtime equational theory (Theorem 5.7)). *[N]. Let E be the full runtime equational theory: the union of the ring theory (Theorem 5.1–Theorem 5.3), the parser theory (Theorem 5.4), and the transport theory, whose rewrite rules are the batch-flattening rewrites of Theorem 5.5, presented over a finite signature in which the three theories occupy disjoint sorts. Then E is decidable via normalization: for terms s and t of the same sort,*

$$s =_E t \iff \text{NF}(s) = \text{NF}(t),$$

and both normal forms are effectively computable.

Proof. Within each sort the theory is presented by the rules that Theorem 5.1–Theorem 5.5 prove terminating and confluent, so the rewrite relation and the equational theory coincide: $s =_E t$ iff s and t rewrite, in both directions, to a common term, iff $\text{NF}(s) = \text{NF}(t)$, the last equivalence by confluence. The systems act on disjoint sorts, so no rule of one contains a symbol of another and redexes never overlap. Termination of the combined engine: each component normalizes every term of its sort in finitely many steps, and steps of one theory leave the other sorts’ terms untouched; any infinite run would therefore contain infinitely many steps of a single sort, each strictly decreasing that sort’s well-founded size measure while the others stay constant, a contradiction. Confluence: every maximal reduction sequence normalizes each sort component, and each component has a unique normal form, so all maximal sequences from the same term end at the same mixed normal form: NF is a function. Decidability: the signature is finite, so terms are finite trees over a finite alphabet, every step is an effective replacement, and normalization terminates, so $\text{NF}(s)$ and $\text{NF}(t)$ are computable in finite time, and comparing them decides $s =_E t$. Coherence contributes no equations: by Mac Lane’s coherence theorem and strictification the tensor is strictly associative with a strict unit, and the braiding is a morphism rather than a rewrite, so the coherence laws hold in every model without entering E Mac Lane [1971]. $\square$

Typed composition trees and multicategories. The expressions the engine rewrites are embedded in molecules: a molecule over an atom signature Σ is a typed composition tree whose leaves are atoms and whose internal nodes are applications of composition $\circ$ and tensor $\otimes$, each node carrying the types of its domain and codomain. Such trees, with the grafting substitution that plugs the output of one tree into an input position of another, form a *multicategory*: objects are the types, and a multimorphism is a tree with a list of typed inputs and one typed output Leinster [2004]. The rewrite rules are exactly the identities this substitution structure respects: batch flattening is the associativity of substitution, parser factoring the distributivity of sequence over choice, ring cancellation the mutual inverseness of the ring operations: and Kelly’s *clubs* axiomatize the general situation: a club is a category of typed trees whose substitution structure presents the free algebras of a 2-monad, and the free symmetric monoidal category $F(\Sigma)$ of chapter 6 is presented by the club of permutations acting on the atoms Kelly [1974], Leinster [2004]. The engine is therefore not an ad-hoc optimizer but the computational reflection of the multicategorical structure of Mol, and the normal forms of Theorem 5.1–Theorem 5.7 are the canonical forms of typed composition trees, whose equality Theorem 5.7 makes decidable.

5.1 Extending the Engine: Modular Completion

A dataplane engine is never closed: new protocols and new atoms adjoin new equational theories (new rewrite rules) to the systems of chapter 4. The value of the completed systems 5.1–5.7 is that they are *stable under extension*: this section proves the conditions under which adjoining a theory

preserves termination and confluence, so that the normal forms, the decision procedure 5.7, and the lookup table of chapter 6 survive the addition. The disjoint-union part is the classical modularity theorem of Toyama [Toyama, 1987]; the hierarchical part restates the standard non-interference conditions and makes their content explicit.

Definition 5.8 (Disjoint signatures and commuting systems). Two rewrite systems $(\Sigma_1, E_1, \rightarrow_1)$ and $(\Sigma_2, E_2, \rightarrow_2)$ have *disjoint signatures* when Σ_1 and Σ_2 share no function symbols. They *commute* when, for all terms t, u, v with $t \rightarrow_1 u$ and $t \rightarrow_2 v$, there exists w with $u \rightarrow_2^* w$ and $v \rightarrow_1^* w$. A sufficient condition for commutation is that every $\rightarrow_2$ -rule has a $\rightarrow_1$ -normal left- and right-hand side and every $\rightarrow_1$ -rule has a $\rightarrow_2$ -normal left- and right-hand side, in a non-deleting signature: no rule discards proper subterms, so redexes of the two systems never interfere. The *componentwise normal form* of a term under a disjoint union is the term obtained by reducing each Σ_i -colored subterm to its $\rightarrow_i$ -normal form.

Theorem 5.9 (Modular completion, Theorem 5.9 [N]). / Let $(\Sigma_1, E_1, \rightarrow_1)$ and $(\Sigma_2, E_2, \rightarrow_2)$ be terminating rewrite systems, each locally confluent.

- (i) If the signatures are disjoint, the union $\rightarrow_1 \cup \rightarrow_2$ is terminating and confluent, hence complete for $E_1 \cup E_2$; the normal form of a term is its componentwise normal form, and the union decides $E_1 \cup E_2$.
- (ii) If the systems commute, the union is terminating and confluent, hence complete for $E_1 \cup E_2$.
- (iii) Under either combination, the equational theory of the union is decidable whenever the theories of both components are.

Proof. (i) *Termination.* We first prove the commuting lemma: with disjoint signatures, the two systems commute. Let $t \rightarrow_1 u$ and $t \rightarrow_2 v$. A $\rightarrow_1$ -redex is a term whose root symbol lies in Σ_1 ; by disjointness it contains no $\rightarrow_2$ -redex at its root, and symmetrically a $\rightarrow_2$ -redex contains no $\rightarrow_1$ -redex at its root. If the two redexes are disjoint, contract both, obtaining the common reduct w directly. If they are nested, the outer redex has the inner redex strictly below its root: contracting the outer first, then the inner inside the result, or the inner first and then the outer, yields the same term w , because the two contractions act on disjoint positions of the term tree. Hence the systems commute. Now let $t_0 \rightarrow t_1 \rightarrow \dots$ be a $\rightarrow_1 \cup \rightarrow_2$ -reduction. By commutation, the steps can be rearranged into a $\rightarrow_1$ -reduction followed by a $\rightarrow_2$ -reduction: every $\rightarrow_2$ -step can be commuted leftward past every subsequent $\rightarrow_1$ -step without changing the final term of any finite prefix. Each phase is finite by termination of the component systems, so the union is terminating (this is the disjoint-union theorem of Toyama [Toyama, 1987], obtained here from the commuting lemma and the termination of the components). *Confluence.* By Newman's lemma 5.2, local confluence and termination give confluence. Let $t \rightarrow_1 u$ and $t \rightarrow_2 v$; by the commuting lemma $u \rightarrow_2^* w$ and $v \rightarrow_1^* w$ for a common w , so the two steps are joinable; single-system peaks are joinable by local confluence of the components. Hence the union is locally confluent and, by termination, confluent. *Normal forms.* By disjointness, a normal form of the union is a term with no $\rightarrow_1$ -redex and no $\rightarrow_2$ -redex, i.e. the componentwise normal form; confluence makes it unique.

(ii) Commutation gives the same rearrangement: any reduction reorders to $\rightarrow_1^* \rightarrow_2^*$, both phases finite, so the union terminates. For confluence, it suffices again to join the mixed peaks: by commutation $u \rightarrow_2^* w$ and $v \rightarrow_1^* w$; single-system peaks join by local confluence of the components. Termination plus local confluence yield confluence by 5.2. Completeness follows.

(iii) By (i) or (ii) the union is complete, so by the argument of 5.7 two terms are equal in $E_1 \cup E_2$ if and only if their (unique, computable) union normal forms coincide. Each component normal

form is computable by the decidability of the component theory; computing the componentwise normal form therefore decides the union theory. $\square$

Remark 5.10. The hypotheses are the content. Disjoint signatures make the systems commute, which is exactly the non-interference that modularity needs; the hierarchical case (ii) relaxes disjointness to commutation, whose natural sufficient conditions: $\rightarrow_2$ -rules $\rightarrow_1$ -normal on both sides and $\rightarrow_1$ -rules $\rightarrow_2$ -normal on both sides, in a non-deleting signature: are the engine’s extension contract: a new theory may add rules over existing symbols provided it neither creates nor destroys the old theory’s redexes. The engine’s rule-addition API is therefore safe by construction: adding a theory whose rules satisfy the conditions preserves the normal forms, the decision procedure, and the lookup table, so the guarantees of 5.1–5.7 and chapter 6 hold for the extended system unchanged.

Chapter 6

Free Molecules and Canonical Normal Forms

A molecule is a composition of atoms, but a behavior admits many different molecules, and what matters for engineering is structure: which atoms are fused, in what order they run, how their wires are crossed. This chapter separates the syntax of composition from its semantics. We construct $F(\Sigma)$, the *free symmetric monoidal category* on an atom signature Σ : molecules built from the atoms of Σ , quotiented only by the laws of a symmetric monoidal category. Up to coherence, a molecule is a morphism of $F(\Sigma)$ (a wiring diagram whose nodes are atoms) and its meaning in Mol is the image of that diagram under the unique monoidal evaluation.

Three facts about $F(\Sigma)$ do the work of the chapter. *Existence and the universal property*: every molecule over Σ factors uniquely, up to coherent isomorphism, through $F(\Sigma)$ (Theorem 6.4, Theorem 6.5). *Finiteness*: for a finite signature with bounded types and a bounded number of atoms, hom-sets are finite (Theorem 6.6). *Normal forms*: the runtime rewrite system of chapter 5 is terminating and confluent, so every molecule has a unique normal form, and $M \simeq M'$ exactly when $NF(M) = NF(M')$ (Theorem 6.8, Theorem 6.9). The pay-off is the *precomputed lookup table* of optimized molecules: one canonical, cost-optimal molecule per behavior class, built once by exhaustive normalization (verified by the `normal_forms` tool, Appendix A) and consulted at runtime for dense-integer dispatch; section 6.6 closes the chapter with it and its role in the design problems of chapter 13.

6.1 Types, atoms, and signatures

Definition 6.1 (Types and atom signatures). Fix a set T of *base types* (bytes, packets, contexts). A *type* is a finite sequence $A = (X_1, \dots, X_m)$ of base types, of *length* $|A| = m$; the empty sequence $I = ()$ is the unit type and the tensor is concatenation, $A \otimes B = (X_1, \dots, X_m, Y_1, \dots, Y_n)$. A type universe is *bounded* when T is finite and lengths are bounded by d ; there are then exactly $\sum_{m=0}^d |T|^m$ types. An *atom signature* Σ is a set of *atoms* typed $a: A \rightarrow B$ over the base types. A *molecule over* Σ is an expression built from the atoms, identities, and coherence isomorphisms by $\circ$ and $\otimes$.

Types are exactly the objects of Mol, with tensor the tuple product and unit the empty tuple (chapter 3, Theorem 3.12); Mol is a *strict* symmetric monoidal category, since tensor is literal concatenation. The behavioral equivalence $\simeq$ (Theorem 3.23) is a congruence for $\circ$ and $\otimes$ and is the relation for which the normal forms of this chapter are complete (Theorem 6.9).

6.2 The free category on a signature

$F(\Sigma)$ is the quotient of the molecules over Σ by the symmetric monoidal axioms; we use the classical concrete model, morphisms as *wiring diagrams*, in which the axioms hold by construction.

Definition 6.2 (The category $F(\Sigma)$). $F(\Sigma)$ has objects the types over Σ and morphisms $A \rightarrow B$ the isomorphism classes of *wiring diagrams* with boundary (A, B) : a finite set of *nodes* labeled by atoms $a: A_a \rightarrow B_a$ (input ports labeled by the base types of A_a , output ports by those of B_a), the $|A|$ boundary inputs and $|B|$ boundary outputs, and a *wiring*: a bijection between the source ports (boundary inputs and node outputs) and the target ports (node inputs and boundary outputs) pairing only ports of the same base type, such that the directed graph of nodes and wires is acyclic. Composition $g \circ f$ plugs the boundary outputs of f into the boundary inputs of g ; the identity wires each input to the corresponding output; the tensor is disjoint union; the braiding $\sigma_{A,B}: A \otimes B \rightarrow B \otimes A$ crosses the wires of A and B .

Remark 6.3. $F(\Sigma)$ is a small strict symmetric monoidal category: tensor is concatenation, the unit is I , associator and unitors are identities, and the braiding satisfies $\sigma_{B,A} \circ \sigma_{A,B} = \text{id}$ and the hexagon laws, so Mac Lane’s coherence conditions hold Mac Lane [1971]. Acyclicity is automatic for diagrams built from atoms (plugging never creates a wire back to a lower layer) and is what separates the free category from the traced structure of chapter 10.

6.3 Existence and the universal property

Theorem 6.4 (Existence of $F(\Sigma)$). *The free symmetric monoidal category $F(\Sigma)$ on an atom signature Σ exists: it is the initial algebra for the theory of symmetric monoidal categories over Σ : the category of wiring diagrams of section 6.2: and its existence also follows abstractly from accessible and locally presentable categories Adámek and Rosický [1994].*

Proof. Explicit construction. The wiring diagrams form a symmetric monoidal category: plugging is associative and unital; disjoint union is a bifunctor, associative and unital with the empty diagram; the braiding is a natural isomorphism satisfying the symmetry and hexagon laws. The category carries the canonical Σ -structure ρ_Σ (base type $X \mapsto (X)$; atom $a \mapsto$ the one-node diagram) and is freely generated by it: every morphism is a finite composite and tensor of atoms, identities, and braidings, with no equations beyond the symmetric monoidal axioms, since the quotient by them is part of the construction. Being initial among Σ -pointed symmetric monoidal categories (Theorem 6.5), it is the free object. *Abstract existence.* Small symmetric monoidal categories are exactly the models of a finitary essentially algebraic theory (category axioms, tensor bifunctor, coherence isos and equations); such model categories are locally presentable Adámek and Rosický [1994]. The forgetful functor $U: \mathbf{SMC} \rightarrow \mathbf{Graph}$ creates limits and preserves filtered colimits (computed pointwise in $\mathbf{Graph}$), hence is accessible, so by the adjoint functor theorem for locally presentable categories: a limit-preserving accessible functor between such categories has a left adjoint Adámek and Rosický [1994]: U has a left adjoint; applied to the graph with vertices the base types and edges the atoms of Σ it yields $F(\Sigma)$. The two constructions coincide by the uniqueness in Theorem 6.5. $\square$

Theorem 6.5 (Universal property of $F(\Sigma)$). *Let $\mathcal{C}$ be a symmetric monoidal category with a Σ -structure ρ : an object $\rho(X)$ per base type X and, per atom $a: A \rightarrow B$, a morphism $\rho(a): \rho(A) \rightarrow \rho(B)$ (with $\rho(A)$ the tensor of the images of the components of A). Then there is a unique strict monoidal functor $\Phi: F(\Sigma) \rightarrow \mathcal{C}$ extending ρ ; any strong monoidal functor extending ρ is isomorphic*

to Φ by a unique monoidal natural isomorphism Mac Lane [1971]. Consequently every molecule over Σ factors uniquely, up to coherent isomorphism, through $F(\Sigma)$.

Proof. Existence. Set $\Phi(I) = I_{\mathcal{C}}$ and $\Phi((X_1, \dots, X_m)) = \rho(X_1) \otimes \dots \otimes \rho(X_m)$; send the one-node diagram of a to $\rho(a)$, identities to identities, crossings to the braiding of $\mathcal{C}$. Since a diagram is a finite arrangement of atoms, identities, and braidings by $\circ$ and $\otimes$, and the evaluation of a diagram in a symmetric monoidal category is well-defined by Mac Lane’s coherence theorem Mac Lane [1971], Φ extends to all diagrams and is a strict monoidal functor. *Uniqueness.* Any strict monoidal functor extending ρ is forced on objects (tensor of the $\rho(X)$), on one-node diagrams ($\rho(a)$), on identities and crossings, and then on all diagrams by induction, as it preserves $\circ$ and $\otimes$. *Up to coherent iso.* If Ψ is strong monoidal, its constraints $\varphi_{A,B}: \Psi(A) \otimes \Psi(B) \rightarrow \Psi(A \otimes B)$ define $\alpha_{(X)} = \text{id}$ and $\alpha_{A \otimes B} = \varphi_{A,B} \circ (\alpha_A \otimes \alpha_B)$ by induction on the type length: the unique monoidal natural isomorphism $\alpha: \Phi \Rightarrow \Psi$ (Mac Lane’s coherence theorem) Mac Lane [1971]. *Factorization.* Mol is strict symmetric monoidal (Theorem 3.12), so the atoms of Σ form a Σ -structure in Mol and $\text{ev}: F(\Sigma) \rightarrow \text{Mol}$ is the unique strict monoidal functor extending it. A molecule M over Σ is an expression, hence a unique morphism $[M]$ of $F(\Sigma)$ up to coherence, with meaning $\text{ev}([M])$ in Mol; so molecules factor through $F(\Sigma)$ as ev , uniquely up to the coherent isomorphism above. $\square$

6.4 Finiteness of hom-sets

Theorem 6.6 (Finiteness for bounded signatures). *Let Σ be a finite atom signature with bounded types: finitely many base types, types of length at most d , and every atom with source and target of length at most d . Fix a bound n on the number of atoms in a molecule. Then for all types A, B the set of morphisms of $F(\Sigma)$ from A to B whose diagrams contain at most n atoms is finite. In particular, with a fixed atom bound only finitely many normal forms exist (Theorem 6.8), so the lookup table of Theorem 6.9 is finite and precomputable.*

Proof. Only finitely many types occur, $\sum_{m=0}^d |T|^m$. A diagram $A \rightarrow B$ with $m \leq n$ nodes has at most $s^m \leq s^n$ labelings, s being the number of atoms. Each node has at most d input and d output ports, so the diagram has at most $|A| + md \leq d + nd$ source ports and at most $|B| + md \leq d + nd$ target ports, and a wiring is a label-respecting bijection between them, of which there are at most $(d + nd)!$. Hence at most $s^n(d + nd)!$ concrete diagrams, and no more isomorphism classes: $F(\Sigma)(A, B)$ restricted to at most n atoms is finite. No rewrite rule introduces atoms (Theorem 6.8), so the normal forms of molecules with at most n atoms are a subset of this finite set; by Theorem 6.9 they are in bijection with the behavior classes. $\square$

Remark 6.7 (The atom bound is essential). Bounded types alone do not make hom-sets finite: for a single base type X and a single atom $f: X \rightarrow X$, the morphisms $\text{id}, f, f \circ f, \dots$ in $F(\Sigma)(X, X)$ are pairwise distinct, since the universal property (Theorem 6.5) applied to the model in **Set** with $X = \mathbb{N}$ and f the successor function would otherwise force $\text{succ}^m = \text{succ}^n$ for $m \neq n$. What a dataplane uses is finiteness of *bounded-size* molecules; a cost budget bounds the atom count because every atom has positive cost (Theorem 13.1). This is the bounded, enumerable class that the `normal_forms` tool exhausts (Appendix A).

6.5 Unique normal forms up to coherence

The free category fixes the *structural* equations: molecules differing only by coherence denote the same morphism of $F(\Sigma)$. The *behavioral* equations are handled by the runtime rewrite system of

chapter 5: the behavior-preserving rules of the runtime theories ring cancellation (Theorem 4.8), parser left factoring (Theorem 5.4), batch flattening (Theorem 5.5): read as local rewrites of a sub-diagram with fixed boundary, so the system is a well-defined relation on the morphisms of $F(\Sigma)$.

Theorem 6.8 (Unique normal forms). *Let R be the runtime rewrite system of chapter 5 on the molecules over a signature Σ . Then every molecule M over Σ has a unique normal form $NF(M)$: the unique R -irreducible morphism of $F(\Sigma)$ reachable from M , independent of the order of rewriting. The normal form is unique up to coherence: $NF(M)$ is a well-defined morphism of $F(\Sigma)$, and molecules with the same wiring diagram have the same normal form. Moreover $NF(M)$ contains at most as many atoms as M .*

Proof. Termination. No rule introduces atoms, and each rule strictly decreases a well-founded measure: ring cancellation removes two nodes, batch flattening merges nested batch nodes, left factoring lowers the measure of Theorem 5.4; the measures of Theorem 5.1, Theorem 5.4, Theorem 5.5 rule out infinite reductions. *Confluence.* The system is locally confluent: every critical pair joins (Theorem 5.2, Theorem 5.4, Theorem 5.5), and redexes of different theories do not overlap, their patterns being labeled by disjoint atoms. Termination and local confluence imply confluence by Newman’s lemma Newman [1942]. *Uniqueness.* By termination every reduction from M reaches an irreducible morphism; if $M \rightarrow^* u$ and $M \rightarrow^* v$ with u, v irreducible, confluence gives a common reduct w of u and v , and irreducibility forces $u = w = v$. Hence exactly one normal form $NF(M)$ exists. *Up to coherence.* The rules replace a sub-diagram by another with the same boundary, so they act on wiring diagrams: molecules with the same diagram have the same normal form, a morphism of $F(\Sigma)$, and the atom count does not increase. $\square$

At the level of expressions the structural rules $(x \circ y) \circ z \rightarrow x \circ (y \circ z)$, $\text{id} \circ x \rightarrow x$, $x \circ \text{id} \rightarrow x$ compute canonical representatives: they are terminating (the sum over composition nodes of the atoms in their left subtree strictly decreases) and locally confluent (the critical pairs $(\text{id} \circ x) \circ z$, $(x \circ y) \circ \text{id}$, $(x \circ \text{id}) \circ z$, $((x \circ y) \circ z) \circ w$ join in at most three steps), hence confluent Newman [1942], with normal forms the right-nested chains without identities; they preserve the wiring diagram, so they may be interleaved with the behavioral rules without changing $NF(M)$.

Theorem 6.9 (Completeness and the lookup table). *Let R be the runtime rewrite system of chapter 5 on the molecules over a signature Σ : terminating and confluent (Theorem 5.1–Theorem 5.5) and complete for behavioral equivalence, in that the congruence generated by the rules of R is exactly $\simeq$ on the molecules over Σ (a complete system in the sense of Knuth–Bendix Knuth and Bendix [1970]; established for the runtime theories by Theorem 5.3, Theorem 5.6, Theorem 5.7). Then for all molecules M, M' over Σ :*

$$M \simeq M' \quad \text{iff} \quad NF(M) = NF(M'),$$

and $NF(M) \simeq M$ for every M . If Σ is finite with bounded types and bounded atom count, the normal forms form a finite, precomputable lookup table of optimized molecules, one entry per behavior class.

Proof. Soundness. Each rule preserves behavior and is cost-nonincreasing (Theorem 4.8, Theorem 5.4–Theorem 5.6), so $M \rightarrow^* NF(M)$ implies $M \simeq NF(M)$ by induction on the reduction. *Completeness* ($\Rightarrow$). By hypothesis $M \simeq M'$ means that M and M' are connected by R -steps, and this relation is joinability: joinability is reflexive, symmetric, transitive, and preserved by single steps on either side (if $u \downarrow v$ and $u' \rightarrow u$ then $u' \rightarrow^* w$; if $v \rightarrow v'$, confluence applied to v yields a common reduct of w and v'). Hence M and M' have a common reduct, which by termination

and confluence reduces to $NF(M) = NF(M')$. *Completeness* ($\Leftarrow$). If $NF(M) = NF(M')$, then $M \simeq NF(M) \simeq NF(M') \simeq M'$ by soundness. *Lookup table*. By Theorem 6.6 the normal forms of bounded molecules are finite in number, and the two completeness directions make $M \mapsto NF(M)$ a bijection between behavior classes and normal forms; hence the table is one cost-optimal molecule per class (Theorem 7.5, Theorem 7.13), indexed by the type pair and a dense integer: is finite and precomputed by exhaustive normalization. The `normal_forms` tool (Appendix A) performs exactly this enumeration on the signature $f: A \rightarrow B, g: B \rightarrow C, h: C \rightarrow D$: all 3,929 well-typed terms with at most six atom occurrences reduce to the 10 distinct normal forms, with equality of normal forms agreeing with equality of the realized functions on all 7,716,556 pairs: PASS. $\square$

6.6 The precomputed lookup table

A dataplane cannot optimize at runtime; it dispatches. The lookup table is the theory of this chapter compiled into a finite artifact: for the runtime signature (finite, bounded types, bounded hot-path size) it assigns to each type pair $A \rightarrow B$ its finitely many behavior classes (Theorem 6.6), each represented by its normal form (Theorem 6.8), which is exactly its behavior (Theorem 6.9); with the costs of chapter 7 the stored representative is cost-optimal within its class (Theorem 7.5, Theorem 7.13). Using the table is a lookup followed by dense-integer dispatch (policy [MOL] of the companion standard): constant-time and allocation-free.

For the signature $f: A \rightarrow B, g: B \rightarrow C, h: C \rightarrow D$ the table is the following; the third column counts the well-typed terms of depth at most six that normalize to each entry (`normal_forms`, Appendix A).

Normal form $NF(M)$	Type	Terms in class
id_A	$A \rightarrow A$	65
f	$A \rightarrow B$	351
$g \circ f$	$A \rightarrow C$	807
$h \circ g \circ f$	$A \rightarrow D$	1002
id_B	$B \rightarrow B$	65
g	$B \rightarrow C$	351
$h \circ g$	$B \rightarrow D$	807
id_C	$C \rightarrow C$	65
h	$C \rightarrow D$	351
id_D	$D \rightarrow D$	65
Total		3929

Table 6.1: The lookup table for the signature $\{f, g, h\}$: the ten behavior classes (normal forms) and, for each, the number of enumerated well-typed terms of depth at most six that normalize to it.

Each row is the unique normal form of its behavior class (Theorem 6.8, Theorem 6.9), so the table is consistent by construction; syntactically distinct terms in a class one entry per class: $h \circ (g \circ f)$ and $(h \circ g) \circ f$, or f and $f \circ \text{id}_A$: share one entry: the fusion promised by cut elimination (chapter 12, Theorem 12.1) is, for bounded signatures, a table lookup. The same enumeration resolves the design decision problems of chapter 13: a situation $(A \rightarrow B, \Phi, \bar{c})$ seeks a molecule realizing class Φ within cost $\bar{c}$, and the table restricts the search to one candidate per class, computed once (Theorem 13.1).

Remark 6.10 (Summary). $F(\Sigma)$ is the syntax of molecules: existence and the universal property (Theorem 6.4, Theorem 6.5) make it the canonical interface between expressions and Mol; finiteness (Theorem 6.6) makes optimization exhaustive; unique normal forms (Theorem 6.8) and completeness (Theorem 6.9) make it unambiguous, so every behavior class has exactly one canonical, cost-optimal representative, precomputable before runtime and dispatched in constant time.

Chapter 7

Enrichments and Quantitative Semantics

Mol describes what can be composed and which behaviors are equivalent; it says nothing about cost. This chapter enriches the structure with numbers. The central idea is that Mol is a *resource-sensitive monoidal category*: every molecule carries a cost vector with components for execution time, memory footprint, and cache misses, and the composition operations of chapter 3 act on costs in a controlled way. Sequential composition adds costs, tensor composition is subadditive (parallel work shares fixed costs), and a *refinement* preorder orders molecules by quality inside a behavior class.

The chapter is the quantitative backbone of the thesis. Theorem 7.3 makes the cost enrichment well-defined: additive composition cost, the triangle inequality, and tensor subadditivity. Theorem 7.5 reduces global optimality to shortest paths in the finite type graph (Dijkstra, complete). Theorem 7.8 proves the fusion bound: fusing costs no more than composing, and strictly less when the intermediate type never materializes. Theorem 7.11 and Theorem 7.13 formalize refinement and fast-path specialization: substitution preserves refinement, behavior, and cost monotonicity, and every finite refinement lattice has a unique most-refined element per behavior class, so the most-refined dispatcher is sound and cost-minimal. Theorem 7.15 closes the complexity class of the dataplane: atoms whose costs lie in a class closed under the resource operations yield molecules in that class. Theorem 7.17 is the descriptive-complexity anchor: finite-state molecules realize exactly the regular behaviors, and minimal-state molecules are exactly the minimal DFAs Nerode [1958]Hopcroft [1971]. Theorem 7.19 is projection reduction: state that the step function consults only through a projection can be replaced by the projection’s image.

7.1 The cost model

Definition 7.1 (Cost vectors and atom cost assignments). A *cost vector* is an element of $\mathbb{R}_{\geq 0}^3$, the three coordinates being *time*, *memory*, and *cache misses*; write $c(M) = (t(M), m(M), k(M))$ for the cost of a molecule M . The order on cost vectors is componentwise: $u \leq v$ iff $u_i \leq v_i$ for $i = 1, 2, 3$, and $u + v$ is componentwise addition. An *atom cost assignment* for a signature Σ is a function $c: \text{At}(\Sigma) \rightarrow \mathbb{R}_{\geq 0}^3$ from the atoms of Σ to cost vectors.

Definition 7.2 (Additive extension). Given an atom cost assignment, the *additive extension* assigns to every molecule over Σ a cost vector by the clauses $\text{id}_A \mapsto 0$, $\sigma_{A,B} \mapsto 0$, $g \circ f \mapsto c(f) + c(g)$, and $f \otimes g \mapsto c(f) + c(g)$. Theorem 7.3 shows that this assignment is well-defined (independent of the order in which the clauses are applied), additive over composition, and subadditive over tensor.

Theorem 7.3 (Cost enrichment is well-defined). *Fix an atom cost assignment for a signature Σ (Definition 7.1). The additive extension (Definition 7.2) is a well-defined function on the molecules over Σ : it is invariant under the structural laws of the free symmetric monoidal category: reassociation of composition, insertion and deletion of identities, and the coherence equations of the braiding; and it satisfies, for all molecules f, g , the three quantitative laws of a resource-sensitive monoidal category: (1) additive composition cost: $c(g \circ f) = c(f) + c(g)$; (2) triangle inequality: $c(g \circ f) \leq c(f) + c(g)$, with equality, and more generally $c(f_1 \circ \dots \circ f_n) \leq \sum_{i=1}^n c(f_i)$; (3) tensor subadditivity: $c(f \otimes g) \leq c(f) + c(g)$, with equality for the additive extension and strict inequality possible for implementations that share fixed costs across the two factors (batching, chapter 11). Consequently cost is a well-defined resource annotation of Mol: every molecule has a cost vector computed from its atoms by the inductive rules, independent of how the molecule is bracketed.*

Proof. Invariance under the structural laws. Define the cost syntactically: the cost of a molecule is the sum of the costs of its atoms, identities and braidings costing zero. This agrees with the clauses of Definition 7.2, and we show it is invariant under the symmetric monoidal axioms, checked instance by instance. *Associativity:* $c((f \circ g) \circ h) = c(f) + c(g) + c(h) = c(f \circ (g \circ h))$, and similarly for $\otimes$. *Unit laws:* $c(f \circ \text{id}) = c(f) + 0 = c(f)$, and $c(\text{id} \otimes f) = c(f)$. *Braiding:* $\sigma_{A,B}$ contains no atoms, so $c(\sigma_{A,B}) = 0$, and both sides of the symmetry law $\sigma_{B,A} \circ \sigma_{A,B} = \text{id}$ and of the hexagon equations carry cost 0. *Interchange law:* the two sides of $(f \otimes g) \circ (h \otimes k) = (f \circ h) \otimes (g \circ k)$ both sum the costs of f, g, h, k . In every case the sum over atoms is unchanged, so the syntactic cost is invariant under the axioms. Mac Lane’s coherence theorem Mac Lane [1971] states that these axioms generate exactly the equational theory of the free symmetric monoidal category, so the cost of a molecule is independent of the representative chosen, hence well-defined. *The three laws.* *Additivity* holds because the composite $g \circ f$ has as its atoms exactly the atoms of f together with those of g : plugging introduces no new nodes. *Triangle inequality.* By additivity the inequality holds with equality; iterating over a chain of length n gives the general bound. *Tensor subadditivity.* The tensor $f \otimes g$ is the disjoint union of the two machines, so the additive extension gives equality. Any implementation may charge shared setup costs (a single loop header, one syscall amortization) only once; formally, any cost assignment satisfying (1)–(3) bounds its tensor component by the additive one, and the additive extension is the pointwise largest such assignment. Hence (3) holds in general, with equality in the canonical model. $\square$

Remark 7.4 (Cost is defined on molecules, not behavior classes). Well-definedness is with respect to the structural equations, not behavioral equivalence: behaviorally equivalent molecules may carry different costs (Theorem 3.23 only equates behaviors). This is exactly the freedom that refinement (section 7.4) exploits: among the realizations of one behavior, the dataplane may pick the cheapest.

7.2 Global optimality as a shortest-path problem

Cost is additive along composition (Theorem 7.3), so the cost of a pipeline that is a sequential composite of atoms is the sum of the atom costs. Optimization is then graph theory: the atoms are the edges of a finite *type graph*, and the cheapest pipeline is a shortest path in it.

Theorem 7.5 (Global optimality via shortest paths). *Let Σ be a finite signature with bounded types and nonnegative atom costs, additively extended (Theorem 7.3). The type graph G has as vertices the types over Σ and, for every atom $a: A \rightarrow B$, an edge $A \rightarrow B$ of weight $c(a) \geq 0$. For all types A, B : (1) the minimum cost of a path-molecule $A \rightarrow B$: a molecule composed of atoms and identities by sequential composition alone: equals the shortest-path distance $d(A, B)$ in G ; (2) Dijkstra’s algorithm computes $d(A, B)$ for every pair of types, terminates on the finite graph, and*

is complete: whenever $d(A, B) < \infty$ it returns a shortest path, and that path is a min-cost path-molecule; (3) if every atom has a single-type source and a single-type target (the slice signature of the dataplane hot path), then every molecule $A \rightarrow B$ is a path-molecule, and (1)–(2) hold for all molecules: globally optimal molecules are exactly the shortest paths of G , of cost $d(A, B)$.

Proof. (1): the two optimization problems coincide. A path-molecule $A \rightarrow B$ is a finite sequence of atoms $a_1, \dots, a_n$ with $a_i: T_{i-1} \rightarrow T_i$, $T_0 = A$, $T_n = B$, interleaved with identities; it denotes the composite. By additivity (Theorem 7.3), $c(a_1 \circ \dots \circ a_n) = \sum_{i=1}^n c(a_i)$, the total weight of the corresponding path $A \rightarrow B$ in G . Conversely every path in G is the atom sequence of a path-molecule. The two sets are in bijection with equal costs, so the minimum over path-molecules equals the minimum over paths, $d(A, B)$. (2): *Dijkstra’s algorithm*. Maintain for every vertex a tentative distance, and repeatedly extract the unvisited vertex of minimal tentative distance, relaxing its outgoing edges. *Correctness by induction on the extraction order*. Let v be the vertex extracted at step k ; all previously extracted vertices carry their true distances. Any path from the source s to v leaves the extracted set at some edge $u \rightarrow w$ with u extracted and w not yet; its total weight is at least $\text{dist}(u) + c(u \rightarrow w) \geq \text{tentative}(w) \geq \text{tentative}(v)$, the last by the extraction order, and $\text{tentative}(v)$ is the weight of an actual path, so $\text{tentative}(v) = d(s, v)$. Nonnegativity is used exactly at $\text{dist}(u) + c(u \rightarrow w) \geq \text{tentative}(w)$, which holds because relaxing $u \rightarrow w$ set $\text{tentative}(w) \leq \text{dist}(u) + c(u \rightarrow w)$ and later relaxations only lower it. The graph is finite (bounded types, chapter 6), so the algorithm terminates after at most V extractions and E relaxations, $O(E \log V)$ with a priority queue. Recording the predecessor on each relaxation yields a shortest path to every reachable target; by (1) that path is a min-cost path-molecule, and running the search from every source type covers all pairs. This is completeness: every pair with $d(A, B) < \infty$ is returned with a witness. (3): *slice signatures*. Assume every atom has a single base type as source and as target. Let $M: A \rightarrow B$ be any molecule, realized as a wiring diagram (chapter 6). Every atom has exactly one input port and one output port, so every port has in-degree and out-degree at most one: the diagram is a disjoint union of directed paths, each component with exactly one boundary input and one boundary output. For single types A and B the boundary is $A \rightarrow B$, so there is a single component: a path $X_0 \rightarrow X_1 \rightarrow \dots \rightarrow X_n$ with $X_0 = A$ and $X_n = B$, i.e. a path-molecule. Its cost is the weight of that path, at least $d(A, B)$, and by (2) a shortest path attains $d(A, B)$; hence the global minimum over all molecules $A \rightarrow B$ is exactly $d(A, B)$. $\square$

Remark 7.6 (Scope of the statement). For a general signature, a molecule may route through tensor products: a parallel composite of paths joined by a braiding. Its cost is then the sum of its components’ costs (Theorem 7.3), and such parallelism is absorbed into the signature by batch atoms (Theorem 5.5, chapter 11, Theorem 11.2). The theorem’s engineering content is unchanged: with nonnegative additive costs, optimal composition is a shortest-path problem, and Dijkstra’s algorithm is complete for it. The lookup table of chapter 6 (Theorem 6.9) stores exactly the shortest paths of each behavior class.

7.3 The fusion bound

Cut elimination fuses a pipeline into a single morphism (Theorem 12.1, chapter 12); fusion pays for itself. The naive run of $g \circ f$ writes the intermediate value of type B to memory and reads it back; the fused run keeps that value in registers.

Definition 7.7 (Measured cost of materialization). Let $m(B) \geq 0$ be the memory charge of a materialized intermediate value of type B , with $m(B) > 0$ whenever B is nonempty. The *measured cost* of a naive run of the composite $g \circ f$ is $c^{\text{meas}}(g \circ f) = c(f) + c(g) + m(B)$: the atoms do

their nominal work and the intermediate is stored and reloaded. The *canonical fused molecule* for the pair (f, g) is the same atoms rewired so that the intermediate is consumed in registers; it realizes $g \circ f$ and its measured cost is $c^{\text{meas}}(h) = c(f) + c(g)$: no materialization charge. Any *fused realization* of the behavior is a molecule h with $h \simeq g \circ f$ and measured cost at most the canonical one.

Theorem 7.8 (Fusion bound). *Let $f: A \rightarrow B$ and $g: B \rightarrow C$ be molecules. (1) The naive composite realizes $g \circ f$ at measured cost $c(f) + c(g) + m(B)$; the canonical fused molecule realizes the same behavior at cost $c(f) + c(g)$. Hence the optimal fused realization h^* satisfies the fusion bound $c(h^*) \leq c(f) + c(g)$. (2) The bound is strict exactly when the intermediate type never materializes: if B is nonempty, then $m(B) > 0$ and $c^{\text{meas}}(h^*) \leq c(f) + c(g) < c(f) + c(g) + m(B) = c^{\text{meas}}(g \circ f)$: fusing strictly decreases the measured cost, and in particular the memory and cache-miss components improve by the eliminated store and load. If B is empty, $m(B) = 0$ and the two coincide.*

Proof. (1): the canonical fused molecule exists and realizes $g \circ f$. The composite $g \circ f$ has state space $S_f \times S_g$: first f transforms the input and updates its state, then g consumes the intermediate and updates its state. The canonical fused molecule is the same state space and the same two steps, with the intermediate held in registers instead of memory; its atom multiset is exactly that of $g \circ f$ (plugging introduces no atoms, Theorem 7.3), so its nominal cost is $c(f) + c(g)$ and, since it never materializes B , its measured cost is $c(f) + c(g)$ (Definition 7.7). It realizes $g \circ f$ by construction, while the naive run additionally stores and reloads the intermediate, costing $c(f) + c(g) + m(B)$. The optimal fused realization is no more expensive than the canonical one, giving $c(h^*) \leq c(f) + c(g)$. (2): *strictness*. If B is nonempty then $m(B) > 0$, so $c(f) + c(g) < c(f) + c(g) + m(B)$: the left side is the canonical fused cost, the right side the naive composite cost, and fusing strictly decreases the measured cost. In the memory and cache-miss components the difference is exactly the charge of the eliminated store and load, so those components improve strictly. If B is the empty type, $m(B) = 0$, both runs cost $c(f) + c(g)$, and the bound is tight. $\square$

Remark 7.9 (Fusion, normalization, and the lookup table). Fusion is cost-decreasing, and the normalization of chapter 5 is cost-nonincreasing (Theorem 5.6). Together with the fusion bound they imply that the normal form of a composite (the fused pipeline) is a refinement of every enumerated representative of its behavior class; this is the mechanism by which the lookup table of chapter 6 stores cost-optimal molecules (Theorem 6.9, Theorem 7.5, Theorem 7.13).

7.4 Refinement and fast-path specialization

Inside a behavior class, molecules are ordered by cost. A *refinement* replaces a molecule by a cheaper one with the same behavior: the formal counterpart of fast-path specialization: the general path is correct, and the specialized path is also correct and faster.

Definition 7.10 (Refinement). For molecules $f, f': A \rightarrow B$ with $f \simeq f'$, write $f \sqsubseteq f'$ and call f' a *refinement* of f when $c(f') \leq c(f)$ componentwise. Write $f \equiv f'$ when both $f \sqsubseteq f'$ and $f' \sqsubseteq f$ hold (equal behavior and equal cost). The relation $\sqsubseteq$ is a preorder on each behavior class, and a partial order modulo $\equiv$.

Theorem 7.11 (Refinement substitution). *Refinement is preserved by substitution into compositions. Let $f, f': A \rightarrow B$ and $g, g': B \rightarrow C$ be molecules with $f \sqsubseteq f'$ and $g \sqsubseteq g'$. Then $g \circ f \sqsubseteq g' \circ f'$: the composite $g' \circ f'$ is a refinement of $g \circ f$. Behavior is preserved ($g' \circ f' \simeq g \circ f$) and cost is monotone: $c(g' \circ f') \leq c(g \circ f)$. The same holds for the tensor: $f \otimes g \sqsubseteq f' \otimes g'$ whenever the tensor cost is additive on the pairs involved (in particular under the additive extension, Theorem 7.3).*

Proof. Behavior preserved. From $f \sqsubseteq f'$ we have $f \simeq f'$, and from $g \sqsubseteq g'$ we have $g \simeq g'$. Behavioral equivalence is a congruence for $\circ$ and $\otimes$ (Theorem 3.23), so $g \circ f \simeq g' \circ f'$ and $f \otimes g \simeq f' \otimes g'$. *Cost monotone, sequential.* By additivity (Theorem 7.3), $c(g \circ f) = c(f) + c(g)$ and $c(g' \circ f') = c(f') + c(g')$. Since $c(f') \leq c(f)$ and $c(g') \leq c(g)$ componentwise, adding componentwise gives $c(f') + c(g') \leq c(f) + c(g)$, i.e. $c(g' \circ f') \leq c(g \circ f)$. Together with the congruence, $g \circ f \sqsubseteq g' \circ f'$. *Cost monotone, tensor.* Under the additive extension, $c(f \otimes g) = c(f) + c(g)$ and $c(f' \otimes g') = c(f') + c(g')$ (Theorem 7.3), and the same inequality gives $c(f' \otimes g') \leq c(f \otimes g)$; with the tensor congruence, $f \otimes g \sqsubseteq f' \otimes g'$. For an arbitrary cost assignment the same conclusion holds whenever the tensor cost equals the sum on both pairs, which is the stated hypothesis. $\square$

Remark 7.12 (Fast-path specialization). The substitution property is the soundness certificate of specialization: any component of a pipeline may be replaced, at any depth, by a refinement of itself, and the whole pipeline remains a refinement of the original: same behavior, no greater cost. This is how the dataplane’s fast paths are introduced without re-verifying the full pipeline: each specialized atom is checked once against the general one (behavioral equivalence, Theorem 3.23 and chapter 14), and Theorem 7.11 lifts the guarantee to every composite.

Theorem 7.13 (Maximal refinement). *Let $\mathcal{C}$ be a finite set of molecules $A \rightarrow B$ with a common behavior (a behavior class), ordered by $\sqsubseteq$ modulo $\equiv$ (Definition 7.10). (1) Existence. $\mathcal{C}$ has a maximal element: a molecule refined by no strictly better molecule of $\mathcal{C}$: and every element of $\mathcal{C}$ is refined by some maximal element. (2) Uniqueness. If $\mathcal{C}$ is cost-closed: it contains a molecule whose cost is the componentwise minimum of the costs occurring in $\mathcal{C}$: then the maximal element is unique up to $\equiv$: the most-refined representative, of cost $\min_{f \in \mathcal{C}} c(f)$. (3) The most-refined dispatcher, which selects for every behavior class its most-refined representative, is sound: each selected molecule has the class behavior, because it lies in the class (and cost-minimal) no other selection realizes the class at strictly smaller cost in any component.*

Proof. (1): maximal elements exist. If $f \in \mathcal{C}$ is not maximal, some $f_1 \in \mathcal{C}$ strictly refines it: $c(f_1) \leq c(f)$ componentwise and $c(f_1) \neq c(f)$. A strictly refining chain $f, f_1, f_2, \dots$ cannot be infinite: each step lowers at least one component, and the componentwise-decreasing sequences in the finite set of cost vectors of $\mathcal{C}$ are finite. Hence a maximal element is reached, and every element is refined by one (iterate the same argument from it). *(2): uniqueness under cost-closure.* Let $v^* = \min_{f \in \mathcal{C}} c(f)$ componentwise; by cost-closure some $f^* \in \mathcal{C}$ has $c(f^*) = v^*$. Let g be any maximal element of $\mathcal{C}$. Since $v^* \leq c(g)$ componentwise, if $c(g) \neq v^*$ then f^* strictly refines g , contradicting maximality; hence $c(g) = v^*$. All maximal elements therefore carry the cost v^* and the class behavior, so they are pairwise $\equiv$; and f^* itself is maximal, so maximal elements exist and are unique up to $\equiv$. *(3): soundness and cost-minimality of the dispatcher.* The most-refined representative of a class is an element of the class, so it realizes the class behavior: the dispatcher is sound. Its cost is $v^* \leq c(f)$ for every f in the class, so no other selection realizes the class at strictly smaller cost in any component: the dispatcher is cost-minimal. $\square$

Remark 7.14 (Tradeoffs and decision matrices). Without cost-closure there can be several incomparable maximal refinements: one fast but memory-heavy, one slow but memory-light: and this is exactly the tradeoff that the decision matrices of chapter 13 resolve with a weight vector over the cost components. For the dataplane, cost-closure is supplied by the lookup table of chapter 6: normalization is cost-nonincreasing (Theorem 5.6), so the normal form of a class is a refinement of every enumerated representative (Theorem 6.6–Theorem 6.9), and fusion (Theorem 7.8) only improves it further.

7.5 Complexity closure

The complexity of the dataplane is decided by its atoms: whatever resource class the atoms belong to, and which the two compositions preserve, belongs to every molecule built from them.

Theorem 7.15 (Complexity closure). *(1) Structural closure. Let P be a property of molecules preserved by the composition operations: if f and g have P then $g \circ f$ and $f \otimes g$ have P , and all identities and braidings have P . If every atom of a signature Σ has P , then every molecule over Σ has P . (2) Resource closure. Let F be a class of nonnegative cost bounds closed under addition (covering the time and cache-miss components), and let G be a class of state-size bounds closed under multiplication and containing 1 (covering the memory component, because state spaces multiply under both compositions). If every atom of Σ has time and cache-miss cost in F and state size in G , then every molecule over Σ has time and cache-miss cost in F and state size in G .*

Proof. (1): *induction on the construction of molecules.* Every molecule over Σ is built from atoms, identities, and braidings by finitely many applications of $\circ$ and $\otimes$: this is the definition of a molecule (Theorem 3.12, chapter 3), and the free-category construction of chapter 6 shows that the construction is well-founded. Induct on the number of operations. Base cases: an atom has P by hypothesis, and the identities and braidings have P by hypothesis. Induction steps: if f and g have P , then $g \circ f$ and $f \otimes g$ have P because P is preserved by both operations. Hence every molecule over Σ has P . (2): *resource bounds by induction, using the composition laws of Theorem 7.3.* For the time and cache-miss components, additivity gives $t(g \circ f) = t(f) + t(g)$ and $k(g \circ f) = k(f) + k(g)$, and tensor subadditivity gives $t(f \otimes g) \leq t(f) + t(g)$ and $k(f \otimes g) \leq k(f) + k(g)$; since F is closed under addition, the components stay in F at every induction step. For the memory component, the state space of $g \circ f$ is $S_f \times S_g$, and the state space of $f \otimes g$ is likewise $S_f \times S_g$, so $|S_{g \circ f}| = |S_f| \cdot |S_g| = |S_{f \otimes g}|$; the identity has $|S_{\text{id}}| = 1 \in G$, and by induction over the construction the state sizes remain in G , which is closed under multiplication. This proves the resource closure. $\square$

Remark 7.16 (The bounded-resource consequence). For F the bounded functions (atoms with bounded time) and $G = \{1\}$ up to a constant (atoms with bounded state), closure gives: every molecule has bounded time per input and bounded state: subject to the same constants raised by the multiplicative composition of state bounds. The zero-allocation discipline of chapter 12 (Theorem 12.2) is the engineering form of this statement: the complexity class of the dataplane is the complexity of its atoms, never more.

7.6 Descriptive complexity

Finite-state molecules sit exactly at the regular level of the Chomsky hierarchy. Fix an input alphabet A and an output alphabet B for a finite-state molecule $M = (S, \text{step})$, an initial state $s_0 \in S$, and a set $F \subseteq S$ of accepting states. The *language recognized* by M is $L(M) = \{w \in A^* : \text{the run of } M \text{ on } w \text{ ends in } F\}$; the *transduction computed* by M from s_0 is the input–output map of chapter 3.

Theorem 7.17 (Descriptive complexity of Mol). *(1) Regularity. Every finite-state molecule recognizes a regular language, and its transduction is a deterministic Mealy transduction Mealy [1955]: finite state realizes no behavior beyond the regular ones. (2) Completeness. Every regular language is recognized by some finite-state molecule, namely the minimal DFA of the language, whose states are the Myhill–Nerode equivalence classes Nerode [1958]. (3) Minimality. Among the finite-state molecules recognizing a regular language L , the minimum number of states is the number $\kappa(L)$ of*

Nerode classes of L ; every molecule attaining it: a minimal-state molecule: is a minimal DFA, unique up to state-space isomorphism, and Hopcroft’s algorithm computes it in time $O(n \log n)$ from any n -state realization Hopcroft [1971].

Proof. (1): finite state stays inside the regular languages. Write $\delta(s, a)$ for the state component of $\text{step}(s, a)$. The molecule is a finite deterministic automaton (S, A, δ, s_0, F) , and every such automaton recognizes a regular language. Concretely, by state elimination: let $L_s \subseteq A^*$ be the language recognized from state s . The languages L_s satisfy the finite system of equations

$$L_s = \bigcup_{a \in A} a \cdot L_{\delta(s, a)},$$

with the term ε added when $s \in F$. Solving the system by substituting equations into each other: the derivative method of Brzozowski [1964]: expresses every L_s , in particular $L_{s_0} = L(M)$, as a regular expression over A ; hence $L(M)$ is regular. The transduction reading is literal: a finite-state molecule with deterministic step is a deterministic Mealy machine (Theorem 3.27, chapter 3), so its transduction is a deterministic Mealy transduction Mealy [1955]. (2): *every regular language is recognized by a finite-state molecule.* Let $L \subseteq A^*$ be regular. The Nerode equivalence $u \equiv_L v$ iff $\forall w \in A^*: (uw \in L \Leftrightarrow vw \in L)$ has finitely many classes exactly when L is regular Nerode [1958]. The quotient $S_L = A^* / \equiv_L$ with transition $\delta([u], a) = [ua]$ and accepting set $\{[u] : u \in L\}$ is a finite deterministic automaton recognizing L , well-defined because the class definition makes the transition and the accepting predicate independent of the representative. It is a finite-state molecule with state space S_L and $\text{step}([u], a) = (o, [ua])$ for any fixed $o \in B$, the accepting set carried separately, and $L(M) = L$. (3): *minimal states are Nerode classes. Lower bound.* Any automaton recognizing L has at least $\kappa(L)$ states: words u, v with $u \not\equiv_L v$ reach states that must be distinct, since one state cannot both accept and reject the distinguishing continuation w . *Upper bound and uniqueness.* The automaton of (2) has exactly $\kappa(L)$ states, so the minimum is $\kappa(L)$. Let N be any $\kappa(L)$ -state automaton recognizing L and let $\text{st}_N(u)$ be the state it reaches on u . The map $\phi: \text{st}_N(u) \mapsto [u]$ is well-defined: if u and v reached the same state, that state cannot separate uw from vw , so $u \equiv_L v$. It is injective ($u \not\equiv_L v$ forces $\text{st}_N(u) \neq \text{st}_N(v)$, by the lower-bound argument) and surjective (both sets have $\kappa(L)$ elements). It preserves the initial state ($\phi(\text{st}_N(\varepsilon)) = [\varepsilon]$), the transitions ($\phi(\text{st}_N(ua)) = [ua] = \delta_L([u], a)$), and the accepting sets ($\text{st}_N(u) \in F_N \Leftrightarrow u \in L \Leftrightarrow [u] \in F_L$). Hence ϕ is an isomorphism of machines, and minimal-state molecules are unique up to state-space isomorphism; by the state-space bijection quotient of chapter 3 (Theorem 3.8) the minimal molecule is unique. *Complexity.* Hopcroft’s algorithm computes the coarsest stable partition of the states of a given n -state automaton: exactly the partition into Nerode classes: in time $O(n \log n)$ Hopcroft [1971]; applied to any n -state realization of L , it returns the minimal molecule. $\square$

Remark 7.18 (The transduction reading). For transductions the same statement holds with the Nerode relation replaced by the behavioral equivalence $\simeq$ of chapter 3: the minimal-state molecule realizing a transduction is the quotient of any realization by $\simeq$, unique up to state-space isomorphism (Theorem 9.6, chapter 9). The theorem shows that for the language projection this quotient is exactly the Myhill–Nerode quotient, so the two minimality theories coincide.

7.7 Projection reduction

Real molecules carry state that matters only through part of itself. When the step function reads the state only through a projection, the state space can be replaced by the projection’s image without changing behavior and without losing minimality.

Theorem 7.19 (Projection reduction). *Let $M = (S, \text{step})$ be a finite-state molecule and let $\pi: S \rightarrow P$ be a surjection such that the step function factors through π : there is a function $\text{step}_P: P \times A \rightarrow B \times P$ with $\text{step}(s, a) = \text{step}_P(\pi(s), a)$ for all $s \in S$ and $a \in A$: the step depends on the state only through π . Then: (1) (P, step_P) is a well-defined finite-state molecule, the projection reduction of M ; (2) for every initial state s_0 , the reduced molecule started at $\pi(s_0)$ is behaviorally equivalent to M started at s_0 ; (3) if π is discriminating: no two distinct elements of P are behaviorally equivalent under step_P : then the minimal state space of the behavior satisfies $S_{\min} \cong \pi(S) = P$: the projection reduction is already the minimal realization.*

Proof. (1): *the reduction is a molecule.* The factorization says exactly that step is constant on the fibers of π : if $\pi(s) = \pi(s')$, then $\text{step}(s, a) = \text{step}_P(\pi(s), a) = \text{step}_P(\pi(s'), a) = \text{step}(s', a)$. Hence step_P is well-defined on P , and (P, step_P) is a finite-state molecule in the sense of chapter 3. (2): *behavioral equivalence.* Let $(s_i)_{i \geq 0}$ be the run of M on the word $a_1 \cdots a_n$ from s_0 , and $(p_i)_{i \geq 0}$ the run of the reduction from $\pi(s_0)$. We show $p_i = \pi(s_i)$ by induction: for $i = 0$ this is the initialization, and if $p_i = \pi(s_i)$, the next reduced state is the state component of $\text{step}_P(p_i, a_{i+1}) = \text{step}_P(\pi(s_i), a_{i+1}) = \text{step}(s_i, a_{i+1})$, which is $\pi(s_{i+1})$; the outputs coincide at every step by the same equality. Hence the two machines emit the same output word on every input word: they are behaviorally equivalent (Theorem 3.23). (3): *minimality when π is discriminating.* Two states in the same fiber of π are behaviorally equivalent: by (2) applied to the sub-machines started at s and s' with $\pi(s) = \pi(s')$, their runs agree forever. Hence every behavioral equivalence class is a union of fibers, and the minimal state space $S_{\min}$: the quotient of S by behavioral equivalence (Theorem 9.6, chapter 9): is a quotient of the fiber quotient $P = S / \ker \pi$: a canonical surjection $P \twoheadrightarrow S_{\min}$ gives $|S_{\min}| \leq |P|$. Since π is discriminating, the elements of P realize pairwise distinct behaviors, so any realization needs at least $|P|$ states; the reduction provides exactly $|P|$, hence $|S_{\min}| = |P|$. The canonical surjection is then a bijection, and it is an isomorphism of machines: both sides send $\pi(s)$ to its behavioral equivalence class, and the step is preserved by construction. Thus $S_{\min} \cong P = \pi(S)$; by the state-space bijection quotient of Mol (Theorem 3.8) the isomorphism is an equality of Mol-morphisms. $\square$

Remark 7.20 (State hygiene in the dataplane). The step functions of real dataplane atoms consult a handful of fields of a large state record. Projection reduction is the formal license to drop the rest: the molecule that carries only the consulted projection is behaviorally identical and, when the projection is discriminating, already minimal: the layout discipline of policy [CACHE] of the companion standard.

7.8 Resource-Sensitive Cost Enrichment

The cost enrichment of Theorem 7.3–Theorem 7.8 accounts for a molecule by a single cost number. Real dataplane cost has structure: which cache lines a molecule touches, whether concurrently written hot fields collide in a line (false sharing), and how often the branch predictor guesses wrong. This section refines the enrichment to a *resource-graded* structure: a graded monad whose grades live in a resource monoid: and shows that the optimality theorems of 7.5–7.8 survive the refinement.

Definition 7.21 (Resource grades). Let $M = \mathbb{N} \times \mathbb{N} \times \mathbb{R}_{\geq 0}$ be the *resource monoid*, with elements $r = (L, F, B)$ and componentwise addition and order. For a molecule $f: A \rightarrow B$: $L(f)$ is the number of distinct cache lines its working set touches (a structure of size s occupies $\lceil s/64 \rceil$ lines at the standard 64-byte line granularity); $F(f)$ is the number of *false-sharing penalty events*: a penalty event occurs when a write to a hot field of f hits a cache line also touched by a concurrently written

hot field of another molecule running alongside it; and $B(f) = \sum_i p_i \pi_i$ is the *expected branch-misprediction cost*, the sum over branch points of f of the misprediction probability p_i times the penalty π_i in cycles. The *grade* of f is $g(f) = (L(f), F(f), B(f)) \in M$.

Theorem 7.22 (Resource-graded cost enrichment, Theorem 7.22 [N]). */ The assignment $f \mapsto g(f)$ of Theorem 7.21 is a well-defined enrichment of Mol over the monoidal category $(M, +)$:*

- (i) *the identity has grade 0 and composition is additive: $g(\text{id}_A) = (0, 0, 0)$ and $g(h \circ f) = g(f) + g(h)$;*
- (ii) *tensor grades are subadditive in the cache-line component with ceiling arithmetic and additive in the other components: $L(f \otimes h) \leq L(f) + L(h)$, with equality when the working sets of f and h are disjoint; $F(f \otimes h) = F(f) + F(h)$ when the hot fields of f and h lie in distinct cache lines and $F(f \otimes h) = F(f) + F(h) + 1$ when they share a line; $B(f \otimes h) = B(f) + B(h)$;*
- (iii) *the induced pseudo-metric $d(A, B) = \inf\{g(f)_1 + g(f)_2 + g(f)_3 : f : A \rightarrow B\}$ satisfies the triangle inequality.*

Consequently the shortest-path theorem 7.5 extends: with grades additive, a composition from A to B minimizing the total grade is exactly a shortest path in the finite type graph whose edge weights are the grades, and any locally minimal molecule: one no single-step refinement of which lowers the total grade: is globally optimal.

Proof. (i) The identity molecule is empty: it touches no cache lines, has no concurrently written hot fields, and has no branch points, so $g(\text{id}_A) = (0, 0, 0)$. For composition, the accounted costs are additive by definition of the account: the composite $h \circ f$ is accounted to touch the lines of f and the lines of h ($L(h \circ f) = L(f) + L(h)$; the actual number of distinct lines touched is at most this account, so the account is a conservative bound), its false-sharing account is the sum of the components' accounts, and its branch points are the disjoint union of the branch points of f and of h , so by linearity of expectation

$$B(h \circ f) = \sum_{i \in \text{bp}(f)} p_i \pi_i + \sum_{j \in \text{bp}(h)} p_j \pi_j = B(f) + B(h),$$

with no independence hypothesis required: expectation is linear.

(ii) A structure of size s occupies $\lceil s/64 \rceil$ lines, so the tensor $f \otimes h$ (working sets of sizes s_f, s_h) occupies at most $\lceil (s_f + s_h)/64 \rceil \leq \lceil s_f/64 \rceil + \lceil s_h/64 \rceil = L(f) + L(h)$ lines, with equality when the working sets are disjoint. The false-sharing account adds a collision term: the hot fields of $f \otimes h$ are the pairs of hot fields of the components; a penalty event arises exactly when a pair shares a cache line, which contributes 0 when the components' hot fields are separated by the layout (the hot/cold separation of the standard's [CACHE] policy) and 1 otherwise. The branch component is additive by linearity of expectation, as in (i), over the disjoint union of branch points.

(iii) For any $f : A \rightarrow B$ and $h : B \rightarrow C$, the composite $h \circ f : A \rightarrow C$ has grade $g(f) + g(h)$ by (i), so

$$d(A, C) \leq g(f) + g(h) \quad \text{for all } f, h.$$

Taking the infimum over f and h on the right gives $d(A, C) \leq d(A, B) + d(B, C)$: the triangle inequality. (The three grade components combine additively along a path, so the componentwise order makes the infimum well-defined and the inequality holds componentwise.)

Extended shortest-path. By (i), the grade of a composition is the sum of the grades of its atomic steps, so the total grade of a molecule is the weight of its path in the type graph. Minimizing

the grade is therefore a shortest-path problem in the finite type graph (finite whenever the type signature is bounded, as in 6.6), solvable by Dijkstra’s algorithm with the extraction-order induction of 7.5. For local optimality: suppose a molecule $M: A \rightarrow B$ is not globally optimal, so some M' has strictly smaller total grade. The composite paths from A to B form a finite graph; by additivity, moving from M to M' along a cheaper route decomposes into a finite sequence of single-step grade decreases, the first of which is a one-step refinement of M with strictly lower grade: contradicting local minimality. Hence local minima are global optima. $\square$

Remark 7.23. The three components license the three layout disciplines of the companion standard: L is what the cache-line accounting of [CACHE] minimizes; F is exactly the quantity the hot/cold separation and per-core data-structure policies drive to zero; and B is the quantity the refinement poset of 7.11–7.13 orders, so that the most-refined dispatch structure (dense integer matches, branchless cmov patterns) is precisely the one minimizing expected branch cost. The graded enrichment is a graded monad in the sense that grades compose additively along Kleisli-like composition; the resource annotation is the type-level shadow of the runtime account.

7.9 Summary

Theorem	Tag	Content
Theorem 7.3	[N]	cost enrichment well-defined: additivity, triangle inequality, tensor subadditivity
Theorem 7.5	[N]	optimal molecules are shortest paths in the type graph; Dijkstra complete
Theorem 7.8	[N]	fusion bound $c(h^*) \leq c(f) + c(g)$; strict when the intermediate never materializes
Theorem 7.11	[N]	refinement substitution; behavior preserved, cost monotone
Theorem 7.13	[N]	unique most-refined element per class; dispatcher sound and cost-minimal
Theorem 7.15	[N]	complexity closure under $\circ$ and $\otimes$
Theorem 7.17	[C]	finite-state molecules = regular behaviors; minimal states = minimal DFA
Theorem 7.19	[N]	projection reduction $S_{\min} \cong \pi(S)$

Table 7.1: The theorems of this chapter.

This chapter gives Mol its quantitative layer. Costs form a well-defined enrichment of the category (Theorem 7.3): composition adds, tensor is subadditive, and the triangle inequality holds. With nonnegative additive costs, global optimality is a shortest-path problem in the finite type graph, and Dijkstra’s algorithm is complete for it (Theorem 7.5). Fusion always pays: the fused composite costs no more than the sum of its parts, and strictly less when the intermediate type never materializes (Theorem 7.8). Refinement gives fast-path specialization a soundness certificate: substitution preserves behavior and monotonically improves cost (Theorem 7.11), and the most-refined dispatcher is sound and cost-minimal (Theorem 7.13). Complexity closes over composition: the resource class of the dataplane is the resource class of its atoms (Theorem 7.15). The expressive ceiling is exactly the regular level of the hierarchy (Theorem 7.17), and projection reduction prunes state that is only consulted through a projection (Theorem 7.19). Together with the normal forms of chapter 6 and the decision matrices of chapter 13, these results turn “choose a molecule for this behavior within this budget” into a finite, decidable, and computable problem (Theorem 13.1).

Chapter 8

Kleisli Adjunction and Pure/Effectful Decomposition

Two regimes, one category. The subcategories of chapter 3 divide molecules by what they carry. A *pure* molecule has unit state space, hence is a total function between types, and **PureMol** is equivalent to the category of types and total functions (Theorem 3.14). An *effectful* molecule over a context type Ctx has state space Ctx , and **EffMol**(Ctx) is isomorphic to the Kleisli category of the state monad on Ctx (Theorem 3.17, section 2.6). A *hybrid* molecule is any molecule of **Mol**: an arbitrary state space with an arbitrary transition.

Dataplane stages are hybrids: a parser keeps a position, a checksummer an accumulator, a transport sequence numbers, and around each piece of state there is pure work (masking, scaling, reformatting) that must not itself hold state. This chapter proves that the two regimes coexist canonically: every hybrid molecule M factors, uniquely up to isomorphism, as $M = q \circ e \circ p$ with p and q pure and e effectful (Theorem 8.1); the effectful core carries exactly the state and the pure wings carry exactly the stateless work. This is the Kleisli decomposition of **Mol**: every morphism is a Kleisli morphism of the state monad (section 2.6) read between two pure maps.

Loops and hoisting. **Mol** is a traced monoidal category: the trace $\text{Tr}_{A,B}^U$, constructed in chapter 10, feeds the U -output of a loop body back into its U -input and satisfies the axioms of section 2.5 [Joyal et al., 1996]. The trace laws: sliding (Theorem 8.2), vanishing and yanking (Theorem 8.3), superposing (Theorem 8.4): govern where computation may live relative to a feedback loop, and together with the decomposition theorem they license *hoisting*: pure work is moved out of loops, and loop state that is purely determined by the rest of the state is compressed away. Hoisting shrinks the loop-state footprint, and the cache footprint of loop state is monotone in the size of that state (Theorem 8.5). These are the theorems behind the engineering rule : state in one place, pure work outside the loop: that chapter 9 quantifies by minimization and chapter 10 by iteration bounds, and that the software standard turns into policy.

Theorem 8.1 (Kleisli decomposition (Theorem 8.1)). *Every hybrid molecule $M : A \rightarrow B$ decomposes as $M = q \circ e \circ p$ with p, q pure and e effectful, uniquely up to isomorphism. Given a Mealy presentation (S, s_0, step) of M , with $\text{step} : S \times A \rightarrow B \times S$ [Mealy, 1955], the canonical factorization is*

$$p : A \rightarrow A \otimes S, \quad p(a) = (a, s_0), \quad q : B \otimes S \rightarrow B, \quad q(b, s) = b,$$

and $e : A \otimes S \rightarrow B \otimes S$ is the effectful molecule with state space S and initial state s_0 whose

transition runs one step of M from its own state and writes the successor state to its output wire:

$$\delta_e(s, (a, s')) = ((b, s''), s''), \quad (b, s'') = \text{step}(s, a).$$

Uniqueness is twofold. (i) *The effectful core is invariant: if $M = q' \circ e' \circ p'$ is any factorization with p', q' pure and e' effectful with state space S' , then $S' \cong S$.* (ii) *The canonical triple is well defined on presentations: if $(S_1, s_1, \text{step}_1)$ and $(S_2, s_2, \text{step}_2)$ present the same morphism through the state bijection φ , then $p_2 = (1_A \otimes \varphi) \circ p_1$, $q_1 = q_2 \circ (1_B \otimes \varphi)$, and e_2 is the φ -conjugate of e_1 ; hence the decomposition is unique up to isomorphism.*

Proof. Existence. Compose the three machines. The composite $q \circ e \circ p$ has state space $() \otimes S \otimes ()$, initial state s_0 up to the canonical identification, and transition: on input a , p emits (a, s_0) ; e in state s computes $(b, s'') = \text{step}(s, a)$, emits (b, s'') , and advances to s'' ; q projects to b . Hence the composite transitions $(s, a) \mapsto (b, s'')$ with $(b, s'') = \text{step}(s, a)$, which is exactly the transition of the presentation (S, s_0, step) (Theorem 3.7): the machines are equal in Mol, so $M = q \circ e \circ p$.

(i). In Mol, equality of morphisms is equality up to state-space bijection (Theorem 3.8). The composite $q' \circ e' \circ p'$ has state space $() \otimes S' \otimes () \cong S'$ (Theorem 3.7); since it equals M , presented by the state space S , the two state spaces are isomorphic: $S' \cong S$.

(ii). The bijection φ satisfies $\varphi(s_1) = s_2$ and $\text{step}_2(\varphi(s), a) = (b, \varphi(s''))$ whenever $(b, s') = \text{step}_1(s, a)$. Then $p_2(a) = (a, s_2) = (a, \varphi(s_1)) = (1_A \otimes \varphi)(p_1(a))$, and $q_2 \circ (1_B \otimes \varphi)(b, s) = q_2(b, \varphi(s)) = b = q_1(b, s)$. For e , writing δ_e^i for the transition of e_i ,

$$\delta_e^2(\varphi(s), (a, s')) = ((b, \varphi(s'')), \varphi(s'')), \quad (b, s'') = \text{step}_1(s, a),$$

which is exactly the φ -conjugate of δ_e^1 : the machines e_1 and e_2 are the same morphism of Mol (Theorem 3.8), and the canonical triples are carried to each other by φ . $\square$

Theorem 8.2 (Sliding (Theorem 8.2)). *Let $M : A \otimes U \rightarrow B \otimes U$ be the body of a feedback loop and let $f : B \rightarrow B'$ be pure. Then*

$$\text{Tr}_{A, B'}^U((f \otimes 1_U) \circ M) = f \circ \text{Tr}_{A, B}^U(M),$$

written in the catalog as $\text{Tr}(f; M) = f; \text{Tr}(M)$: a pure computation applied inside every iteration may be slid out of the loop and applied exactly once after it [Joyal et al., 1996].

Proof. The trace of chapter 10 makes Mol a traced monoidal category in the sense of section 2.5, whose axioms are the laws of Joyal et al. [1996]; in particular the trace is natural in the output object B : for every morphism $g : B \rightarrow B'$, $\text{Tr}_{A, B'}^U((g \otimes 1_U) \circ M) = g \circ \text{Tr}_{A, B}^U(M)$. Instantiating $g = f$, which is pure, gives the equation. The machine-level content is direct: the feedback register evolves by the loop body's transition alone, and $f \otimes 1_U$ touches only the feed-forward output factor, so the register trajectory is unchanged whether f runs inside every iteration or once after the loop; the accumulated effect is the single application of f to the loop's output. $\square$

Theorem 8.3 (Vanishing and yanking (Theorem 8.3)). *The trace on Mol satisfies the vanishing and yanking laws of Joyal et al. [1996]:*

$$\text{Tr}_{A, B}^I(M) = M, \quad \text{Tr}_{A, B}^{U \otimes U}(M) = \text{Tr}_{A, B}^U(\text{Tr}_{A \otimes U, B \otimes U}^U(M)),$$

the second written in the catalog as $\text{Tr}(\text{Tr}(M)) = \text{Tr}(M)$: tracing over a doubled feedback register is iterating the trace; and

$$\text{Tr}_{U, U}^U(\sigma_{U, U}) = 1_U,$$

where $\sigma_{U, U} : U \otimes U \rightarrow U \otimes U$ is the symmetry (section 2.4): a feedback loop on a swap is the identity.

Proof. These are axioms of a traced monoidal category (section 2.5), and the trace on Mol satisfies them by construction (chapter 10). The machine-level readings are immediate: tracing over the unit object adds no register, so the machine is unchanged; tracing over $U \otimes U$ adds two coupled registers, which is the same as adding one register and feeding it back once more; and $\text{Tr}(\sigma)$ closes the loop on the pure swap, which absorbs the input into the register and returns it on the next step, so on stabilized loop values the loop is the identity. $\square$

Theorem 8.4 (Superposing (Theorem 8.4)). *Let $M : A \otimes U \rightarrow B \otimes U$ and let $N : C \rightarrow D$ be any molecule. Then*

$$\text{Tr}_{A \otimes C, B \otimes D}^U(M \otimes N) = \text{Tr}_{A, B}^U(M) \otimes N,$$

written in the catalog as $\text{Tr}(M) \otimes N = \text{Tr}(M \otimes N)$: a feedback loop running beside an independent molecule is the loop and the molecule side by side.

Proof. This is the superposing axiom of a traced monoidal category (section 2.5), satisfied by Mol's trace by construction (chapter 10). At the machine level, tensoring N beside M places the state space of N beside that of M , and the feedback register couples only to the M -factor, so closing the loop leaves the dynamics of N untouched; the state space of the loop is $S_M \otimes U \otimes S_N$, which is exactly the tensor of $\text{Tr}(M)$ with N (Theorem 3.12). $\square$

Theorem 8.5 (Hoisting and loop-state footprint (Theorem 8.5)). *Let $M : A \otimes U \rightarrow B \otimes U$ be the body of a feedback loop, presented by a machine (S, step) , and suppose the loop state carries purely redundant information: $U = U' \otimes U''$ with a pure map $h : U' \rightarrow U''$ such that the register is initialized in the graph $i = \langle 1_{U'}, h \rangle : U' \rightarrow U$ and every register value in the run lies in that graph, and the loop body preserves the graph: $M \circ (1_A \otimes i) = (1_B \otimes i) \circ M'$ for the compressed body $M' : A \otimes U' \rightarrow B \otimes U'$. Then: (a) Hoisting. $\text{Tr}^U(M) \simeq \text{Tr}^{U'}(M')$ (Theorem 3.23); the pure redundancy h is hoisted out of the loop, and the loop-state footprint shrinks from $|U| = |U'| \cdot |U''|$ to $|U'|$. (b) Monotonicity. With fixed-size state records of r bytes, the resident cache footprint of a loop with loop state W is $|W| \cdot r$ bytes, strictly increasing in $|W|$; hence the cache footprint of loop state is monotone in state size, and the hoisted loop of (a) is strictly smaller whenever $|U''| > 1$.*

Proof. (a). The loop $\text{Tr}^U(M)$ carries state space $S \otimes U$; its register is initialized in the graph, and since the body preserves the graph, the register stays in the graph at every step, so all reachable states lie in $S \otimes i(U')$. Conjugate by the bijection $\varphi : S \otimes i(U') \rightarrow S \otimes U'$, $\varphi(s, (u', h(u')))) = (s, u')$. On a reachable state with input a the full loop computes $M(a, (u', h(u')))) = (1_B \otimes i)(M'(a, u'))$, so it emits the same output and advances to $(s', (u'_1, h(u'_1)))$, whose image under φ is (s', u'_1) ; the compressed loop $\text{Tr}^{U'}(M')$ on (s, u') with input a emits the same output and advances to (s', u'_1) ; the register evolves by the loop body's transition in both loops (chapter 10). The two machines are therefore behaviorally equivalent, $\text{Tr}^U(M) \simeq \text{Tr}^{U'}(M')$ (Theorem 3.23), and their minimal realizations (chapter 9) are isomorphic. The compressed loop stores only u' ; the value $u'' = h(u')$ is recomputed by the pure map h where it is consumed, outside the loop, so the resident loop state shrinks from $|U'| \cdot |U''|$ to $|U'|$ elements.

(b). Fixed-size records of r bytes make the resident working set of a loop with loop state W exactly $|W| \cdot r$ bytes, a strictly increasing function of the state cardinality $|W|$. Replacing U by U' as in (a) multiplies the footprint by the factor $1/|U''| \leq 1$, and strictly so whenever $|U''| > 1$: the cache footprint of loop state is monotone in state size. $\square$

Hoisting in practice. Together the theorems give the operational shape of a hot loop. By Theorem 8.1 a hybrid stage $M = q \circ e \circ p$ runs as pure initiation p , the effectful core e , pure completion q ; the state of the stage is exactly the state of e , and pure work holds none. By

Theorem 8.2–Theorem 8.4 any feedback loop over such a stage can be rearranged so that pure work and independent side computations live outside the loop, and by Theorem 8.5 loop state that is purely determined by the remainder of the loop state is compressed away. The loop that runs on the hot path is therefore the one with the smallest resident state: the state of e alone, after compression. This is what the software standard’s policies [ALLOC] and [CACHE] enforce: no allocation in the hot path, the loop state small enough to stay resident: and what the following chapters quantify: chapter 9 makes the state minimal among behavioral equivalents (Theorem 9.6), and chapter 10 bounds the number of iterations such a loop needs (Theorem 10.5).

Chapter 9

Coalgebraic Minimization

A molecule of Mol is a deterministic Mealy machine: a finite state space, an initial state, and a transition $step: S \times A \rightarrow B \times S$ reading one input symbol and emitting one output symbol (chapter 3). Read coalgebraically, such a transition is a *coalgebra* for a polynomial functor on sets: states carry structure instead of being generated by operations, and the behavior of a state: the outputs it produces on every future input word: is the image of the unique map into a *final* coalgebra. Minimization is then categorical, not syntactic: the minimal state space is the Nerode quotient of the reachable states by behavioral indistinguishability [Nerode, 1958], computed by Hopcroft’s partition refinement [Hopcroft, 1971].

The chapter proves four theorems. Theorem 9.6: every finite-state molecule has a minimal state space, unique up to isomorphism; Theorem 9.7: minimization preserves behavior. Both restate classical results of sequential-machine theory [Moore, 1956, Mealy, 1955] and carry the tag [C]. Theorem 9.8 bounds the minimal state spaces of composed and tensored molecules by the product of the component state spaces; Theorem 9.9 shows behaviorally equivalent molecules have isomorphic minimal state spaces; both are original and carry the tag [N].

Molecules as coalgebras. Fix an input alphabet A and an output set B ; in Mol these are the types of chapter 3. For a set X , write X^A for the functions $A \rightarrow X$. State spaces are finite, as in the dataplane setting of chapter 3.

Definition 9.1 (The machine functor). The *machine functor* is the polynomial functor $F_{A,B}: \mathbf{Set} \rightarrow \mathbf{Set}$, $F_{A,B}(X) = B \times X^A$, acting on $h: X \rightarrow Y$ by $(b, g) \mapsto (b, h \circ g)$. A *coalgebra* for $F_{A,B}$ is a pair (S, c) with $c: S \rightarrow F_{A,B}(S)$; writing $c(s) = (out(s), nxt(s))$, $out(s) \in B$ is the output carried by s and $nxt(s) \in S^A$ its successor map. A *homomorphism* $h: (S, c) \rightarrow (T, d)$ is a function $h: S \rightarrow T$ with $d \circ h = F_{A,B}(h) \circ c$.

Definition 9.2 (Molecules as coalgebras). A molecule presentation $(S, step)$, $step: S \times A \rightarrow B \times S$, is a coalgebra for the machine functor with output object B^A : currying the input letter out of $step$ gives

$$\widehat{step}: S \rightarrow (B \times S)^A \cong B^A \times S^A = F_{A,B^A}(S),$$

with output component $out(s) = (a \mapsto out(s, a)) \in B^A$ and successor component $nxt(s) = (a \mapsto nxt(s, a)) \in S^A$. A molecule of Mol is an isomorphism class of such coalgebras: the state-space bijections of chapter 3 are exactly the isomorphisms of coalgebras. A *presentation* carries an initial state $s_0 \in S$; the behavior of the molecule is the behavior of s_0 .

Behavior and Nerode equivalence. For a state s , the *response* $r_s: A^* \rightarrow B^*$ is defined by recursion: $r_s(\varepsilon) = \varepsilon$ and $r_s(aw) = \text{out}(s, a) \cdot r_{\text{nxt}(s, a)}(w)$; so $r_s(w)$ is the output word produced from state s on input w , and the behavior of the molecule is r_{s_0} .

Definition 9.3 (Nerode equivalence). States $s, t \in S$ are *Nerode equivalent*, $s \sim t$, iff $r_s = r_t$ [Nerode, 1958].

Lemma 9.4 (One-step refinement). *For all states s, t : $s \sim t$ iff for every letter $a \in A$, $\text{out}(s, a) = \text{out}(t, a)$ and $\text{nxt}(s, a) \sim \text{nxt}(t, a)$.*

Proof. ($\Rightarrow$) If $r_s = r_t$, then $\text{out}(s, a) = r_s(a) = r_t(a) = \text{out}(t, a)$ for every a . For every $v \in A^*$, the recursion gives $r_s(av) = \text{out}(s, a) \cdot r_{\text{nxt}(s, a)}(v)$ and the analogous identity for t ; the equal words $r_s(av)$ and $r_t(av)$ have equal first symbols, so their tails agree: $r_{\text{nxt}(s, a)}(v) = r_{\text{nxt}(t, a)}(v)$ for all v , i.e., $\text{nxt}(s, a) \sim \text{nxt}(t, a)$.

($\Leftarrow$) Suppose outputs agree at every letter and successors are equivalent. Induct on $|w|$. For $|w| = 1$, $r_s(a) = \text{out}(s, a) = \text{out}(t, a) = r_t(a)$. For $w = av$ with $v \neq \varepsilon$, the induction hypothesis and equal first symbols give $r_s(av) = \text{out}(s, a) \cdot r_{\text{nxt}(s, a)}(v) = \text{out}(t, a) \cdot r_{\text{nxt}(t, a)}(v) = r_t(av)$. Hence $r_s = r_t$. $\square$

Final coalgebra semantics. Fix an output object C and write $\Omega = C^{A^*}$ for the set of all functions $A^* \rightarrow C$.

Lemma 9.5 (Final coalgebra). *For the machine functor $F_{A, C}(X) = C \times X^A$, the pair $(\Omega, \text{step}_\Omega)$ with $\text{step}_\Omega(f) = (f(\varepsilon), (f_a)_{a \in A})$, where $f_a(w) = f(aw)$, is the final coalgebra: for every coalgebra (S, c) there is exactly one homomorphism $\text{beh}: S \rightarrow \Omega$.*

Proof. Each f_a is a function $A^* \rightarrow C$, so $(\Omega, \text{step}_\Omega)$ is a coalgebra. For (S, c) , $c(s) = (\text{out}(s), \text{nxt}(s))$, define $\text{beh}(s)(\varepsilon) = \text{out}(s)$ and $\text{beh}(s)(aw) = \text{beh}(\text{nxt}(s, a))(w)$. Then

$$\text{step}_\Omega(\text{beh}(s)) = (\text{beh}(s)(\varepsilon), (w \mapsto \text{beh}(s)(aw))_a) = (\text{out}(s), (\text{beh}(\text{nxt}(s, a)))_a) = F_{A, C}(\text{beh})(c(s)),$$

so beh is a homomorphism. Any homomorphism h is forced by the defining equation to satisfy $h(s)(\varepsilon) = \text{out}(s)$ and $h(s)(aw) = h(\text{nxt}(s, a))(w)$, determining $h(s)(w)$ by induction on $|w|$; hence $h = \text{beh}$. $\square$

For a molecule, $C = B^A$: by induction on $|w|$, $\text{beh}(s)(w)(a) = \text{out}(\text{nxt}(s, w), a)$, the output emitted when the state reached after reading w reads a next. Response and semantic map determine each other: the i -th symbol of $r_s(w)$ is $\text{beh}(s)(w_1 \cdots w_{i-1})(w_i)$, and $\text{beh}(s)(w)(a)$ is the last symbol of $r_s(wa)$: so $s \sim t$ iff $\text{beh}(s) = \text{beh}(t)$: the Nerode equivalence is the kernel of the final map.

The minimal state space. Let (S, s_0, step) be a finite-state presentation. A state s is *reachable* if $s = \text{nxt}(s_0, w)$ for some word w ; the reachable part S_0 is finite and realizes the same behavior r_{s_0} . The quotient

$$S_{\min} = S_0 / \sim$$

carries a coalgebra structure: by Lemma 9.4, $\text{out}(s, a)$ and the class of $\text{nxt}(s, a)$ depend only on the class of s , so $\text{out}([s], a) = \text{out}(s, a)$ and $\text{nxt}([s], a) = [\text{nxt}(s, a)]$ are well defined. The pointed machine $(S_{\min}, [s_0], \text{step}_{\min})$ is the *minimization* $M_{\min}$ of $M = (S, s_0, \text{step})$, with state space $S_{\min}(M)$. In the final coalgebra it is the image $\text{beh}(S_0)$, a subcoalgebra of Ω (successors of reachable states are reachable), and $[s] \mapsto \text{beh}(s)$ is an isomorphism of pointed coalgebras onto it.

Theorem 9.6 (Minimal state spaces (Theorem 9.6)). *[C] Every finite-state molecule has a minimal state space, unique up to isomorphism (Nerode/Hopcroft) [Nerode, 1958, Hopcroft, 1971].*

Proof. We show $M_{\min}$ realizes the behavior of M with the fewest states, and that any two minimal realizations are isomorphic.

Existence. By well-definedness, $M_{\min}$ reaches $[nxt(s_0, w')]$ after a prefix w' of w and emits $out([s_{w'}], a) = out(s_{w'}, a)$ exactly where M emits that symbol; induction on $|w|$ gives $r_{[s_0]}(w) = r_{s_0}(w)$ for all w .

Minimality. Let $(T, t_0, step')$ realize r_{s_0} , with reachable part T_0 , and write r' for responses in T . Put $s_w = nxt(s_0, w)$ and $t_w = nxt'(t_0, w)$. Both machines emit the same output on wu ; splitting each response after $|w|$ symbols, the tail is the response of the state reached by w , so $r'_{t_w}(u) = r_{s_w}(u)$ for every u . The map $t_w \mapsto [s_w]: T_0 \rightarrow S_{\min}$ is well defined and surjective (every state of S_0 is s_w for some w), so $|T| \geq |T_0| \geq |S_{\min}|$.

Uniqueness. If $(T, t_0, step')$ is minimal, then $|T| = |S_{\min}|$ and the surjection is a bijection $\varphi: T \rightarrow S_{\min}$ with $T_0 = T$. If $\varphi(t) = [s]$, then $out'(t, a)$, the first symbol of $r'_t(a) = r_s(a)$, equals $out(s, a) = out_{\min}(\varphi(t), a)$; and $r'_{nxt'(t, a)} = r_{nxt(s, a)}$ (both are the tail of the common response after a), so $\varphi(nxt'(t, a)) = [nxt(s, a)] = nxt_{\min}(\varphi(t), a)$. Thus φ is a homomorphism of pointed coalgebras and, being a bijection, an isomorphism, with $\varphi(t_0) = [s_0]$. Every minimal realization is isomorphic to $M_{\min}$. This is the classical theorem of Nerode [Nerode, 1958]: the quotient by indistinguishability, following Moore [Moore, 1956]: made effective by Hopcroft's refinement [Hopcroft, 1971]. $\square$

Theorem 9.7 (Minimization preserves behavior (Theorem 9.7)). *[C] Minimization preserves behavior (Nerode quotient equivalence) [Nerode, 1958].*

Proof. The states reached by M and $M_{\min}$ on a word w are s_w and $[s_w]$, and every emitted symbol agrees by well-definedness, so $r_{[s_0]} = r_{s_0}$: the minimized machine produces exactly the behavior of M . In the language of Mol's behavioral equivalence (Theorem 3.23), the relation

$$R = \{(s_w, [s_w]) : w \in A^*\}$$

contains the initial pair $(s_0, [s_0])$ and is a bisimulation: related states emit equal outputs on every letter and their successors $(s_{wa}, [s_{wa}])$ are again related. Hence $M \simeq M_{\min}$. This is Nerode's quotient equivalence: minimization quotients the state space by behavioral indistinguishability, and the quotient machine is equivalent to the original. $\square$

Theorem 9.8 (Compositional state bounds (Theorem 9.8)). *[N] For molecules f and g with state spaces S_f and S_g ,*

$$|S_{\min}(g \circ f)| \leq |S_f| \cdot |S_g| \quad \text{and} \quad |S_{\min}(f \otimes g)| \leq |S_f| \cdot |S_g|.$$

Proof. Let f be presented by (S_f, s_f, δ_f) and g by (S_g, s_g, δ_g) .

Sequential. The composite $g \circ f$ is realized by the product machine with state space $S_f \times S_g$, initial state (s_f, s_g) , and transition $\delta_h((u, v), a) = (c, (u', v'))$, where $(b, u') = \delta_f(u, a)$ and $(c, v') = \delta_g(v, b)$ (Theorem 12.1). By Theorem 9.6, the minimal realization has no more states than any realization, so $|S_{\min}(g \circ f)| \leq |S_f \times S_g| = |S_f| \cdot |S_g|$.

Parallel. The tensor $f \otimes g$ processes the components of an input pair (a, b) independently and is realized by the machine with state space $S_f \times S_g$ and transition $\delta_{\otimes}((u, v), (a, b)) = (c, d), (u', v'))$, where $(c, u') = \delta_f(u, a)$ and $(d, v') = \delta_g(v, b)$. The same minimality argument gives $|S_{\min}(f \otimes g)| \leq |S_f| \cdot |S_g|$.

The bounds hold for any presentations of f and g ; they are sharpest when S_f and S_g are minimal, since minimization discards unreachable states of the product machine. $\square$

Theorem 9.9 (Minimization respects behavioral equivalence (Theorem 9.9)). *[N] If $M \simeq M'$, then $S_{\min}(M) \cong S_{\min}(M')$.*

Proof. Let M and M' be presented by (S, s_0, step) and (S', s'_0, step') . Since $M \simeq M'$ (Theorem 3.23), some bisimulation relates s_0 to s'_0 ; related states emit equal outputs on every letter and have related successors, so by induction on $|w|$, $r_{s_0}(w) = r_{s'_0}(w)$ for all w : both molecules realize the same behavior. Their minimizations are minimal realizations of one common behavior, and by the uniqueness clause of Theorem 9.6 any two minimal realizations of the same behavior are isomorphic. Hence $S_{\min}(M) \cong S_{\min}(M')$ as pointed coalgebras. $\square$

Partition refinement and Hopcroft’s algorithm. The Nerode classes of a finite presentation are computable by *partition refinement*: start with the partition of S_0 by the output map $s \mapsto \text{out}(s)$, and repeatedly split any block whose members send some letter into different blocks. By Lemma 9.4, the partition by $\sim$ is the unique coarsest stable partition refining the output partition: it is stable because equivalent states have equivalent successors, and every stable refinement has each block contained in one $\sim$ -class (induction on word length: symbols agree by the output refinement, successor blocks by stability). Hopcroft’s algorithm computes this partition in $O(k n \log n)$ steps for n states and k input letters [Hopcroft, 1971].

The coalgebraic reading. Minimization is categorical: the minimal coalgebra is the image of the unique map into the final coalgebra, the quotient by the kernel of the semantic map being the coarsest quotient coalgebra with the same behavior [Colcombet and Petriřan, 2017, Jacobs and Wiřmann, 2022]; in Mol it is realized concretely by $S_{\min}(M) \cong \text{beh}(S_0)$. Theorem 9.6–Theorem 9.7 give the minimal-state reading of Theorem 7.17 (finite-state molecules realize exactly the regular behaviors [Brzozowski, 1964]); Theorem 9.8 bounds state growth under the constructions of chapter 3; Theorem 9.9 shows minimization is well defined on Mol itself, descending to $\text{Mol}/\simeq$ (Theorem 3.23). In practice this justifies compiling every hot-path molecule to its minimal state space before building dispatch tables: behavior is unchanged (Theorem 9.7), the tables have at most the product of the component sizes (Theorem 9.8), and the result does not depend on how the molecule was written (Theorem 9.9).

Chapter 10

Traced Monoidal Structure, Feedback, and Fixed-Iteration Loops

The symmetric monoidal structure of chapter 3 describes how molecules are joined; the traced monoidal structure of §2.5 describes how a molecule is turned back on itself. This chapter develops the traced structure of Mol and its engineering content: the reactor loop, the feedback laws of Joyal–Street–Verity [Joyal et al., 1996] (yanking, vanishing, sliding, superposing), and the *fixed-iteration loops* of a dataplane: loops whose trip count is decided in advance, unrolled, and therefore branch-predictable.

The quantitative heart of the chapter is contraction. A loop whose state update shrinks distances converges to a unique fixed point at a geometric rate (Theorem 10.5, Banach [Banach, 1922]); the iteration bound Theorem 10.6 bounds the number of unrolled iterations needed to guarantee a tolerance ε ; Theorem 10.8 shows how contraction rates combine under the two composition schemes of Mol, tensor and sequential; and Theorem 10.10 extends the analysis to stochastic state updates, where total-variation distance contracts by the Dobrushin coefficient [Dobrushin, 1956]. The chapter closes with the information-geometric reading of coordinate choice, recorded as future work.

10.1 The Reactor Loop

Feedback is the operation that turns a molecule into a loop. In the terminology of §2.5, a trace takes a morphism $A \otimes U \rightarrow B \otimes U$ and produces a morphism $A \rightarrow B$ by threading the feedback object U through every step. Mol realizes this construction directly.

Definition 10.1 (Reactor loop). Let $M: A \otimes U \rightarrow B \otimes U$ be a molecule of Mol with state space S and step function $step: S \times A \times U \rightarrow B \times U \times S$. The *reactor loop* $\text{Tr}(M): A \rightarrow B$ has state space $S \times U$ and step function

$$step'((s, u), a) = (b, (s', u')) \quad \text{where} \quad step(s, a, u) = (b, u', s').$$

The U -component is the *loop-carried register*: the register value consumed on each step is the value produced by the previous step, so U is genuinely fed back.

Remark 10.2. The construction is total and deterministic, hence a well-defined morphism of Mol. It realizes the trace operation of §2.5; the Joyal–Street–Verity axioms [Joyal et al., 1996] (vanishing, superposing, sliding, yanking) are stated and proved for it as Theorem 8.2–Theorem 8.4 in chapter 8.

Sliding is the hoisting law: a pure transformation moves across the feedback boundary and out of the loop.

A dataplane loop rarely runs to convergence. The loop is *unrolled* for a number of iterations fixed in advance: the trip count is a compile-time constant, there is no convergence test, and the branch predictor always predicts correctly.

Definition 10.3 (Fixed-iteration loop). Let $M: A \otimes U \rightarrow B \otimes U$ be as in Theorem 10.1 and let $k \geq 1$. The *fixed-iteration loop* $L_k(M): A \rightarrow B$ has state space $S \times U$; on input a it applies the state update $T_a: S \times U \rightarrow S \times U$, defined by $T_a(s, u) = (s', u')$ where $step(s, a, u) = (b, u', s')$, exactly k times, emits the output b of the last application, and carries the final register value into the next input. For $k = 1$ this is the reactor loop $\text{Tr}(M)$.

Remark 10.4. The fixed-iteration loop is total and deterministic for every k : fixed trip count, no test on the state, no branch on the data; unrolled, it is straight-line code with perfectly predictable control flow. The quantitative questions (when the register update converges, and how large k must be for the output to be within a tolerance ε of the converged value) are answered next, under the hypothesis that each update T_a is a contraction with rate $\alpha < 1$ uniform in a ; then comes the stochastic analogue.

10.2 Contraction and Convergence

The register update T_a of a fixed-iteration loop is a map on a metric space; when it shrinks distances the loop converges, and at a geometric rate. This is the classical Banach fixed-point principle [Banach, 1922], restated for the loop semantics of Mol.

Theorem 10.5 (Contraction state update, Theorem 10.5 [C]). *Let (S, d) be a complete metric space and let $F: S \rightarrow S$ satisfy $d(Fx, Fy) \leq \alpha d(x, y)$ for some rate $\alpha < 1$ and all $x, y \in S$. Then F has a unique fixed point x^* , and for every $x_0 \in S$ the iterates $x_n = F^n x_0$ converge to x^* , with the tail bound*

$$d(x_n, x^*) \leq \frac{\alpha^n}{1 - \alpha} d(x_0, Fx_0) \quad (n \geq 0).$$

Proof. Uniqueness: if $Fx^* = x^*$ and $Fy^* = y^*$ then $d(x^*, y^*) = d(Fx^*, Fy^*) \leq \alpha d(x^*, y^*)$, so $(1 - \alpha)d(x^*, y^*) \leq 0$ and $x^* = y^*$.

Existence: put $x_n = F^n x_0$ and $d_0 = d(x_0, Fx_0)$. By induction $d(x_n, x_{n+1}) \leq \alpha^n d_0$. For $n < m$,

$$d(x_n, x_m) \leq \sum_{j=n}^{m-1} d(x_j, x_{j+1}) \leq d_0 \sum_{j=n}^{\infty} \alpha^j = \frac{d_0 \alpha^n}{1 - \alpha} \rightarrow 0$$

as $n \rightarrow \infty$; the sequence (x_n) is Cauchy, and completeness yields a limit x^* . Since F is Lipschitz, $Fx^* = \lim Fx_n = \lim x_{n+1} = x^*$. Letting $m \rightarrow \infty$ in the displayed inequality gives the tail bound. $\square$

Theorem 10.6 (Iteration bound, Theorem 10.6 [N]). *Let (S, d) , F , $\alpha < 1$, and x^* be as in Theorem 10.5. Let $x_0 \in S$ with $d_0 = d(x_0, Fx_0) > 0$ and let $\varepsilon > 0$ with $\varepsilon(1 - \alpha) < d_0$. Define*

$$k(\alpha, \varepsilon, d_0) = \left\lceil \frac{\ln(\varepsilon(1 - \alpha)/d_0)}{\ln \alpha} \right\rceil.$$

Then after $k = k(\alpha, \varepsilon, d_0)$ iterations of F starting from x_0 ,

$$d(x_k, x^*) \leq \varepsilon.$$

Consequently the fixed-iteration loop of Theorem 10.3 whose register update T_a is an α -contraction with rate uniform in a is, after k unrolled iterations per input, within ε of the converged loop state; its outputs differ from the converged outputs by at most $L\varepsilon$, where L is the Lipschitz constant of the output map. The loop has no convergence test: the trip count is fixed and branch-predictable.

Proof. By the tail bound of Theorem 10.5, $d(x_k, x^*) \leq d_0 \alpha^k / (1 - \alpha)$. This is at most ε exactly when $\alpha^k \leq \varepsilon(1 - \alpha)/d_0$. Since $\ln \alpha < 0$, taking logarithms gives

$$k \geq \frac{\ln(\varepsilon(1 - \alpha)/d_0)}{\ln \alpha};$$

the ceiling of the right-hand side satisfies this, so $d(x_k, x^*) \leq \varepsilon$. The loop claims follow from Theorem 10.3: the update is applied exactly k times with no intervening test, so the state after the k unrolled iterations is x_k , and the output error is bounded by the Lipschitz constant of the output map. $\square$

Remark 10.7 (Script verification). The bound arithmetic of Theorem 10.6 is *script-verified*: the executable Rust tool `contraction` of Appendix A computes $k(\alpha, \varepsilon, d_0)$ and simulates the worst-case geometric model $x_{n+1} = \alpha x_n$ on the line (fixed point 0, initial value $x_0 = d_0$), in which the residual-based bound of the proof is conservative. For all samples ($\alpha \in \{0.1, \dots, 0.9\}$, $\varepsilon \in \{10^{-3}, 10^{-6}, 10^{-9}\}$, $d_0 \in \{1, 10\}$) it asserts $|x_k| \leq \varepsilon$ (relative tolerance 10^{-12}), and it asserts the monotonicity of k in α and in the tightening of ε . All 54 samples pass; the log is `verify/logs/contraction.log`.

10.3 Contraction Rates under Composition

Loops are joined by the two composition schemes of Mol; a bound on the combined contraction rate is what makes the iteration bound Theorem 10.6 compositional.

Theorem 10.8 (Contraction rates combine, Theorem 10.8 [N]). *Let F_1 and F_2 be state updates of fixed-iteration loops, contractions with rates $\alpha_1, \alpha_2 \in (0, 1)$ in metrics d_1 and d_2 .*

- (i) *Tensor. On the product state space $S_1 \times S_2$ with the sup metric $d_\infty((s_1, s_2), (t_1, t_2)) = \max(d_1(s_1, t_1), d_2(s_2, t_2))$, the tensor update $F_1 \otimes F_2$ is a contraction with rate $\max(\alpha_1, \alpha_2)$, and its fixed point is the pair of fixed points (x_1^*, x_2^*) .*
- (ii) *Sequential. On a common state space S with metric d , the composite $F_2 \circ F_1$ is a contraction with rate at most $\alpha_1 \alpha_2 \leq \alpha_1 + \alpha_2$; the additive rate $\alpha_1 + \alpha_2$ is therefore a valid, conservative sequential rate in the sup metric.*

Proof. (i) For $x = (s_1, s_2)$ and $y = (t_1, t_2)$,

$$\begin{aligned} d_\infty((F_1 \otimes F_2)x, (F_1 \otimes F_2)y) &= \max(d_1(F_1 s_1, F_1 t_1), d_2(F_2 s_2, F_2 t_2)) \\ &\leq \max(\alpha_1 d_1(s_1, t_1), \alpha_2 d_2(s_2, t_2)) \leq \max(\alpha_1, \alpha_2) d_\infty(x, y). \end{aligned}$$

The bound is sharp: for $F_i(x) = \alpha_i x$ on $\mathbb{R}$ with the sup metric on $\mathbb{R}^2$, the rate along the coordinate with the larger α_i equals $\max(\alpha_1, \alpha_2)$. The fixed point is the pair of fixed points by Theorem 10.5.

(ii) For all $x, y \in S$,

$$d(F_2 F_1 x, F_2 F_1 y) \leq \alpha_2 d(F_1 x, F_1 y) \leq \alpha_2 \alpha_1 d(x, y),$$

so $F_2 \circ F_1$ is an $\alpha_1 \alpha_2$ -contraction, and $\alpha_1 \alpha_2 \leq \alpha_1 + \alpha_2$ makes the additive rate a valid bound. Sharpness: for $\alpha_1 = \alpha_2 = \alpha$, affine maps compose to $F_2 \circ F_1(x) = \alpha^2 x + c$ with rate exactly α^2 , strictly below $\alpha_1 + \alpha_2$; the product, not the sum, is the sharp sequential rate. $\square$

Remark 10.9. By Theorem 10.6, the combined loop is within ε after $k(\alpha, \varepsilon, d_0)$ iterations with $\alpha = \max(\alpha_1, \alpha_2)$ under tensor composition and $\alpha = \alpha_1 \alpha_2$ under sequential composition. Fixed-iteration loops are thus closed under both composition schemes of Mol: the loop of a composed molecule is again a converging loop, with an explicitly bounded trip count.

10.4 Stochastic State Updates and the Dobrushin Coefficient

The analysis so far is deterministic. A molecule whose register update is stochastic (a Markov kernel, modeling input-distribution uncertainty, randomized rounding, or probabilistic scheduling) falls under the classical contraction theory of Markov chains [Dobrushin, 1956]: distances between state distributions contract by the Dobrushin coefficient, and the iteration bound of Theorem 10.6 applies with the coefficient in place of α .

Theorem 10.10 (Markov state updates, Theorem 10.10 [C]). *Let S be a finite set and let K be a Markov kernel on S , viewed as a stochastic state update. Let*

$$\delta(K) = \max_{x, y \in S} \|K(x, \cdot) - K(y, \cdot)\|_{\text{TV}}$$

be its Dobrushin coefficient, where $\|p - q\|_{\text{TV}} = \sup_{A \subseteq S} |p(A) - q(A)|$ is the total-variation distance. Then for all probability distributions μ, ν on S ,

$$\|\mu K - \nu K\|_{\text{TV}} \leq \delta(K) \|\mu - \nu\|_{\text{TV}}.$$

If $\delta(K) < 1$, then K has a unique stationary distribution π , and $\|\mu K^n - \pi\|_{\text{TV}} \leq \delta(K)^n \|\mu - \pi\|_{\text{TV}}$ for every $n \geq 0$: the distribution of the stochastic update converges geometrically.

Proof. Write $\sigma = \mu - \nu$ and split it as $\sigma = \sigma^+ - \sigma^-$; then $\sum_x \sigma^+(x) = \sum_x \sigma^-(x) =: c = \frac{1}{2} \sum_x |\sigma(x)| = \|\mu - \nu\|_{\text{TV}}$. If $c = 0$ the claim is trivial, so put $P = \sigma^+/c$ and $Q = \sigma^-/c$, two distributions on S . Since $\sigma K = c(PK - QK)$ with $\sigma K(y) = \sum_x \sigma(x) K(x, y)$,

$$\begin{aligned} \|\sigma K\|_1 &= c \sum_y \left| \sum_{x, z} P(x) Q(z) (K(x, y) - K(z, y)) \right| \\ &\leq c \sum_{x, z} P(x) Q(z) \sum_y |K(x, y) - K(z, y)| \leq c \max_{x, z} \sum_y |K(x, y) - K(z, y)|. \end{aligned}$$

Using $\|p - q\|_{\text{TV}} = \frac{1}{2} \|p - q\|_1$ for probability measures and the definition of $\delta(K)$,

$$\|\mu K - \nu K\|_{\text{TV}} = \frac{1}{2} \|\sigma K\|_1 \leq \frac{1}{2} c \max_{x, z} \|K(x, \cdot) - K(z, \cdot)\|_1 = c \delta(K) = \delta(K) \|\mu - \nu\|_{\text{TV}}.$$

For the second claim: the probability distributions on S form a complete metric space under total variation, and $\mu \mapsto \mu K$ is a contraction with rate $\delta(K) < 1$. By Theorem 10.5 it has a unique fixed point π , the stationary distribution. Inducting on the first claim,

$$\|\mu K^n - \pi\|_{\text{TV}} = \|(\mu K^{n-1})K - \pi K\|_{\text{TV}} \leq \delta(K) \|\mu K^{n-1} - \pi\|_{\text{TV}} \leq \delta(K)^n \|\mu - \pi\|_{\text{TV}}. \quad \square$$

Remark 10.11. A deterministic state update F has Dirac kernels $K(x, \cdot) = \delta_{F(x)}$, and its Dobrushin coefficient is 1 whenever F is not constant: matching the observation that on a discrete state space the only nontrivial contractions are constant maps. The contractive regime $\delta(K) < 1$ is therefore genuinely stochastic: it arises for mixtures of deterministic updates, for example when the register is drawn from a distribution or the update depends on a randomized input. Theorem 10.6 then bounds the number of distributional iterations, with $\alpha = \delta(K)$. This is the point of contact between the loop analysis of this chapter and the Markov-category viewpoint of [Fritz, 2019]: where a Markov category composes stochastic kernels, the fixed-iteration analysis of Mol bounds their convergence.

10.5 Information Geometry as Future Work

The contraction rate of a state update is a property of the chosen metric: rescaling a register, or passing to adapted coordinates, changes α and therefore the iteration bound of Theorem 10.6. Choosing coordinates that minimize $k(\alpha, \varepsilon, d_0)$ is an optimization over metrics, and the geometry of the resulting loop-state manifolds (curvature, geodesics, and the divergence with respect to which the update contracts) has an information-geometric reading. Working this out, together with its interaction with Theorem 10.6 and Theorem 10.8, is future work.

10.6 Summary

Feedback in Mol is the reactor loop of §10.1: the trace of §2.5, with the Joyal–Street–Verity laws [Joyal et al., 1996] proved as Theorem 8.2–Theorem 8.4 in chapter 8. A dataplane loop does not run to convergence; it is unrolled for a fixed trip count, and this chapter proves the trip count is a theorem: convergence to a unique fixed point (Theorem 10.5); the closed form $k(\alpha, \varepsilon, d_0) = \lceil \ln(\varepsilon(1 - \alpha)/d_0) / \ln \alpha \rceil$ for any tolerance, script-verified (Theorem 10.6); the composed rates $\max(\alpha_1, \alpha_2)$ under tensor and $\alpha_1 \alpha_2 \leq \alpha_1 + \alpha_2$ sequentially (Theorem 10.8); and Dobrushin contraction for stochastic updates (Theorem 10.10). A fixed-iteration loop is branch-predictable by construction, within a provable tolerance of its converged value, and closed under the composition of molecules: the engineering practice of unrolled dataplane loops, expressed as a theorem.

Chapter 11

Operadic Composition and Batching

The dominant cost of moving data through a network stack is the boundary crossings: each system call traps into the kernel with a fixed overhead that is largely independent of how many bytes are moved [Power, 2013]. The standard remedy is batching: collect several items, cross the boundary once, process them together. In Mol, batching is an algebraic structure: batches are n -ary operations, the family of all batch operations forms an *operad* in the sense of Leinster [2004], and the composition law of that operad is precisely the reassociation of batches, whose associativity is a theorem about Mol. This chapter proves the three batch theorems: flattening (Theorem 11.1), syscall amortization (Theorem 11.2), and the ring capacity invariant (Theorem 11.3).

The batch construction. Let $f : A \rightarrow B$ be a molecule with a fixed Mealy presentation (S, s_0, δ) , $\delta : S \times A \rightarrow B \times S$ [Mealy, 1955]. For $n \geq 1$ the n -batch of f is the molecule $f^{(n)} : A^n \rightarrow B^n$ with the same state space S , the same initial state s_0 , and the transition function

$$\delta^{(n)}(s, (a_1, \dots, a_n)) = ((b_1, \dots, b_n), s_n), \quad (b_i, s_i) = \delta(s_{i-1}, a_i), \quad s_0 = s.$$

Thus $f^{(n)}$ reads its inputs in order, threading the state of f through the n positions, and emits its outputs in order; a batched dataplane stage is exactly such a machine, one I/O operation delivering the tuple $(a_1, \dots, a_n)$ and collecting $(b_1, \dots, b_n)$. For $n = 2$ we write $\text{batch}(x, y)$ for the element-level reading of $f^{(2)}(x, y)$.

Batching is an operad. An operad is a family of n -ary operations with a composition law that plugs a k_j -ary operation into the j -th slot of an m -ary operation to obtain a $(k_1 + \dots + k_m)$ -ary operation, subject to associativity and unit laws [Leinster, 2004]. For batching the composition is *flattening*: a batch of m batches, the j -th of size k_j , is reassociated to a single batch of size $k_1 + \dots + k_m$, with unit $f^{(1)} = f$. The associativity of this composition is exactly the flattening theorem, Theorem 11.1.

Theorem 11.1 (Batch flattening (Theorem 11.1)). *Let $f : A \rightarrow B$ be a molecule and let $m, k \geq 1$. Under the canonical coherence isomorphisms $\alpha : (A^k)^m \cong A^{km}$ and $\beta : (B^k)^m \cong B^{km}$ of the symmetric monoidal structure of Mol (the reassociation of tuples), the square*

$$\begin{array}{ccc} (A^k)^m & \xrightarrow{\alpha} & A^{km} \\ \downarrow & & \downarrow \\ (B^k)^m & \xrightarrow{\beta} & B^{km} \end{array}$$

whose vertical arrows are $(f^{(k)})^{(m)}$ and $f^{(km)}$ commutes in *Mol*; equivalently $(f^{(k)})^{(m)} = \beta^{-1} \circ f^{(km)} \circ \alpha$. At the level of elements with $m = k = 2$ this is the identity

$$\text{batch}(\text{batch}(a, b), \text{batch}(c, d)) = \text{batch}(a, b, c, d).$$

Proof. Both morphisms are presented by machines with the same state space S and the same initial state s_0 . Unwinding the batch construction, $(f^{(k)})^{(m)}$ is the machine whose transition on a block-structured input in $(A^k)^m$ applies δ exactly once to each element in the flattened order $a_{1,1}, a_{1,2}, \dots, a_{m,k}$ and emits the outputs in the same order; the outer and the inner batches only group these applications, and the state threads across blocks because $f^{(k)}$ returns the state after each block. The machine $f^{(km)}$ applies δ once to each element of $\alpha(a_{1,1}, \dots, a_{m,k})$ in the same order with the same state threading, so on corresponding inputs the two machines traverse identical state sequences and emit identical outputs: they are the same machine up to the renaming of the input and output objects by α and β , and *Mol* identifies machines up to state-space isomorphism (Theorem 3.8). The element-level identity for $m = k = 2$ is the special case $a_{1,1} = a$, $a_{1,2} = b$, $a_{2,1} = c$, $a_{2,2} = d$. $\square$

Remark. This associativity makes batched pipelines reassociable without behavioral change (the rewrite engine of chapter 5 uses the batch-flattening rule (Theorem 5.5)) and it makes the operad of batches the terminal operad: each $O(n)$ is a single operation, so no further laws are needed [Leinster, 2004, Kelly, 1974].

Cost accounting. The cost model of chapter 7 assigns each molecule a vector (time, memory, cache misses), additive under composition (Theorem 7.3); here only the time component is needed. Let t_s be the fixed cost of one system call (trap, dispatch, return) and t_p the per-item cost of processing one element once the batch has crossed the boundary. These are constants of the platform and of the atom f , independent of the batch size as long as the batch fits in a preallocated buffer; the exact bound is fixed by Theorem 11.3.

Theorem 11.2 (Syscall amortization (Theorem 11.2)). *Let a single I/O item cost $\text{cost}(\text{single}) = t_s + t_p$ (one system call plus per-item processing). Then the n -batch satisfies*

$$\text{cost}(f^{(n)}) = t_s + n \cdot t_p \leq n \cdot (t_s + t_p) = n \cdot \text{cost}(\text{single}),$$

and with $\text{fixed}(n) = t_s$ the catalog form $\text{cost}(f^{(n)}) \leq n \cdot \text{cost}(\text{single}) + \text{fixed}(n)$ holds, where $\text{fixed}(n)/n = t_s/n \rightarrow 0$. Equivalently, the amortized per-item cost of a batch tends to the bare processing cost:

$$\frac{\text{cost}(f^{(n)})}{n} = t_p + \frac{t_s}{n} \longrightarrow t_p.$$

Proof. By construction $f^{(n)}$ crosses the boundary exactly once, with the whole tuple $(a_1, \dots, a_n)$, so it pays the syscall cost t_s once; each of the n items costs t_p , and the per-item costs are additive in the number of items, giving $\text{cost}(f^{(n)}) = t_s + nt_p$. The single operation is the case $n = 1$, so $\text{cost}(\text{single}) = t_s + t_p$. Since $t_s \geq 0$, $t_s + nt_p \leq n(t_s + t_p)$ for every $n \geq 1$, which is the first inequality and implies the catalog form with $\text{fixed}(n) = t_s$. The identities for the amortized cost and the limits $t_s/n \rightarrow 0$ are immediate. The constants are valid exactly for $n \leq 2^k - 1$ by the capacity invariant (Theorem 11.3); the limit is over growing configurations, with k chosen so that $2^k \geq n + 1$. $\square$

Rings. Batches move through preallocated ring buffers. A ring of capacity $N = 2^k$ is an array of N slots addressed by a read pointer r and a write pointer w , kept as unbounded monotone counters and mapped into the array by the mask $i \mapsto i \bmod N$; because N is a power of two the mask is a single bitwise-and instruction $i \& (N - 1)$, which is why ring capacities are powers of two. With $w - r \geq 0$ items in flight, a read removes one item at r and a write inserts one item at w , refusing accesses that would exceed the occupancy bound. The only bookkeeping question is whether the masked pointers alone distinguish *empty* from *full*.

Theorem 11.3 (Ring capacity invariant (Theorem 11.3)). *Let a ring have $N = 2^k$ slots, $k \geq 1$, indexed by the bitmask $i \mapsto i \bmod N$. Bitmask indexing is correct (the predicate $\text{empty} \iff r \bmod N = w \bmod N$ is sound, and the full state is distinguishable from the empty state) if and only if the number of items in flight satisfies $w - r \leq N - 1 = 2^k - 1$ at all times.*

Proof. The masked pointers determine the pointer difference only modulo N : $r \bmod N = w \bmod N$ iff $w - r \equiv 0 \pmod{N}$, which for $w \geq r$ means $w - r \in \{0, N, 2N, \dots\}$. If $w - r \leq N - 1$ at all times, then $w - r \equiv 0 \pmod{N}$ forces $w - r = 0$, so masked pointer equality is exactly emptiness, and the full state ($w - r = N - 1$, write refused) has mask difference $N - 1 \neq 0$ and is distinguishable: indexing is correct. Conversely, if $w - r = N$ is ever reached, the ring is full, yet $r \bmod N = w \bmod N$ reproduces the mask pattern of the empty state, so no test on masked pointers can separate full from empty: correctness fails. Hence correctness forces $w - r \leq N - 1$ at all times. $\square$

Remark. The invariant is the mathematical content of policy [R] of the software standard (power-of-two capacity, bitmask indexing, bounded in-flight count) and it bounds the batches to which Theorem 11.2 applies: a batch must fit in the ring, $n \leq 2^k - 1$, or the fixed cost t_s is no longer constant.

Chapter 12

Linear Logic, Cut Elimination, and Zero Allocation

Linear logic is the logic of resources used exactly once [Girard, 1987]: its formulas cannot be duplicated or discarded freely, and its cut rule composes proofs exactly as Mol composes molecules. The reason a dataplane can promise zero allocation on the hot path is that a dataplane is a linear program (every datum is produced once and consumed once) and the intermediate structures a classical program would materialize are eliminated by fusion. That elimination is cut elimination, known in programming as deforestation [Wadler, 1989]. This chapter proves that cut elimination fuses every pure pipeline into a single molecule (Theorem 12.1), that linear atoms force zero allocation (Theorem 12.2), and records the geometry-of-interaction and Curry–Howard readings of these results.

Pipelines and cuts. A pure pipeline is a composite $p = g_n \circ \cdots \circ g_1 : A_0 \rightarrow A_n$ of molecules $g_i : A_{i-1} \rightarrow A_i$. In the categorical semantics of linear logic, each composition point $g_{i+1} \circ g_i$ is a *cut*: a proof of $A_{i-1} \rightarrow A_i$ glued to a proof of $A_i \rightarrow A_{i+1}$ across the intermediate formula A_i . Cut elimination rewrites a proof until no cut remains; its semantic content is that the composite morphism is computed directly, without ever constructing the intermediate object of type A_i [Girard, 1987]. In a dataplane the intermediate object is a materialized buffer, and cut elimination is deforestation: the buffer is fused away [Wadler, 1989].

Theorem 12.1 (Cut elimination as deforestation (Theorem 12.1)). *Every pure pipeline $p = g_n \circ \cdots \circ g_1 : A_0 \rightarrow A_n$ in Mol is equal to a single molecule $h : A_0 \rightarrow A_n$, the fused pipeline, with the following properties:*

1. $h \simeq p$: fusion preserves behavior;
2. the intermediate types $A_1, \dots, A_{n-1}$ never materialize as buffers during the execution of h : no datum of type A_i is stored between stages;
3. h is constructed from the presentations of the g_i by an explicit finite procedure of worst-case complexity $O(|S_1| \cdots |S_n| \cdot (|A_0| + \cdots + |A_{n-1}|))$, where $|S_i|$ is the size of the state space of g_i .

Proof. First fuse two molecules. Let $f : A \rightarrow B$ and $g : B \rightarrow C$ be presented by (S_f, s_f, δ_f) and (S_g, s_g, δ_g) . Define $h : A \rightarrow C$ with state space $S_f \times S_g$, initial state (s_f, s_g) , and transition

$$\delta_h((u, v), a) = (c, (u', v')), \quad \text{where } (b, u') = \delta_f(u, a), \quad (c, v') = \delta_g(v, b).$$

This is a valid Mealy machine, and by the definition of composition in Mol (chapter 3) it is exactly the composite $g \circ f$: on every input word, h follows the same state trajectory $(u, v) \mapsto (u', v')$ and emits the same output word as the two-stage machine, so $h \simeq g \circ f$. In a single transition of h , the value b is produced by δ_f and consumed by δ_g within that same transition; it is held in a register and never stored, so no buffer of type B exists in this realization.

Now iterate. By induction on n , the pipeline $g_{n-1} \circ \dots \circ g_1$ is equal to a single molecule $h' : A_0 \rightarrow A_{n-1}$ with state space $S_1 \times \dots \times S_{n-1}$ and no intermediate buffers; fusing h' with g_n by the two-molecule construction gives the single molecule h with state space $S_1 \times \dots \times S_n$, behavior equal to p , and no buffer of any type A_i ($1 \leq i \leq n-1$). The transition table of h has $|S_1| \cdots |S_n| \cdot |A_0|$ entries, each computed by one step of every g_i (work $O(|A_0| + \dots + |A_{n-1}|)$), giving the stated construction cost. This is the cut-elimination theorem of linear logic [Girard, 1987] realized concretely in Mol; in programming terms it is deforestation, the elimination of the intermediate data structure [Wadler, 1989]. $\square$

Remark (fusion cost). The fused machine is a morphism of Mol, so by the additivity of the cost model (Theorem 7.3) its cost is at most the sum of the component costs; because the intermediate value is transient, the buffer copy that a two-stage realization pays is absent, which is the content of the fusion bound (Theorem 7.8): $c(g \circ f) \leq c(f) + c(g)$, strict when the intermediate type never materializes. Theorem 12.1 supplies the mechanism behind that bound.

Linearity and ownership. Call an atom $f : A \rightarrow B$ *linear* if for every state s and every input a in its domain, the transition $(b, s') = \delta(s, a)$ reads a exactly once and produces exactly one output b . A linear atom never copies its input, never drops it, and never holds it past the transition that consumes it; at every instant each datum is *owned* by exactly one atom. This is the ownership discipline of Rust restated as a property of molecules.

Theorem 12.2 (Linearity forces zero allocation (Theorem 12.2)). *Let M be a molecule built from linear atoms by the operations $\circ$, $\otimes$, and the batch construction of chapter 11. Then:*

1. *M is linear: it consumes every input datum exactly once;*
2. *along every execution of M the number of live data tokens is invariant: each transition consumes exactly one token and produces exactly one token;*
3. *consequently, if the buffers of M are preallocated to their maximum occupancy (bounded by the ring invariant, Theorem 11.3), the hot path performs no allocation and no deallocation.*

Proof. (i) By induction on the construction of M . An atom is linear by hypothesis. Composite: in $g \circ f$, an input token a is read exactly once by f (linearity), producing exactly one token b ; b is read exactly once by g (linearity), producing exactly one token c ; so the composite reads a exactly once and emits exactly one output. Tensor: $f \otimes g$ on a pair (a, a') reads a exactly once in f and a' exactly once in g and emits exactly one pair. Batch: $f^{(n)}$ reads each of its n inputs exactly once by the definition of the batch transition and emits exactly n outputs. Every molecule is obtained by finitely many such steps, so (i) holds.

(ii) Fix an execution and let $L(t)$ be the number of data tokens in the buffers and in flight at time t . By (i) every transition consumes exactly one token and produces exactly one token, so L is unchanged by every transition: $L(t) = L(0)$ for all t .

(iii) By (ii) occupancy never exceeds $L(0)$. If each stage buffer is preallocated with capacity at least its share of $L(0)$ (a ring of capacity $2^k \geq L(0) + 1$ per stage satisfies the in-flight bound

$w - r \leq 2^k - 1$ of Theorem 11.3), then every write fits in preallocated storage and every read empties a slot in place. No token is copied, created, or destroyed, so the execution acquires and releases no heap memory: zero allocations and zero deallocations on the hot path. $\square$

Geometry of interaction. In Girard’s geometry of interaction a proof is interpreted as an operator, and cut elimination is the *execution* of that operator: a token travels through the proof, and cuts are crossed by iterating the operator until the token exits [Girard, 1987, Seiller, 2015]. The reading in Mol is literal: a datum is a token, an atom is a transition δ that carries the token from its input port to its output port, and a molecule is the composition of these port-to-port maps. The fusion theorem (Theorem 12.1) is the execution: it computes in one pass the effect of the whole pipeline on any token, without materializing the cut formulas. The token interpretation is also the performance claim: the working set is the set of live tokens, invariant by Theorem 12.2, which bounds the cache-miss component of the cost vector of chapter 7.

Curry–Howard for molecules. Read propositions as types and proofs as molecules: the linear implication $A \multimap B$ is the type of molecules $A \rightarrow B$, and composition is the cut rule; a well-typed dataplane element is a proof of its interface type. The theorems of this chapter upgrade this typing discipline to a resource guarantee: if the type system records that every atom is linear (the hypothesis of Theorem 12.2) and that fusion removes intermediate structure (Theorem 12.1), then a well-typed hot path is allocation-free by construction. This is the categorical content of ownership in Rust: a discipline of exact-once consumption is precisely the data that parts (ii)–(iii) of Theorem 12.2 require, so the guarantee is enforced at compile time rather than by runtime measurement [Wadler, 1989].

12.1 The Affine Type System and the No-Allocation Theorem

The ownership discipline of Theorem 12.2 is stated as a semantic hypothesis: atoms are assumed to consume their input exactly once. This section shows that the hypothesis can be *enforced* rather than assumed, by a small affine type system Λ whose well-typed terms are exactly the molecules of Mol built by composition and tensor. The system is affine in the sense of the affine fragment of linear logic: every variable may be used at most once, so no term can copy or drop a datum, and reduction (the operational semantics of Λ) never duplicates a subterm and never allocates a fresh node. We prove this as Theorem 12.6, the no-allocation theorem, which sharpens Theorem 12.2 by replacing its semantic hypothesis with a typing discipline.

Definition 12.3 (Affine type system). The types of Λ are generated from a fixed set $\mathbf{B}$ of *base types* by the grammar

$$A, B ::= \beta \mid A \multimap B \mid A \otimes B \mid I, \quad \beta \in \mathbf{B},$$

where $A \multimap B$ is the linear function type, $A \otimes B$ the tensor, and I the unit. A *linear context* is a finite partial map from variables to types; contexts are combined by the disjoint union Γ, Δ , defined only when the domains of Γ and Δ are disjoint, and the rules below ensure that no derivable context assigns a type to any variable twice. The *terms* are generated by the grammar

$$t, u ::= x \mid a \mid t; u \mid t \otimes u \mid (),$$

where x ranges over variables and a ranges over a fixed *signature* Σ of *atom constants* $a : A \multimap B$ (the atom library of chapter 3); $t; u$ is *composition* (the cut), $t \otimes u$ the tensor of terms, and $()$

the unit term. The typing rules derive judgments $\Gamma \vdash t : A$; they are the rules of multiplicative intuitionistic linear logic restricted to this syntax:

(var) $x : A \vdash x : A$;

(atom) $\vdash a : A \multimap B$ for every atom $a : A \multimap B$ in Σ ;

(comp) from $\Gamma \vdash t : A \multimap B$ and $\Delta \vdash u : A$ with Γ and Δ disjoint, $\Gamma, \Delta \vdash t; u : B$;

(tensor) from $\Gamma \vdash t : A$ and $\Delta \vdash u : B$ with Γ and Δ disjoint, $\Gamma, \Delta \vdash t \otimes u : A \otimes B$;

(unit) $\vdash () : I$.

There are no rules of weakening or contraction: a variable cannot be dropped and cannot be duplicated. A *molecule term* is a closed well-typed term, i.e. a derivable judgment $\vdash t : A$ with empty context; these are the molecules of Mol in the fragment generated by composition and tensor.

Definition 12.4 (Operational semantics of Λ). The reduction relation $t \longrightarrow t'$ on well-typed terms is the smallest relation closed under the term constructors (if $t \longrightarrow t'$ then $t; u \longrightarrow t'; u$, $u; t \longrightarrow u; t'$, $t \otimes u \longrightarrow t' \otimes u$, and $u \otimes t \longrightarrow u \otimes t'$) and containing the following redex rules.

(CE) *Composition elimination* (associativity of the cut): $(t; u); v \longrightarrow t; (u; v)$.

(TE) *Tensor elimination* (interchange): for $t : A \multimap B$, $u : C \multimap D$, $v : B \multimap E$, $w : D \multimap F$,

$$(t \otimes u); (v \otimes w) \longrightarrow (t; v) \otimes (u; w).$$

(UE) *Unit elimination*: $t \otimes () \longrightarrow t$ and $() \otimes t \longrightarrow t$.

Rule (CE) reassociates a pipeline so that composition is flattened; rule (TE) is the interchange law of Theorem 4.7, eliminating a cut whose two premises are tensor introductions by pushing the cut into its components; rule (UE) eliminates a tensor with the unit. Every rule consumes exactly one redex (the displayed term occurrence) and its right-hand side is built from subterms of that redex. We work up to the strict-monoidal convention for the unit, identifying $A \otimes I \equiv A \equiv I \otimes A$ (the convention of Mol, whose tensor unit is the empty tuple) so that the unit rules preserve types.

Lemma 12.5 (Subject reduction). *If $\Gamma \vdash t : A$ and $t \longrightarrow t'$, then $\Gamma \vdash t' : A$.*

Proof. It suffices to check the redex rules; the congruence steps then follow by the typing rules. Rule (CE): if $(t; u); v : B$, then $t : A \multimap B'$, $u : A$, and $v : B' \multimap B$ for some A, B' ; the reduct $t; (u; v)$ derives the same type B by two applications of (comp). Rule (TE): with the displayed types, the left-hand side has type $(A \otimes C) \multimap (E \otimes F)$; the reduct $(t; v) \otimes (u; w)$ has components of type $A \multimap E$ and $C \multimap F$ by (comp), hence the same type by (tensor). Rule (UE): $t \otimes ()$ has type $A \otimes I$, identified with A by the strict-monoidal convention, and the reduct t has type A ; symmetrically for $() \otimes t$. Every redex rule therefore preserves the judgment, and induction on the derivation of the reduction (the congruence rules compose derivations of the same judgment) gives subject reduction. $\square$

Theorem 12.6 (The no-allocation theorem, Theorem 12.6 [N]). *] Let Λ be the affine type system of Theorem 12.3 with the operational semantics of Theorem 12.4. Then:*

- (i) *Linearity invariant. In every derivation of a judgment $\Gamma \vdash t : A$, every variable of Γ occurs exactly once in t : each free variable is consumed by exactly one rule instance.*

- (ii) No duplication. *Every reduction rule of the operational semantics is non-duplicating: no rule copies a subterm.*
- (iii) No allocation. *The evaluation of every well-typed closed molecule term is a finite reduction sequence, and no step of it allocates fresh memory: every step consumes one redex and produces only subterms already present, so the set of allocated addresses is bounded by the initial term; consequently well-typed molecules run with zero heap allocation.*
- (iv) *This sharpens Theorem 12.2: there the exact-once hypothesis is assumed of the atoms; here it is enforced by the type system rather than assumed. Rust's ownership (move semantics as linear consumption, borrows as read-only views that carry no resource) is the operational realization of Λ .*

Proof. (i) By induction on the derivation of $\Gamma \vdash t : A$. Rule (var): the judgment is $x : A \vdash x : A$, and the variable x occurs exactly once in x . Rules (atom) and (unit): the context is empty, so there is nothing to check. Rule (comp): the derivation ends in $\Gamma, \Delta \vdash t; u : B$ with the domains of Γ and Δ disjoint; by the induction hypothesis every variable of Γ occurs exactly once in t and every variable of Δ exactly once in u , and since the domains are disjoint, every variable of Γ, Δ occurs exactly once in the composite $t; u$, whose free variables are exactly the union of the two domains. Rule (tensor) is the same argument with $t \otimes u$ in place of $t; u$. This is precisely where the disjointness conditions act: without them, a variable could occur in both premises and hence twice in the conclusion, i.e. be consumed twice. The same induction shows that every derivable context assigns each variable at most one type, so the system admits neither contraction nor weakening, and the invariant holds for every well-typed term, closed or open. (ii) Case analysis on the reduction rules of Theorem 12.4. Rule (CE): the redex $(t; u); v$ contains the subterms t, u, v once each, and the reduct $t; (u; v)$ contains exactly those subterms, once each: no subterm is copied. Rule (TE): the redex $(t \otimes u); (v \otimes w)$ contains t, u, v, w once each; the reduct $(t; v) \otimes (u; w)$ contains exactly the same four subterms, once each. Rule (UE): the redex $t \otimes ()$ (or $() \otimes t$) contains t once and the closed unit term $()$; the reduct is t : nothing is copied, and the only subterm erased is the closed unit term, which carries no variable. Hence no rule copies a subterm. The linearity invariant is what makes this structural fact type-theoretically necessary: by subject reduction (Theorem 12.5) every reduct of a well-typed term is well-typed, and by (i) a well-typed term uses every free variable exactly once; a duplicating step would produce a well-typed term with some free variable occurring twice, contradicting (i). So the reduction rules of Λ are exactly the non-duplicating ones, and reduction in Λ never duplicates data. (iii) We first show that every reduction sequence is finite. Define three measures on terms: $|t|$, the number of nodes of the syntax tree of t ; $p(t)$, the number of pairs (m, n) of composition nodes of t such that m lies in the left subtree of n ; and $q(t)$, the number of pairs (m, n) with m a tensor node and n a composition node such that m lies in the left subtree of n . Order these lexicographically, $M(t) = (q(t), p(t), |t|)$, compared componentwise from the first. All three measures are additive over subterms, so it suffices to check the redex rules. Write c_x for the number of composition nodes of x and d_x for the number of tensor nodes. Rule (CE): on the left-hand side $(t; u); v$ the outer composition node has left subtree $(t; u)$, containing $c_t + c_u$ composition nodes and $d_t + d_u$ tensor nodes, and the inner node has left subtree t ; on the right-hand side $t; (u; v)$ the outer node has left subtree t and the inner node left subtree u . Hence q decreases by d_t , and if $d_t = 0$ then p decreases by $c_t + 1 \geq 1$; in either case M strictly decreases. Rule (TE): on $(t \otimes u); (v \otimes w)$ the unique composition node has left subtree $t \otimes u$, containing $d_t + d_u + 1$ tensor nodes; on $(t; v) \otimes (u; w)$ the two composition nodes have left subtrees t and u , contributing $d_t + d_u$; so q decreases by exactly 1. Rule (UE): the measures q and p are unchanged, but $|t \otimes ()| = |t| + 2 > |t|$ (and symmetrically for $() \otimes t$), so the third component strictly decreases.

Every reduction step therefore strictly decreases the lexicographic triple $M(t) \in \mathbb{N}^3$, and $\mathbb{N}^3$ with the lexicographic order is well-founded, so no infinite reduction sequence exists: the evaluation of every well-typed closed molecule term is a finite sequence $t_0 \longrightarrow t_1 \longrightarrow \cdots \longrightarrow t_k$. It remains to show that no step of the sequence allocates. Each redex rule replaces the redex by a term whose nodes are all nodes of the redex: rule (CE) reuses the subterms t, u, v ; rule (TE) reuses t, u, v, w ; rule (UE) reuses t and erases the closed unit term; and the congruence rules only relocate the redex deeper into the term. Hence by induction along the sequence, every node of every reduct t_j is a node (a subterm occurrence) of the initial term t_0 . The allocation count of the step $t_j \longrightarrow t_{j+1}$, defined as the number of fresh nodes it introduces, is therefore 0 for every j : every step consumes one redex and produces only subterms already present. The set of addresses occupied during the evaluation is consequently contained in the set of addresses of t_0 and bounded by $|t_0|$, so the peak live memory is bounded by the initial term, and well-typed molecules run with zero heap allocation. $\square$

Ownership in Rust. Item (iv) is the content of the theorem for the runtime. In Theorem 12.2, exact-once consumption is a hypothesis to be verified of each atom; Theorem 12.6 upgrades it to a consequence of typing, so it holds of every well-typed molecule by construction, before the program runs. Rust’s ownership realizes Λ operationally. *Move* semantics are linear consumption: a moved value is transferred to exactly one new owner and the old binding is invalidated: the (comp) rule passing a datum from the producer that owns it to the consumer that consumes it, each free variable consumed by exactly one rule instance. *Borrows* are read-only views: a borrow carries no ownership and hence no resource obligation, so a borrowed datum is not a variable occurrence of the term and is not counted by the linearity invariant; the borrow checker’s rule that a value cannot be moved while borrowed is the dynamic shadow of the disjointness conditions of the (comp) and (tensor) rules. Where Theorem 12.2 proves that linear atoms force zero allocation given preallocated buffers, Theorem 12.6 proves the type-level content: well-typed molecules are linear by construction, their evaluation duplicates nothing and allocates nothing, and cut elimination as deforestation (Theorem 12.1) is the composition-elimination rule (CE), which computes a fused pipeline without materializing the cut formula [Wadler, 1989].

Remark (scope of the guarantee). Theorem 12.6 is a guarantee about the fragment of Mol generated by composition, tensor, and the unit: the affine core of the language of chapter 3. It is the type-level counterpart of the buffer-level guarantee of Theorem 12.2: the type system makes exact-once consumption structural, so the allocation-free execution of Theorem 12.2 (iii) is available to every well-typed molecule without per-atom verification. This is the ownership discipline of Rust realized as an affine type system.

Chapter 13

Situations and Applications

A *situation* is a requirement (an interface, a behavior, a budget) and a *solution* is a molecule that meets it. The decision matrices of the software standard are exactly the formal optimization over the set of solutions; the two theorems of this chapter make that set finite and decidable (Theorem 13.1) and make zero-copy elimination sound (Theorem 13.2).

Situations. Fix the finite type universe of the dataplane and the cost model of chapter 7, which assigns every molecule a cost vector $\text{cost}(M) = (t(M), m(M), c(M))$ (time, memory, cache misses) additive under composition (Theorem 7.3) and monotone under refinement (Theorem 7.11).

A *situation* is a triple $\sigma = (A \rightarrow B, \Phi, \bar{c})$:

- $A \rightarrow B$: the interface, with finite input type A and finite output type B (*bounded types*);
- Φ : a behavioral specification, a decidable predicate on molecules $A \rightarrow B$ invariant under behavioral equivalence: e.g. “ M realizes the regular language L ” (decidable by minimization, chapter 9);
- $\bar{c} = (c_T, c_M, c_C)$: a cost budget vector, where the memory component counts the states of the canonical presentation, $m(M) = |S_M|$.

A molecule $M : A \rightarrow B$ is a *solution* of σ , written $M \models \sigma$, when $M \vdash \Phi$ and $\text{cost}(M) \leq \bar{c}$ (componentwise); the feasible set is $F(\sigma) = \{M : A \rightarrow B \mid M \models \sigma\}$. A *decision matrix* for σ is the computation

$$M^* = \arg \min_{M \in F(\sigma)} \text{cost}(M),$$

returning *infeasible* when $F(\sigma) = \emptyset$. Under the refinement preorder of chapter 7 the feasible set is a down-set: refinement preserves behavior and is cost-monotone (Theorem 7.11), so $M' \sqsubseteq M \models \sigma$ implies $M' \models \sigma$; the decision matrix therefore optimizes downward through the refinement poset, and by maximal refinement (Theorem 7.13) its output is the most-refined feasible molecule.

Theorem 13.1 (Situation solvability (Theorem 13.1)). *Let $\sigma = (A \rightarrow B, \Phi, \bar{c})$ be a situation with finite types A, B , a decidable specification Φ , and a memory budget c_M that bounds state size via $m(M) = |S_M|$. Then:*

1. *the feasible set $F(\sigma)$ is finite;*
2. *membership in $F(\sigma)$ is decidable: for every molecule $M : A \rightarrow B$, both $M \vdash \Phi$ and $\text{cost}(M) \leq \bar{c}$ are decidable predicates;*

3. the decision-matrix computation, the minimization over $F(\sigma)$, terminates.

Proof. (i) A solution satisfies $m(M) \leq c_M$, hence $|S_M| \leq K$ with $K = \lfloor c_M \rfloor$. Fix a state set $S = \{1, \dots, K\}$: a Mealy machine $A \rightarrow B$ with state space S is determined by its initial state $s_0 \in S$ and its transition $\delta : S \times A \rightarrow B \times S$, so there are at most $K \cdot (|B| \cdot K)^{K \cdot |A|}$ such machines, a finite number. The molecules of Mol are the state-isomorphism classes of such machines, and a quotient of a finite set is finite; since every solution has at most K states, $F(\sigma)$ is finite.

(ii) For a fixed molecule M , presented finitely as a Mealy machine, the predicate $M \vdash \Phi$ is decidable by hypothesis on Φ . The cost $\text{cost}(M)$ is computed from the declared constant cost vectors of the atoms of M by the additive rules (Theorem 7.3): composition adds componentwise and the tensor combines components; the result is a vector of rationals, and comparing it with $\bar{c}$ is decidable. Hence membership in $F(\sigma)$ is decidable.

(iii) By (i) the candidate set is finite and by (ii) membership is decidable, so enumerating $F(\sigma)$ and taking the minimum (or reporting infeasibility when $F(\sigma)$ is empty) terminates. $\square$

Well-formed domains and zero copy. Let B be the byte-stream type and T a structured type. A parser is a molecule $\text{parse} : B \rightarrow T$ and a serializer is a molecule $\text{serialize} : T \rightarrow B$. A subset $D \subseteq T$ is *well-formed* when the roundtrip property holds on it: $\text{parse} \circ \text{serialize} = \text{id}$ on D , that is, $\text{parse}(\text{serialize}(t)) = t$ for every $t \in D$. The *well-formed byte domain* is $W = \text{serialize}(D) \subseteq B$.

Theorem 13.2 (Zero-copy roundtrip (Theorem 13.2)). *Let $\text{parse} : B \rightarrow T$ and $\text{serialize} : T \rightarrow B$ be molecules with $\text{parse} \circ \text{serialize} = \text{id}$ on a well-formed domain $D \subseteq T$, and let $W = \text{serialize}(D)$. Then:*

1. $\text{serialize} \circ \text{parse} = \text{id}$ on W ;
2. for every molecule $X : T \rightarrow T$ with $X(D) \subseteq D$ and $X = \text{id}$ on D , the roundtrip segment $\text{serialize} \circ X \circ \text{parse} : B \rightarrow B$ is the identity on W ;
3. the elimination is sound: if $X \neq \text{id}$ on D , then $\text{serialize} \circ X \circ \text{parse} \neq \text{id}$ on W , so replacing the segment by a pass-through would change behavior on well-formed traffic.

Proof. (i) For $b \in W$, write $b = \text{serialize}(t)$ with $t \in D$. The roundtrip property gives $\text{parse}(b) = t$, hence

$$(\text{serialize} \circ \text{parse})(b) = \text{serialize}(\text{parse}(b)) = \text{serialize}(t) = b.$$

(ii) For $b \in W$, write $b = \text{serialize}(t)$ with $t \in D$. Since $\text{parse}(b) = t$ and $X = \text{id}$ on D ,

$$(\text{serialize} \circ X \circ \text{parse})(b) = \text{serialize}(X(\text{parse}(b))) = \text{serialize}(X(t)) = \text{serialize}(t) = b.$$

(iii) The roundtrip property makes serialize injective on D : if $t, t' \in D$ and $\text{serialize}(t) = \text{serialize}(t')$, then $t = \text{parse}(\text{serialize}(t)) = \text{parse}(\text{serialize}(t')) = t'$. If $X \neq \text{id}$ on D , take $t \in D$ with $X(t) \neq t$; since $X(D) \subseteq D$, both t and $X(t)$ lie in D , and injectivity gives $\text{serialize}(X(t)) \neq \text{serialize}(t)$. For $b = \text{serialize}(t) \in W$: $(\text{serialize} \circ X \circ \text{parse})(b) = \text{serialize}(X(t)) \neq \text{serialize}(t) = b$, so the segment differs from the identity on well-formed input and eliminating it would change behavior. $\square$

Remark. This is the license for decision matrix D-6 (zero-copy vs. copy): when X is a no-op on the well-formed domain, Theorem 13.2 guarantees that a proxy's parse-process-serialize segment may be replaced by a pass-through that forwards the buffer (no parse, no serialize, no allocation) while behavior on well-formed traffic is preserved; when X moves within D , part (iii) shows the copy is semantically necessary.

Precomputed dispatch. By Theorem 13.1 the decision matrix is a finite computation at design time, and its output is a finite table of (discriminant, molecule) pairs. At runtime, dispatch is a lookup keyed by a dense integer discriminant: the precomputed lookup table of chapter 6 (Theorem 6.9): no dynamic dispatch, no search, a single indexed jump. The decision-matrix output is thus exactly the runtime dispatch table.

Canonical situations. The software standard treats four situations as worked recipes; each is a special case of the formal setup above.

- **Web server (HTTP over TCP).** B = TCP byte stream, T = HTTP request; Φ : RFC-conformant request framing; $\bar{c}$: bounded per-request latency and memory. The solution is a pipeline of linear atoms (Theorem 12.2): a ring-fed parser (Theorem 11.3), a framing atom, a dispatch atom (Theorem 13.1 lookup); the batch size is set by the amortization curve of Theorem 11.2 (matrix D-4), and for proxies Theorem 13.2 makes a no-op relay a pass-through.
- **DNS (UDP and TCP).** Two transport modes, selected by matrix D-3; DNS messages are small, so batches are bounded by the ring invariant and the amortization law (Theorem 11.2, Theorem 11.3), and the response molecule is precomputed by Theorem 13.1.
- **FTP (control and data).** Two coupled pipelines: a session state machine over the control channel, a bulk transfer path over the data channel, joined by a shared session-state atom; the memory budget is split between the two rings (Theorem 11.3).
- **Custom protocol.** A fixed binary protocol gives a finite interface: Theorem 13.1 enumerates the feasible set, Theorem 6.9 precomputes the dispatch table, and the parser/serializer pair is inverted on its well-formed domain (Theorem 13.2, zero copy) or kept with copy.

The decision matrices of the standard and the results that govern them are collected in Table 13.1.

Matrix	Decision	Governing result
D-1	Ring/buffer sizing (L3-aware)	Theorem 11.3
D-2	SoA vs. AoS layout	cost model, chapter 7
D-3	Protocol selection (UDP/TCP/SCTP)	Theorem 13.1 (feasibility)
D-4	Batch size	Theorem 11.2
D-5	Polling: busy-poll vs. SQPOLL vs. AF_XDP	Theorem 11.2
D-6	Zero-copy vs. copy	Theorem 13.2
D-7	Dispatch: monomorphization vs. indirect	Theorem 6.9
D-8	SIMD width	cost model, chapter 7
D-9	Lock-free vs. locking	cost model, chapter 7
D-10	Hot/cold field split	cost model, chapter 7
D-11	Allocation policy	Theorem 12.2
D-12	Plugin placement	Theorem 13.1 (feasibility)

Table 13.1: The decision matrices of the standard and the results that govern them.

In every case a solution is a molecule M with $M \vdash \Phi$ and $\text{cost}(M) \leq \bar{c}$, found by the terminating decision-matrix computation of Theorem 13.1 and refined to zero copy by Theorem 13.2. The situations of the standard mirror this chapter by construction, so each policy cites one of the results Theorem 11.1–Theorem 13.2 instead of restating a proof.

Chapter 14

Testing Strategies Derived from the Framework

The algebraic structure of Mol doubles as a testing oracle. Every theorem cataloged in Appendix B states a law that a compiled artifact must satisfy; each law can be discharged automatically as a property over finite input traces. This chapter turns the framework’s theorems into a testing program: what the mathematics guarantees, the test harness re-checks on the artifacts, so that a dataplane build is certified against its own specification and not against a hand-written checklist. Each strategy below names the governing theorem and the class of tests it generates. The executable tools behind the computational claims are described in Appendix A and cataloged in Appendix B.

14.1 Equational Law Testing (Theorem 4.3–Theorem 4.8)

The Lawvere theories of chapter 4 present the ring, parser, and transport theories as sets of equations, and 4.3–4.8 assert that every model in Mol satisfies them. These equations are directly testable as properties: generate a finite sequence of operations, evaluate the left-hand side and the right-hand side on a state, and compare the resulting output traces and final states. The ring laws $\text{push} \circ \text{pop} = \text{id}$ and $\text{pop} \circ \text{push} = \text{id}$ on non-empty rings are the canonical pair; the parser laws (sequence distributes over choice, choice is associative, identity laws hold, left factoring is sound) and the transport laws ($\text{batch_recv} = \text{recv} \otimes \cdots \otimes \text{recv}$; the interchange law) become regression suites that run on every build. Because 4.8 guarantees that adjacent push–pop pairs are removable, law testing doubles as a dead-code detector: an implementation that executes a removable pair without eliding it still satisfies the equation but violates the cost contract of 5.6.

14.2 Bisimulation and Equivalence Checking (Theorem 3.23)

Theorem 3.23 states that behavioral equivalence (bisimulation) is a congruence for both $\circ$ and $\otimes$, and that the quotient $\text{Mol}/\approx$ is again a symmetric monoidal category. The congruence property is what makes equivalence checking compositional: to verify that $f \approx g$ it suffices to check the two machines directly, and the result extends to every context in which they are embedded. The `bisim` tool (see Appendix A) enumerates small finite machines and checks, exhaustively, that the bisimulation relation it computes is preserved by composition and tensor. On the test floor this licenses the standard substitution pattern: replace an implementation by its minimized form (chapter 9) or its normal form (chapter 6) and re-certify the whole dataplane by equivalence, not by

re-testing behaviors. The notion goes back to Park [1981], where bisimulation is proposed precisely as the congruence that makes such substitution sound.

14.3 Model-Based Testing

A molecule is, by definition, a stateful transformation with a transparent state space and step function; there is no hidden control flow. Every artifact therefore admits a reference model in the same vocabulary: write the simplest ($S, step$) that realizes the intended behavior, then test the implementation against the model over all inputs up to a bounded length and all reachable states. Because composition in Mol is composition of step functions (theorem 3.7), models compose exactly as the artifacts do, and the model-based suite for a composite molecule is the composition of the suites of its parts. Model-based testing performs an equality check in the sense of 6.9: the model and the artifact are two representatives of the same equivalence class, and the harness verifies the bisimulation directly.

14.4 Conformance to Lawvere Theories

Conformance testing is the operational face of chapter 4: a component that claims to implement an operation of the ring, parser, or transport theory must satisfy the theory’s axioms as a conformance suite, with the axioms treated as MUST-level requirements. This is stronger than ordinary unit testing, because the axioms are closed under instantiation: any input pattern that matches the left side of an equation is a test case. 5.7 upgrades this to a decidability statement for finite signatures: the conformance problem for the whole runtime equational theory is decidable by normalization: so the conformance suite can in principle be exhaustive, not sampled. In practice the suite enumerates all ground instantiations up to a size bound and records coverage as the fraction of rewrite-normal forms exercised.

14.5 Fuzzing for Memory Safety

Fuzzing enters the framework in a precisely delimited role. Linearity (theorem 12.2) guarantees that a well-typed hot path never allocates or deallocates, and totality (atom discipline, chapter 3) guarantees that every declared operation is total on its domain; fuzzing audits the boundaries of these guarantees. The fuzzer generates adversarial byte streams for parsers and rings and checks three invariants: no panic (totality), no allocation in declared hot paths (measured with an allocation counter; theorem 12.2), and no out-of-bounds access (bitmask indexing is correct exactly when the capacity invariant of 11.3 holds). The value of fuzzing here is an audit: it cannot establish the theorems, but it can falsify the implementation’s claim to instantiate them, and it feeds the differential and cost suites below.

14.6 Differential Testing

When two molecules claim the same behavior, the differential harness runs them side by side on identical input streams and compares output traces and final states. The correctness criterion is exactly the bisimulation of 3.23, so a passing differential run is evidence of a bisimulation witness on the tested traces. The most productive instances are: (i) an optimized artifact versus its reference model (chapter 5); (ii) a minimized machine versus the original (chapter 9); and (iii) a batch or

SIMD variant versus the scalar version (chapter 11, chapter 12). Because of the congruence theorem, differential results propagate through composition: artifacts that differ only in subcomponents known equivalent need only the differing part tested.

14.7 Cost-Annotated Regression (Theorem 7.3–Theorem 7.8)

The enrichment of chapter 7 attaches a cost vector to every molecule, and the theorems 7.3–7.8 fix its algebra: additive composition cost, the triangle inequality, tensor subadditivity, and the fusion bound $c(\text{fused } g \circ f) \leq c(f) + c(g)$ with strictness when the intermediate type never materializes. These are quantitative laws, and they turn into regression assertions: each CI run measures time, memory, and cache misses per pipeline and asserts (a) the cost of a composite does not exceed the sum of its parts (Theorem 7.3), (b) fused pipelines are no more expensive than their unfused decompositions (Theorem 7.8), and (c) normalization never increases cost, $c(\text{NF}(M)) \leq c(M)$, by 5.6. Cost regressions are triggered by violations of the algebraic inequalities, which are stable under hardware changes in a way raw latency figures are not.

14.8 Property-Based Testing of Rewrites (Theorem 5.1–Theorem 5.7)

The rewrite engine of chapter 5 is a complete system for the runtime equational theory, and each of its structural theorems is testable as a property of the rewriting procedure itself. Termination (5.1) is tested by instrumenting every reduction with the lexicographic path measure and asserting strict descent. Local confluence (5.2) is tested by generating critical pairs, peaks $a \leftarrow b \rightarrow c$ where two rules overlap, and asserting that their joins coincide; Newman’s lemma (Newman [1942]) then yields global confluence, and the completion procedure is the classical one of Knuth and Bendix [1970]. Normal forms are tested for stability: rewriting a normal form yields itself (5.4, 5.5), and applying any rule sequence to a term and then normalizing yields the same result, which is exactly the soundness statement 5.6. Finally, the completeness statement 5.7 (two terms are equivalent iff their normal forms are equal) is the property that licenses the normal-form lookup table of chapter 6; the property suite asserts it on all terms up to a size bound, complementing the exhaustive tool of Appendix A.

14.9 State-Minimization Verification (Theorem 9.6–Theorem 9.9)

The coalgebraic minimization of chapter 9 produces, for every finite-state molecule, a minimal state space unique up to isomorphism (9.6), and minimization preserves behavior (9.7). Verification of a minimization step therefore has two clauses: the minimized artifact must be behaviorally equivalent to the original (a bisimulation check by 3.23), and no strictly smaller state space may exist (a bound check using 9.8 on the composition’s factors). Since 9.9 makes minimization compatible with equivalence: equivalent molecules have isomorphic minimizations; the test floor can canonicalize: every artifact that purports to be a pipeline stage is minimized and compared against the canonical minimized form, and any discrepancy is a specification-level bug, not a performance regression. The classical algorithms (Nerode [1958], Hopcroft [1971]) underlie the check, and Brzowski [1964] supplies the algebraic counterpart for the regular languages involved.

14.10 Type-Level Guarantees

The deepest tests are the ones that never run. Parametricity (Wadler [1989]) supplies free theorems: a polymorphic pipeline behaves the same on all types over which it is parametric, which the type checker enforces and the test suite uses to justify data-parallel SIMD by naturality (see Appendix B). Linearity, by 12.2, moves the zero-allocation guarantee into the type system: a molecule that consumes its input exactly once cannot allocate in a well-typed hot path, and the ownership/borrow checker of the implementation language enforces this at compile time. The result is a layered test pyramid: type-level guarantees discharge the strongest claims without executing them; the property, differential, and cost suites discharge the algebraic laws; and fuzzing audits the boundaries. Together the layers instantiate every theorem of the catalog as an ongoing, automated obligation.

Chapter 15

Related Work

This chapter situates Mol among the traditions it draws on: the classical theory of sequential machines, categorical semantics, term rewriting, and the engineering literature on high-performance dataplanes. For each field we state what it contributes, where Mol agrees with it, and where the thesis diverges. The comparison is meant to be a map, not a survey; complete bibliographies for each area are cited.

15.1 Sequential Machines: Mealy and Moore

The deterministic Mealy machine (a state space S , input A , output B , and a transition-output function $S \times A \rightarrow B \times S$) is exactly the data of a morphism of Mol, introduced by Mealy [1955] in 1955 for the synthesis of sequential circuits; the output-decorated variant with output a function of state alone is due to Moore [1956]. Mol differs from the classical treatment in two ways. First, the morphisms of Mol are *equivalence classes* of machines under the state-space bijection condition (theorem 3.8), so state is an implementation detail and only the input–output behavior is observable. Second, Mol is organized as a category with a tensor product, so the modular structure of dataplane design (composition and parallelism) is first-class rather than derived. Recent work continues in the same direction: effectful Mealy machines Bonchi et al. [2024] combine Mealy structure with a monad, which in our setting is the content of 3.17 (EffMol as the Kleisli category of the state monad).

15.2 Coalgebra and Automata Theory

Behavioral equivalence and state minimization are the core business of coalgebra. The minimal-realization theorems 9.6–9.9 restate, inside Mol, results whose algorithmic form is classical: Nerode [1958] for the Myhill–Nerode quotient and Hopcroft [1971] for the $n \log n$ minimization algorithm; the algebraic route via derivatives is Brzozowski [1964]. The coalgebraic literature generalizes minimization and bisimulation far beyond automata (see Colcombet and Petrişan [2017] for current partition-refinement algorithms) and Mol is a special case of that program: the step function $S \times A \rightarrow B \times S$ is an endofunctor’s coalgebra once the input is fixed. What Mol adds is the category structure *between* coalgebras: composition, tensor, and the enrichment of chapter 7 operate on machines, not just on their languages.

15.3 Traced Monoidal Categories

The traced monoidal structure of chapter 10 is due to Joyal et al. [1996], who characterized traced monoidal categories as the universal completion of symmetric monoidal categories under feedback and established the sliding, vanishing, superposing, and yanking laws used here as 8.2–8.4. Modern treatments (see Hu and Vicary [2021]) reconstruct trace as a structure in profunctor categories and connect it to the geometry of interaction. The thesis’s contribution is applicative: fixed-iteration loops in a dataplane are traces, the Banach contraction principle (Banach [1922]) gives their convergence, and the trace laws license the hoisting and flattening optimizations of chapter 8 and Appendix B.

15.4 Lawvere Theories

Lawvere’s *functorial semantics of algebraic theories* Lawvere [1963] identifies an equational theory with a small category with finite products, and its models with finite-product-preserving functors. The modern correspondence between Lawvere theories and finitary monads is treated in Lack and Rosický [2008]. The thesis applies the framework, rather than extending it: the ring, parser, and transport theories of chapter 4 are Lawvere theories, their models in Mol are the conforming components, and the completeness of the free ring model (4.3) is the familiar universal-algebra fact restated in Mol. The benefit of the categorical form is that models and rewrite rules are then subject to the machinery of chapter 5.

15.5 Operads, Multicategories, and Clubs

N-ary operations with typed composition are organized by operads and multicategories; the standard references are Leinster [2004] and, for the club formalism that combines operadic composition with substitution, Kelly [1974]; see also Weber [2014] for the connection between operads, clones, and monads. Batch operations (theorem 11.1) are precisely the operadic operations of chapter 11: a batch of n inputs is an n -ary arrow, and reassociation of batching is the associativity law of operadic composition. Kelly’s clubs enter at chapter 5, where typed composition trees with substitution are the objects over which the rewrite system acts.

15.6 Linear Logic and Geometry of Interaction

Girard’s linear logic Girard [1987] makes resource sensitivity (using each input exactly once) a structural principle, and the geometry of interaction gives it an operational, token-passing semantics; the modern development of the program is Seiller [2015]. The thesis connects to this line in three places: cut elimination as deforestation (12.1), the zero-allocation guarantee (12.2) as the engineering reading of linearity, and the trace structure of chapter 10, which is the categorical shadow of feedback in the geometry of interaction. Mol’s linear fragment is thus a small, decidable instance of the linear-logic programme, specialized to stateful dataflow rather than proof theory.

15.7 Effectus Theory and Markov Categories

Effectus theory Cho et al. [2015] axiomatizes partiality and measurement in categorical terms, and Markov categories Fritz [2019] do the same for probability; both are cousins of Mol in the family

of "categories of processes." The connection points are the effectful fragment EffMol (theorems 3.17, 3.24) and the quantitative enrichments of chapter 7, whose cost vectors are a deterministic analogue of the Markov-kernel structure: where a Markov category composes stochastic kernels, Mol composes cost-annotated deterministic transitions. The fixed-point technology differs accordingly: Banach contraction and the Dobrushin coefficient Dobrushin [1956] for Markov updates (10.10) in place of probabilistic convergence.

15.8 Bisimulation as Congruence

The congruence theorem 3.23 is the thesis's use of a principle due to Park [1981]: bisimulation is the finest congruence on processes that respects observable behavior. Park's setting is non-deterministic and concurrent; Mol's is deterministic and sequential, which makes the congruence property elementary to prove (the proof of 3.23) while keeping its practical force: equivalent machines are interchangeable in every context, which the testing strategy of chapter 14 exploits and the minimization of chapter 9 depends on.

15.9 Implicit and Descriptive Complexity

The complexity-theoretic placement of Mol is twofold. Descriptively, finite-state molecules realize exactly the regular behaviors and minimal molecules realize minimal DFAs (7.17); this is the classical result of automata theory in Mol's vocabulary. Implicitly, the resource-sensitive typing of chapter 12 belongs to the tradition of Dal Lago and Hofmann [2005], which characterizes complexity classes by the absence of structural operations rather than by explicit bounds; the zero-allocation theorem 12.2 is the dataplane's version of such a characterization. The accessible-category background of chapter 2 is Adámek and Rosický [1994].

15.10 High-Performance Dataplanes

The engineering literature on dataplanes is extensive and largely empirical: zero-copy networking, batching, SIMD packet processing, and kernel-bypass I/O are surveyed and compared in Power [2013]. The thesis's relation to this body of work is complementary rather than competitive. The empirical literature reports what works and how fast; Mol states, in theorem form, *why* the known optimizations are sound (fusion, batching, and hoisting are consequences of the algebra of chapter 5, chapter 11, and chapter 8) and the standard that accompanies this thesis turns those theorems into enforceable policies. Where the empirical literature is the measurement apparatus, Mol is the specification the measurements certify. EOF

Chapter 16

Conclusion

This thesis formalized Mol, the category of stateful transformations, as the algebraic foundation of zero-allocation network dataplanes. The contribution is a single, compact claim: the engineering practice of high-performance networking (pipeline fusion, batching, hoisting, minimization, zero-copy) is the observable face of a small number of categorical structures, and those structures yield theorems with proofs. Sixty-eight theorems were stated and proved in the body of the thesis. They are collected below as the specification of the framework.

16.1 The Theorem Catalog

The catalog lists all theorems Theorem 3.7–Theorem 3.29. The tag column marks provenance: [N] denotes a statement original to this thesis, proved with elementary machinery; [C] denotes a classical result restated and fully proved inside Mol, with provenance cited in the text. The statements are compressed to one line; full statements and proofs are in the cited chapters.

Table 16.1: Theorems of chapter 3: Mol foundations (Theorem 3.7–Theorem 3.27).

Theorem	Tag	Statement	Chapter
3.7	[N]	Mol is a category: composition of $(S, step)$ classes is associative with identity.	chapter 3
3.8	[N]	The state-space bijection quotient is well-defined: equivalent machines compose to equivalent machines.	chapter 3
3.12	[C]	Mol is symmetric monoidal: tensor = tuple product, unit = $()$, coherent braiding (Mac Lane coherence Mac Lane [1971]).	chapter 3
3.14	[N]	PureMol is equivalent to the category of types and total functions.	chapter 3
3.17	[N]	EffMol (Ctx) is isomorphic to the Kleisli category of the state monad on Ctx.	chapter 3
3.23	[N]	Behavioral equivalence (bisimulation) is a congruence for $\circ$ and $\otimes$; Mol/ $\approx$ is symmetric monoidal.	chapter 3
3.24	[N]	PureMol and EffMol are closed under tensor.	chapter 3
3.27	[N]	Pure molecules and their tensor products are deterministic; behavioral equivalence preserves determinism.	chapter 3

Table 16.2: Theorems of chapter 4 and chapter 5: theories and rewriting (Theorem 4.3–Theorem 5.7).

Theorem	Tag	Statement	Chapter
4.3	[N]	Ring theory equations ($\text{push} \circ \text{pop} = \text{id}$; $\text{pop} \circ \text{push} = \text{id}$ when non-empty) hold in every model and are complete for the free ring model.	chapter 4
4.5	[N]	Parser theory: sequence distributes over choice; choice is associative; identity laws; left factoring is sound.	chapter 4
4.7	[N]	Transport theory: $\text{batch_recv} = \text{recv} \otimes \cdots \otimes \text{recv}$; the interchange law $(f \otimes g); (h \otimes k) = (f; h) \otimes (g; k)$ holds.	chapter 4
4.8	[N]	Ring cancellation: an adjacent $\text{push} \circ \text{pop}$ pair is removable with behavior preserved.	chapter 4
5.1	[N]	The ring rewrite system is terminating (lexicographic path measure).	chapter 5
5.2	[N]	The system is locally confluent; all critical pairs join, hence it is confluent (Newman’s lemma Newman [1942]).	chapter 5
5.3	[N]	Knuth–Bendix completion of the ring theory terminates in a complete system (script-verified, Appendix A).	chapter 5
5.4	[N]	Parser left-factoring rewrites are confluent and terminating; left-factored normal forms are unique.	chapter 5
5.5	[N]	Batch-flattening rewrites are confluent on batch trees; the flattened normal form is unique.	chapter 5
5.6	[N]	Normalization soundness: $\text{NF}(M) \approx M$ and $\text{cost}(\text{NF}(M)) \leq \text{cost}(M)$ for cost-nonincreasing rules.	chapter 5
5.7	[N]	The runtime equational theory is decidable for finite signatures via normalization.	chapter 5
5.9	[N]	Modular completion: adjoining a new equational theory preserves termination and confluence under the disjoint-union or hierarchical conditions; decidability is preserved.	chapter 5

16.2 Limitations

The framework is deliberately scoped, and its limitations should be stated as precisely as its theorems.

Determinism. Mol is a category of *deterministic* mealy machines; nondeterminism, concurrency, and probabilistic behavior are outside the model. The Markov update theorem 10.10 is the only probabilistic result, and it is imported from the classical theory rather than internalized. Extending Mol with an effectus or Markov-category structure (chapter 15) is natural but was not undertaken here.

Finite-state emphasis. The decidability and finiteness results (5.7, 6.6, 13.1) assume finite or bounded type signatures. Dataplanes whose state is genuinely unbounded (arbitrary-size connection tables, streaming state) fall outside the normal-form and decision procedures, and the theorems

Table 16.3: Theorems of chapter 6 and chapter 7: free molecules and enrichments (Theorem 6.4–Theorem 7.19).

Theorem	Tag	Statement	Chapter
6.4	[C]	The free symmetric monoidal category $F(\Sigma)$ on atom signature Σ exists (initial algebra; accessible categories Adámek and Rosický [1994]).	chapter 6
6.5	[C]	Universal property: every molecule over Σ factors uniquely (up to coherent iso) through $F(\Sigma)$.	chapter 6
6.6	[N]	Finiteness: finite Σ with bounded types gives finite hom-sets in $F(\Sigma)$.	chapter 6
6.8	[N]	Every molecule has a unique normal form in $F(\Sigma)$ up to coherence.	chapter 6
6.9	[N]	Completeness: $M \approx M'$ iff $\text{NF}(M) = \text{NF}(M')$; normal forms form a precomputable lookup table.	chapter 6
7.3	[N]	Cost enrichment is well-defined: additive composition cost, triangle inequality, tensor subadditivity.	chapter 7
7.5	[N]	Global optimality: nonnegative additive costs $\Rightarrow$ min-cost molecules are shortest paths in the finite type graph (Dijkstra complete).	chapter 7
7.8	[N]	Fusion bound: $\text{cost}(\text{fused } g \circ f) \leq \text{cost}(f) + \text{cost}(g)$, strict when the intermediate type never materializes.	chapter 7
7.11	[N]	Refinement substitution: $f \sqsubseteq f' \wedge g \sqsubseteq g' \Rightarrow g \circ f \sqsubseteq g' \circ f'$; behavior preserved, cost monotone.	chapter 7
7.13	[N]	Maximal refinement: finite refinement lattices have a unique maximal element per behavior class; the most-refined dispatcher is sound and cost-minimal.	chapter 7
7.15	[N]	Complexity closure: atoms in a class C closed under $\circ$ and $\otimes$ yield molecules in C .	chapter 7
7.17	[C]	Descriptive complexity: finite-state molecules realize exactly the regular behaviors; minimal-state molecules realize minimal DFAs.	chapter 7
7.19	[N]	Projection reduction: if step depends on state only through π , then $S_{\min} \cong \pi(S)$.	chapter 7
7.22	[N]	Resource-graded cost enrichment: cache-line, false-sharing, and expected branch-misprediction costs form a graded monoidal enrichment; shortest-path optimality extends.	chapter 7

Table 16.4: Theorems of chapter 8, chapter 9, and chapter 10: decomposition, minimization, loops (Theorem 8.1–Theorem 10.10).

Theorem	Tag	Statement	Chapter
8.1	[N]	Every hybrid molecule decomposes $M = q \circ e \circ p$ (p, q pure, e effectful), unique up to isomorphism.	chapter 8
8.2	[C]	Sliding: $\text{Tr}(f; M) = f; \text{Tr}(M)$ for pure f (Joyal–Street–Verity Joyal et al. [1996]).	chapter 8
8.3	[C]	Vanishing and yanking: $\text{Tr}(\text{Tr}(M)) = \text{Tr}(M)$; $\text{Tr}(\sigma) = \text{id}$.	chapter 8
8.4	[C]	Superposing: $\text{Tr}(M) \otimes N = \text{Tr}(M \otimes N)$.	chapter 8
8.5	[N]	Hoisting shrinks the loop-state footprint; cache footprint of loop state is monotone in state size.	chapter 8
9.6	[C]	Every finite-state molecule has a minimal state space, unique up to isomorphism (Nerode/Hopcroft Nerode [1958], Hopcroft [1971]).	chapter 9
9.7	[C]	Minimization preserves behavior (Nerode quotient equivalence).	chapter 9
9.8	[N]	Compositional bounds: $ S_{\min}(g \circ f) \leq S_f \cdot S_g $; likewise for $f \otimes g$.	chapter 9
9.9	[N]	Minimization is compatible with equivalence: $M \approx M' \Rightarrow S_{\min}(M) \cong S_{\min}(M')$.	chapter 9
10.5	[C]	A contraction state update (rate $\alpha < 1$) converges to a unique fixed point (Banach Banach [1922]).	chapter 10
10.6	[N]	Iteration bound: $k(\alpha, \varepsilon, d_0) = \lceil \ln(\varepsilon(1 - \alpha)/d_0) / \ln \alpha \rceil$ iterations suffice; the unrolled loop is within ε , branch-predictable.	chapter 10
10.8	[N]	Contraction rates combine: $\max(\alpha_1, \alpha_2)$ under tensor; $\alpha_1 + \alpha_2$ sequential (sup metric).	chapter 10
10.10	[C]	Markov updates: total-variation distance contracts by the Dobrushin coefficient Dobrushin [1956].	chapter 10

Table 16.5: Theorems of chapter 11, chapter 12, and chapter 13: batching, linearity, situations (Theorem 11.1–Theorem 13.2).

Theorem	Tag	Statement	Chapter
11.1	[N]	Batch flattening: $\text{batch}(\text{batch}(a, b), \text{batch}(c, d)) = \text{batch}(a, b, c, d)$.	chapter 11
11.2	[N]	Syscall amortization: $\text{cost}(\text{batch}_n) \leq n \cdot \text{cost}(\text{single}) + \text{fixed}(n)$, with $\text{fixed}(n)/n \rightarrow 0$ (script-verified, Appendix A).	chapter 11
11.3	[N]	Ring capacity invariant: capacity 2^k with bit-mask indexing is correct iff $\text{in-flight} \leq 2^k - 1$.	chapter 11
12.1	[C]	Cut elimination = deforestation: every pure pipeline fuses to a single morphism (Girard Girard [1987], proved for Mol).	chapter 12
12.2	[N]	Linearity: atoms consume input exactly once $\Rightarrow$ a well-typed hot path never allocates or deallocates.	chapter 12
12.6	[N]	Affine typing: well-typed molecules consume resources exactly once and evaluate without allocation.	chapter 12
13.1	[N]	Situation solvability: bounded types and additive costs $\Rightarrow$ the feasible set of $(A \rightarrow B, \Phi, \bar{c})$ is finite and decidable; decision-matrix computation terminates.	chapter 13
13.2	[N]	Zero-copy roundtrip: $\text{parse/serialize inverse}$ on the well-formed domain $\Rightarrow \text{parse;serialize} \approx \text{id}$; roundtrip elimination is sound.	chapter 13

Table 16.6: Theorems of chapter 17 and remaining foundations (Theorem 17.2–Theorem 3.29).

Theorem	Tag	Statement	Chapter
17.2	[N]	Boundary enrichment: syscall count is a well-defined graded cost, lexicographic over userspace cost.	chapter 17
17.4	[N]	Batch fusion: $\beta(\text{batch}_n(R)) = 1$ for $n \geq 1$, strictly less than the unbunched tensor for $n \geq 2$.	chapter 17
17.6	[N]	Affine crossover: two affine payload costs have at most one threshold; the cheaper realization is unique on each side.	chapter 17
17.8	[N]	Optimization lattice: behavior-preserving rewrites of a confluent system form a finite lattice; the minimum is computable.	chapter 17
17.10	[N]	Bypass is a retraction: a kernel-bypass path retracts the socket path on the supported device class.	chapter 17
17.12	[N]	Cache soundness: a cached control-plane read with TTL is equivalent to the uncached read on the TTL window.	chapter 17
17.14	[N]	Band contraction: the integer EMA $x \mapsto x + n - x/8$ has attracting band $[8n, 8n + 7]$.	chapter 17
17.16	[N]	Zero-boundary loops: a typed hot path with $\beta = 0$ performs no syscalls in steady state.	chapter 17
17.18	[N]	Route languages are regular; the pattern trie is a deterministic automaton.	chapter 17
17.19	[C]	The route automaton is the Myhill–Nerode refinement of the pattern language.	chapter 17
17.20	[N]	Hybrid matcher selection: the affine crossover decides linear scan versus automaton.	chapter 17
3.28	[N]	The pure, effectful, and hybrid classes are the unique congruence-compatible extension of the atom classification.	chapter 3
3.29	[N]	Universality: every application expressible as a stateful transformation is expressible as a molecule.	chapter 3

say nothing about them.

Proof style. Proofs are complete and step-justified, with each step stated and discharged in the text. The computational claims of chapter 14 are additionally verified by executable tools (see Appendix A).

Measurement. Cost enrichment makes optimization statements *provable relative to the chosen cost vector*, but the choice of costs (time, memory, cache misses) is empirical, and the strictness claims (7.8) depend on the implementation materializing or eliding intermediate types. The theorems license the optimizations; they do not measure them.

16.3 Future Work

Three directions follow immediately from the limitations and from the open threads noted in the text.

Probabilistic and effectful extensions. Internalizing the Markov structure behind 10.10, and giving Mol an effectus-style account of partiality, would extend the framework from deterministic dataplanes to stochastic and best-effort systems while keeping the cost enrichment intact.

Information geometry of the trace. chapter 10 noted that the coordinate selection for contraction analysis of state updates has an information-geometric reading; working out the geometry of the loop-state manifolds and its interaction with the iteration bound 10.6 is left open.

Compiler and tooling. The optimization and testing catalogs of Appendix B are specifications; building the checker that reads a molecule description, applies the legal optimizations, and discharges the corresponding tests automatically is the engineering continuation of this thesis.

Taken together, the sixty-eight theorems give the dataplane engineer what the empirical literature cannot: a specification against which a networking stack can be certified, with the certification obligations made explicit in chapter 14 and Appendix A.

Chapter 17

Optimization Discovery: Kernel-Boundary Cost, Threshold Theorems, and the Lattice Method

The preceding chapters prove that engineering properties of dataplanes are theorems of Mol. This chapter turns the direction around: given the model, how does one *discover* every optimization of a given molecule? The answer has three parts, each grounded in measurements on production datapaths:

- *The dominant cost is the kernel boundary.* System counters on every benchmarked datapath show $\%_{\text{sys}} + \%_{\text{soft}}$ at roughly twice $\%_{\text{usr}}$: the dominant per-request cost is syscalls and softirqs, not userspace instructions. The cost vector of Definition 7.1 has no kernel coordinate; adding one changes which rewrites are worth doing.
- *Realization choice is a threshold, not a heuristic.* When two realizations of one behavior have affine payload cost, the cheaper one is decided by a single crossover point (Theorem 17.6), confirmed by measurement: the sendfile-versus-wire-cache crossover computed from fitted cost lines (134.9 KiB) matches the measured crossover (about 128 KiB).
- *All optimizations form a finite lattice.* For bounded molecules the behavior-preserving rewrites of a confluent system are finitely many; grading them by the boundary-enriched cost and taking the minimum is a decision procedure (Theorem 17.8).

The chapter closes with two measured case studies in the before-and-after form: an input-output molecule (file copy with checksum) and a pure compute molecule (generation of primitive Pythagorean triples). For each, the baseline realization, each individual optimization, and the combined stack are measured on the same machine with the same input, and reported with instruction counts, miss rates, latency, memory, and the compounding improvement.

17.1 The kernel-boundary enrichment

Definition 17.1 (Boundary count). For a molecule M and a fixed input distribution, the *boundary count* $\beta(M)$ is the number of kernel crossings (syscalls) performed by a steady-state run of M . The *boundary-enriched cost* of M is the pair $(\beta(M), c(M))$ where c is the cost vector of Definition 7.1, ordered lexicographically by β first.

The boundary coordinate is not a refinement of c ; it is a different quantity. A pipeline that performs no syscalls and no allocation has cost bounded by its userspace instruction count; one that performs one syscall per request pays a per-request kernel cost that dominates everything else on typical hardware.

Theorem 17.2 (Boundary enrichment is well-defined). *For molecules over a fixed signature, the boundary count satisfies the quantitative laws: (1) composition: $\beta(g \circ f) = \beta(f) + \beta(g)$ for the naive (unfused) realization, and $\beta(g \circ f) \leq \beta(f) + \beta(g)$, with strict inequality when the composite is realized by a single fused kernel operation; (2) tensor: $\beta(f \otimes g) = \beta(f) + \beta(g)$ in the disjoint realization, with strict inequality when the factors share one syscall (batching, Theorem 17.4); (3) well-definedness: β is invariant under the structural laws of the free symmetric monoidal category (identities and braidings cross no boundary and cost 0). Consequently the boundary count is a well-defined $\mathbb{N}$ -valued graded enrichment of Mol , and the boundary-enriched cost orders realizations lexicographically: eliminate syscalls before optimizing userspace instructions.*

Proof. (1) and (2): by the definition of composition and tensor in chapter 3. A disjoint realization of $f \otimes g$ runs the two factors' syscall streams independently, so the counts add; a fused realization (a single receive batch serving both, a single file transfer carrying both bodies) crosses the boundary once. (3): an identity is a pure no-op and a braiding swaps wires; neither performs a syscall, so the count is unchanged under the symmetric monoidal axioms; Mac Lane's coherence theorem [Mac Lane, 1971] extends this to all structural equations, exactly as in the proof of the cost law. The lexicographic ordering is immediate from the definitions. $\square$

Remark 17.3 (The measured law). A production HTTP server's multiplexed path performed two `/proc/self/status` reads per request inside a memory-governor check: a boundary count of 2 per request injected into the hot path. Removing them (caching resident-set size with a 100 ms TTL) raised the multiplexed throughput by a factor of 5.2, with no other change. Per-request userspace work was untouched; the entire gain is the boundary coordinate. Theorem 17.16 turns this class of incident into a typing rule.

17.2 The batch fusion law

Theorem 17.4 (Batch fusion). *Let R be a syscall atom (receive, send, or file transfer) and let $\text{batch}_n(R)$ be the n -fold batch construction of chapter 11. Then $\beta(\text{batch}_n(R)) = 1$ for $n \geq 1$, while the unbunched realization has $\beta(R \otimes \cdots \otimes R) = n$; for every $n \geq 2$ the batch strictly reduces the boundary count, and the tensor law $\beta(f \otimes g) = \beta(f) + \beta(g)$ of Theorem 17.2 degrades to $\beta(\text{batch}_{m+n}) \leq \beta(\text{batch}_m) + \beta(\text{batch}_n)$.*

Proof. A batched syscall crosses the kernel boundary once for all n items by definition; the tensor additivity and the subadditive degradation are the arithmetic of the counts. For $n \geq 2$, $1 < n$ is strict reduction. The three identities hold for every concrete syscall atom because the batch construction of chapter 11 groups the n items into one syscall invocation regardless of the atom's argument type. $\square$

Remark 17.5. This is the boundary-coordinate reading of the syscall-amortization law of chapter 11; the 64-datagram receive and send batches of the reference reactor loop are the realization, and the reduction from 64 boundaries to 1 per batch is the measured content.

17.3 Threshold theorems: realization choice as a crossover

Two realizations of one behavior rarely have comparable constant cost: one pays a smaller per-byte cost and a larger fixed cost (sendfile versus userspace copy; zerocopy versus copy; kernel versus userspace datapath). The choice is a function of payload size, and the point where the advantage flips is a theorem.

Theorem 17.6 (The affine crossover). *Let R_1, R_2 be realizations of one behavior class with payload-size-parameterized costs $c_i(n) = a_i n + b_i$ with $a_i, b_i \in \mathbb{R}$ and $n \geq 0$. If $a_1 > a_2$, then the difference $c_2(n) - c_1(n) = (a_2 - a_1)n + (b_2 - b_1)$ is strictly decreasing in n , and for the sharp crossover point n^* (the largest integer with $c_1(n^*) \leq c_2(n^*)$):*

$$n \leq n^* \Rightarrow c_1(n) \leq c_2(n), \quad n \geq n^* + 1 \Rightarrow c_2(n) < c_1(n) :$$

the realization with the smaller slope wins above the crossover, the steeper one below it, and the switch happens exactly once. In closed form, $n^ = \lfloor (b_2 - b_1)/(a_1 - a_2) \rfloor$ when $a_1 > a_2$.*

Proof. The difference is affine with negative slope, hence strictly decreasing. It crosses zero exactly once, at the real point $(b_2 - b_1)/(a_1 - a_2)$; n^* is the floor of that point, and monotonicity gives the two implications. The switch is unique because a strictly decreasing function changes sign at most once; the two inequalities are exactly the sign conditions on either side of the crossover. $\square$

Remark 17.7 (Measured confirmation and prediction). The sendfile-versus-wire-cache choice of chapter 13 is an instance. Fitting affine cost lines to the measured per-request costs (wire-cache byte path and sendfile, at 64/128/256 KiB payloads):

$$c_{\text{byte}}(n) = 5.92 \times 10^{-4} n - 1.32 \mu\text{s}, \quad c_{\text{sendfile}}(n) = 2.90 \times 10^{-4} n + 40.45 \mu\text{s}.$$

Theorem 17.6 predicts the crossover at $n^* = (40.45 + 1.32)/(5.92 - 2.90) \times 10^{-4} = 138,143$ bytes, about 134.9 KiB. The measured crossover is about 128 KiB (the byte path wins at 64 KiB, the lines cross near 128 KiB, sendfile wins at 256 KiB). The threshold is computable from the two cost lines, and the measurement lands on the prediction. The same theorem predicts the other crossover pairs of this chapter: zerocopy versus plain copy on a NIC, and the completion-driven versus readiness-driven reactor (section 17.12); each turns a tuning choice into a computed threshold.

17.4 The optimization lattice

Theorem 17.8 (The optimization lattice). *Let M be a molecule over a finite signature with bounded types, and let $\mathcal{R}$ be a terminating, confluent rewrite system on its realizations (the systems of chapter 5). Then: (1) the set $\text{NF}(M)$ of normal-form-equivalent realizations of M , those reachable from M by $\mathcal{R}$ -rewrites, is finite; (2) with the boundary-enriched cost of Theorem 17.1, $\text{NF}(M)$ is a finite lattice under the componentwise cost order, and it has a least element: the realization of minimal boundary count, and among those, minimal cost; (3) the least element is computable by enumerating the finite rewrite paths of $\mathcal{R}$ from M and grading each by (β, c) : a decision procedure, not a search heuristic.*

Proof. (1) is the finiteness of normal forms, restricted to the rewrites of $\mathcal{R}$, which are behavior-preserving. (2) and (3): the rewrites preserve the behavior class, so every realization in $\text{NF}(M)$ is a candidate; the boundary count and cost vector are each $\mathbb{N}$ -valued on a finite set, so the lexicographic minimum exists and is found by the finite enumeration. The enumeration terminates because $\mathcal{R}$ is terminating and the signature is finite: no infinite rewrite chains to new realizations beyond the finite normal forms. $\square$

Remark 17.9 (The discovery method). Theorem 17.8 answers the question this chapter poses: to find all provable optimizations of a molecule, (i) write the molecule; (ii) list the behavior-preserving rewrites of its confluent theory (fusion, batching, boundary fusion, datapath substitution); (iii) grade each realization by (β, c) ; (iv) take the lattice minimum. The measured optimizations of this chapter are exactly the lattice minima of their molecules: the wire cache is the minimal-cost realization of the static-file molecule (one gathered write, zero per-request syscalls); the sendfile choice is the threshold-optimal realization of Theorem 17.6; the bypass datapath is the boundary-minimal realization when the kernel’s per-packet cost dominates (Theorem 17.10).

17.5 Datapath bisimulation and bypass as retraction

Theorem 17.10 (Bypass is a retraction). *Let K be the kernel datapath molecule (socket receive, protocol processing, socket send) and let U be the userspace datapath molecule of a frame pipeline (parse, validate, checksum, echo). Then: (1) K and U are bisimilar on the well-formed frame domain: they emit the same output frames for every well-formed input; (2) U is a retraction of K in Mol on that domain: there is a pure embedding E with $U \circ E \simeq \text{id}$ on the well-formed frames; (3) the datapath choice is a cost comparison, decided by the threshold of Theorem 17.6: use U exactly when $\beta(U) \cdot s + c(U) < \beta(K) \cdot s + c(K)$, where s is the per-syscall kernel cost; on loopback the kernel wins (measured), and on a NIC where $\beta(K)$ is per-packet and $c(K)$ dominates, the userspace datapath wins.*

Proof. (1) is frame-level behavior preservation verified by a veth test harness: a test frame traverses a redirect into the frame socket, is validated, checksummed, and echoed byte-for-byte; bisimulation follows by the congruence theorem for $\circ$ and $\otimes$. (2): the frame processing is total on the well-formed domain and inverts the framing, the parse/serialize inverse. (3): the two datapaths are behaviorally interchangeable by (1), so the choice is the realization choice of Theorem 17.6 with the payload being the packet. $\square$

Remark 17.11. The architectural fact that the bypass avoids the kernel is a consequence of the model: the bypass is sound because $K \simeq U$ (bisimulation), and it is beneficial because of the boundary-count inequality. Neither claim is an engineering opinion.

17.6 Cache soundness as bounded staleness

Theorem 17.12 (Cache soundness). *Let M be a molecule whose behavior depends on a state S that changes only at event times (file writes, configuration reloads, rule swaps), and let C be the caching realization of M that serves the normal form of the state at time t for Δ time units. Then C is behaviorally equivalent to M on every request in $(t, t + \Delta]$ if and only if no event occurs in $(t, t + \Delta]$. Consequently a sound TTL is any Δ not exceeding the time to the next event; the TTL of a cache is a theorem about the event process, not a tuning constant.*

Proof. The cached molecule and the live molecule agree on inputs exactly when their state agrees, because the output is a deterministic function of the state and the input (Mealy determinism, chapter 3). The state agrees on $(t, t + \Delta]$ exactly when no event changed it in that window; the equivalence is then bisimulation on the window domain, and the “if and only if” is the two directions of that agreement. $\square$

Remark 17.13. The time-to-live values of the static module in chapter 13 were chosen by intuition; Theorem 17.12 makes the choice a theorem: the minimum time to the next event, or the empirical

event-gap quantile. The invalidation atoms of the control surface (configuration reload, cache invalidate) are exactly the event process of the theorem.

17.7 The scheduler estimate is a band contraction

The integer admission scheduler of a production HTTP server maintains a demand estimate $d' = d + n - d/8$ over $\mathbb{N}$ (a right shift by 3 in the shipped code). This is the state update of the fixed-iteration loop of chapter 10; the following theorem makes its tuning a proof.

Theorem 17.14 (Band contraction). *Let $F(x) = x + n - x/8$ on $\mathbb{N}$ with fixed demand $n \geq 1$. Then: (1) the band $B = [8n, 8n + 7]$ is the fixed set: $F(x) = x$ for $x \in B$, and every fixed point lies in B ; (2) below the band F strictly increases and stays at or below $8n$; above the band F strictly decreases; (3) F is nondecreasing, so the iterate $F^k(x_0)$ is monotone in x_0 ; (4) consequently the iterate converges to B in finitely many steps, and the crossing time is bounded by the iteration bound of chapter 10 with rate $7/8$: the estimate is a $7/8$ contraction in the continuous relaxation, and the integer rounding leaves a band of width 8 instead of a point.*

Proof. (1): for $8n \leq x \leq 8n + 7$, integer division gives $x/8 = n$, so $F(x) = x + n - n = x$. Conversely, $F(x) = x$ forces $n = x/8$, which by the meaning of integer division means $8n \leq x \leq 8n + 7$, that is, $x \in B$. (2): write $x = 8n + k$ with k an integer. If $x < 8n$ then $k < 0$ and $x/8 \leq n - 1$, so $F(x) = x + n - x/8 \geq x + n - (n - 1) = x + 1 > x$, and $F(x) \leq x + n \leq 8n - 1 + n = 9n - 1$; since $x/8 \geq n - 1$ is false only for $x < 8(n - 1)$, iterating eventually reaches the band because each step increases x by at least 1 while $x < 8n$. If $x > 8n + 7$, then $x/8 \geq n + 1$, so $F(x) = x + n - x/8 \leq x + n - (n + 1) = x - 1 < x$, and $F(x) \geq x/8 \cdot 7 + n \geq 7(n + 1) + n = 8n + 7$, so the iterate stays above the band until it enters it; each step decreases x by at least 1, so entry is finite. (3): if $x \leq y$ then $x/8 \leq y/8$ (integer division is monotone), so $F(x) = x + n - x/8 \leq y + n - y/8 = F(y)$. (4): by (2), the distance to the band strictly decreases at each step outside it, and the state is integer, so the crossing is finite. For the continuous relaxation $y' = \alpha y + n$ with $\alpha = 7/8$, the fixed point is $y^* = n/(1 - \alpha) = 8n$, and $y' - 8n = \alpha(y - 8n)$, an exact contraction with rate $\alpha = 7/8$; the iteration bound of chapter 10 applies with that rate, and the integer rounding leaves the band $[8n, 8n + 7]$ instead of the point $8n$. $\square$

Remark 17.15. The firewall thresholds and the exponential-decay roll-off of the same scheduler have the same contractive shape with rates $7/8$ and $15/16$. Theorem 17.14 gives the constants a theorem: the estimate is within ε of its attractor after the iteration bound of chapter 10 steps, and the fast-path shortcut of the admission gate, checking the global bound before the per-source table, is a cost-monotone rewrite: cheaper, and behavior-preserving on the rejected set.

17.8 Zero-boundary loops: the hot-path effect discipline

Theorem 17.16 (Zero-boundary loops). *Let $L_k(M)$ be the fixed-iteration loop whose body M is a molecule typed in the affine system of chapter 12 with no effectful atoms: no syscall atom and no allocation atom in its signature. Then in steady state $\beta(L_k(M)) = 0$: the loop crosses the kernel boundary only in its setup, and its steady-state cost is purely userspace, $c(L_k(M)) = k \cdot c(M)$ by the additivity of the cost law.*

Proof. The affine typing forces exact-once consumption and no allocation; a signature without syscall atoms performs no syscalls by the definition of the boundary count; the loop is fixed-iteration, so no boundary crossing occurs between iterations; the cost identity is the additive law applied k times. $\square$

Remark 17.17 (The incident as a type error). The syscall-dominance measured throughout this thesis and the 5.2 multiplexing gain of the boundary remark are the same fact seen twice: a hot path that crosses the kernel boundary pays for it per request. Theorem 17.16 is the enforcement: a dataplane module whose loop body is not typed zero-boundary is rejected, and the process-status governor incident is a typing error, not a performance bug. This is the linearity discipline of chapter 12 extended from allocation to syscalls.

17.9 Route matching as a deterministic automaton

A production HTTP server’s rules are a finite set of *disjoint* patterns: exact paths and trailing-prefix patterns, each with a method mask and an exclude list. Disjointness is a load-time theorem (an overlap check rejects conflicting rule sets before any request is served). The data is a regular language, and the obvious matcher, a linear scan over the rules, is only one of the two canonical realizations.

Theorem 17.18 (Route languages are regular; the pattern trie is a deterministic automaton). *Let P be a set of disjoint patterns over the alphabet of path bytes, each pattern either exact or a trailing prefix. Define the pattern automaton $T(P)$ as the trie whose nodes are the distinct pattern prefixes; each exact pattern terminates at its own bytes, each prefix pattern terminates at the prefix separator; every node carries its include terminals (rule, method mask) and exclude terminals (rule), split by whether the terminal is exact or prefix. Then $T(P)$ decides membership of a path p in the route language of P in exactly $|p|$ transitions, without backtracking and without allocating.*

Proof. Walk p byte by byte. A prefix include terminal fires the moment its node is visited, because a prefix pattern matches p if and only if p starts with the prefix, which is exactly the walk reaching that node. An exact terminal fires only at the last visited node, because an exact pattern matches p if and only if the walk ends there. Excludes are collected into a running bitmask: prefix excludes set their bit when visited, exact excludes only at the final node. Disjointness guarantees at most one rule can match p , so the first method-matching include terminal is the candidate, and the candidate matches if and only if no exclude bit of it was set. Both firing rules are exactly the predicate of the linear matcher, so the automaton and the scan agree on every path. The loop is one pass over the bytes: $|p|$ transitions, each a binary search over the node’s sorted children. The equivalence is machine-checked in-crate by a 200,000-path fuzz test comparing the automaton against the scan on every path, including prefix boundaries, method splits, and exclude vetoes. $\square$

Theorem 17.19 (The automaton is the Myhill–Nerode refinement). *The states of $T(P)$ are the distinct pattern prefixes. Under the Myhill–Nerode quotient of the route language, two prefixes are equivalent exactly when their futures, the sets of patterns that can still complete from them, coincide; the trie states are the prefixes that carry distinct futures. Hence $T(P)$ is a refinement of the minimal deterministic automaton of the route language, and it is exactly minimal for prefix-closed pattern sets with pairwise-distinct futures.*

Proof. The Myhill–Nerode relation identifies prefixes with identical right languages [Nerode, 1958]; the quotient is the minimal deterministic automaton [Hopcroft, 1971], and Brzozowski’s derivatives compute it algebraically [Brzozowski, 1964]. For a trie of patterns, the future of a node is determined by the set of terminals in its subtree: two nodes with identical terminal sets are identified by the quotient, and nodes with different terminal sets are distinguished by the suffix that ends at a terminal present in one but not the other. The trie therefore collapses exactly the same states as the quotient when futures are distinct, and refines it otherwise. Prefix languages have minimal

automata of exactly this shape: one state per distinct prefix [Brlek et al., 2010]. For the shipped rule set the futures are in practice always distinct, so the automaton is the minimal one. $\square$

17.10 Realization choice is a measured threshold

Theorem 17.20 (Hybrid matcher selection). *Let the scan cost be affine in the rule count, $c_{\text{scan}}(R) = aR + b$, and the automaton cost be flat in R . Then by the affine crossover theorem (Theorem 17.6) the cheaper realization is decided by a single threshold R^* , and on the measured grid the threshold is the lattice minimum of Theorem 17.8.*

Proof. This is Theorem 17.6 instantiated with payload R . The measured table (release build, nanoseconds per match on identical rule sets and paths):

R	2	4	8	16
scan	5.9	10.6	25.8	59.6
automaton	29.4	31.7	31.8	43.9

The scan is affine in R (fit $\approx 4.2R - 2.5$ ns); the automaton is flat. The fitted crossover is $R^* \approx 10$ –12, and on the measured grid $\{2, 4, 8, 16, \dots\}$ the automaton first wins at $R = 16$. The shipped constant is the lattice minimum of Theorem 17.8: the decision rule, scan below 16 rules and automaton at 16 or above, is correct because the scan is cheaper for every $R \leq 8$ (the production range) and the automaton is cheaper for every $R \geq 16$. $\square$

Remark 17.21. The same threshold method selects between two realizations of the reactor loop at startup (section 17.12) and between the byte path and the file-transfer path of a static-file molecule (section 17.3). In each case the choice is computed from measured cost lines, not chosen by taste.

17.11 The zero-allocation claim is machine-checked

The hot-path discipline of Theorem 17.16 is a typing claim. The claim is enforced by measurement: a counting allocator, installed as the global allocator of a test process, counts every allocation the process makes. The datapath test runs a full echo cycle (receive batch, validate, echo batch) for 50 rounds and asserts that the counter is unchanged. The control test, which allocates a known buffer, proves the counter observes allocations; without the control the assertion could pass vacuously.

17.12 Startup lattice autotuning

The reactor strategy is a realization choice with the same shape as Theorem 17.20: each strategy has a fixed cost and a load-dependent cost, and the cheapest strategy depends on the machine and the kernel. The startup autotuner measures the candidates on the running machine with a fixed benchmark (windowed echo), applies a soundness floor (a candidate whose TCP path cannot sustain half the best TCP rate is disqualified, because a reactor that stalls its connection path is not a viable production choice), and selects the remaining candidate with the highest UDP throughput. On the reference machine the measured order is: readiness-driven first, completion-driven second, completion-driven with a kernel polling thread last; the completion-driven candidates are disqualified on this kernel by the TCP soundness floor. The winner is a lattice minimum: the finite set of strategy realizations, graded by the measured boundary-enriched cost, with the soundness floor as the admissibility predicate.

Algorithm 1 Baseline: the input-output molecule (before).

```
1: for each block do
2:    $buf \leftarrow \text{ALLOC}(block\_size)$  ▷ per-block allocation
3:    $n \leftarrow \text{READ}(src, buf)$  ▷ one syscall
4:    $\text{WRITE}(dst, buf[0..n])$  ▷ one syscall
5:    $acc \leftarrow acc + \text{CHECKSUMSCALAR}(buf[0..n])$  ▷ byte loop
6: end for
```

Algorithm 2 Optimization 1: preallocation and gathering (after).

```
1:  $buf \leftarrow \text{ALLOC}(block\_size)$  ▷ once
2: while  $n \leftarrow \text{READV}(src, buf) > 0$  do
3:    $\text{WRITEV}(dst, buf[0..n])$ 
4:    $acc \leftarrow acc + \text{CHECKSUMSCALAR}(buf[0..n])$ 
5: end while
```

17.13 Measured case studies: before and after

This section applies the discovery method of section 17.4 to two concrete molecules. For each molecule the section defines a baseline realization, applies individual optimizations one at a time, and finally applies the full stack. Every number is measured on one machine (Intel Core i5-5200U, kernel 7.2.0_1) with identical inputs across stages; the raw output is in the source distribution ([Docs/paper/data/demo-metrics.csv](#)). The metric set is: instructions and instructions per cycle (IPC), branch miss rate, L1 data-cache miss rate, L3 (last-level cache, LLC) load miss rate, dTLB miss rate, wall time (latency, with p50 and p99 percentiles over 20 runs per stage), peak resident set (RSS), and page faults. The processor does not expose the stalled-cycles performance events, so pipeline stalls are reported through the cycle and IPC columns, which incorporate them.

17.13.1 Case study 1: the input-output molecule

The molecule copies a 512 MiB file and computes a byte-sum checksum over the data, a miniature of a file-serving pipeline. The baseline realization is the naive atom wiring.

The first optimization fuses the boundary: a preallocated buffer and gathered system calls remove the per-block allocation and the two syscalls per block become one system call per transfer.

The second optimization fuses the compute: the checksum runs in the same pass over the same buffer, vectorized over 32-byte blocks with a scalar remainder, so the data is touched once and the instruction count drops.

The stacking is compounding: gathering alone gains 1.7%; the fused vectorized checksum on top of gathering gains another 13.4%, for a cumulative 15.4% over the baseline. Wall time falls 14.0% at the p50 (357 to 307 ms) and 8.1% at the p99 (447 to 411 ms). The instruction count falls 5.4% and the cycle count falls 5.0%; the throughput gain exceeds the cycle gain because the vectorized checksum reduces memory traffic (L1-dcache loads fall from 647 M to 611 M, and the L3 load miss rate falls from 34.9% to 29.1%).

17.13.2 Case study 2: the pure compute molecule

The molecule enumerates the primitive Pythagorean triples (a, b, c) with $c \leq N$: triples with $a^2 + b^2 = c^2$ and $\text{gcd}(a, b) = 1$. The baseline is the naive wiring: a double loop over a and b with a square-root test per candidate and a per-candidate allocation.

Algorithm 3 Optimization 2: fused vectorized checksum (after).

```
1:  $buf \leftarrow \text{ALLOC}(block\_size)$ 
2: while  $n \leftarrow \text{READV}(src, buf) > 0$  do
3:    $\text{WRITEV}(dst, buf[0..n])$ 
4:    $acc \leftarrow acc + \text{CHECKSUMSIMD}(buf[0..n])$   $\triangleright$  same pass
5: end while
```

Table 17.1: Input-output molecule: measured stages. Baseline (B), plus gathering ($B + G$), plus fused vectorized checksum ($B + G + V$).

Metric	B	$B + G$	$B + G + V$
Throughput (MiB/s)	1381.2	1405.1	1593.9
Wall time, p50 (ms)	357	352	307
Wall time, p99 (ms)	447	442	411
Instructions	2.179 G	2.191 G	2.062 G
Cycles	1.917 G	1.870 G	1.821 G
IPC	1.14	1.17	1.13
Branch misses (%)	0.40	0.40	0.41
L1 data misses (%)	11.66	10.19	11.01
L3 load misses (%)	34.9	32.5	29.1
dTLB misses (%)	0.011	0.010	0.011
Page faults	90	91	91
Peak RSS (KiB)	2016	2076	2028

Optimization 1 replaces the search by Euclid’s formula: every primitive triple arises exactly once from coprime parameters $m > n$ with $m - n$ odd, as $(m^2 - n^2, 2mn, m^2 + n^2)$. No square root and no allocation remain.

Optimization 2 unrolls the inner sweep: two adjacent candidates are evaluated per iteration, which halves the loop control overhead for the dominant parity class.

The comparison is normalized per triple: Euclid’s formula uses 2.29 thousand instructions per triple against 2.04 million for the brute force, a factor of about 890. The wall time per triple (p50 over 20 runs) falls from about $381 \mu\text{s}$ to about 72 ns, a factor of about 5,300 (the brute force tests every pair up to N ; Euclid’s formula tests only the coprime parameter pairs). The unroll on top of Euclid cuts the instruction count a further 12% and the cycle count a further 5%, and the wall time at the p50 falls from 23 to 22 ms. The branch miss rate rises with Euclid because the parity and coprimality tests are genuinely data-dependent branches; the absolute instruction count is what dominates.

17.13.3 How the optimizations stack

For the input-output molecule the stack is baseline $\rightarrow$ gathering $\rightarrow$ fused vectorized checksum, with cumulative improvement 1.7% then 15.4%. For the triple generator the stack is baseline $\rightarrow$ Euclid $\rightarrow$ unroll, with per-triple instruction reduction of factor 890 then a further 12%. In both cases the individual gains compound: each stage preserves the previous stage’s behavior (the checksum and the triple set are identical across stages) and adds a cost reduction on top. This is the lattice method of section 17.4 in measurement form: each stage is a behavior-preserving rewrite, and the stages compose.

Algorithm 4 Baseline: the triple generator (before).

```
1: for  $a \leftarrow 1$  to  $N$  do
2:   for  $b \leftarrow a$  to  $N$  do
3:      $c \leftarrow \lfloor \sqrt{a^2 + b^2} \rfloor$  ▷ float per candidate
4:     if  $c^2 = a^2 + b^2$  and  $c \leq N$  and  $\text{gcd}(a, b) = 1$  then
5:        $\text{EMIT}(a, b, c)$  ▷ per-triple allocation
6:     end if
7:   end for
8: end for
```

Algorithm 5 Optimization 1: Euclid’s formula (after).

```
1: for  $m \leftarrow 2$  while  $m^2 + 1 \leq N$  do
2:   for  $n \leftarrow 1$  to  $m - 1$  do
3:     if  $(m - n)$  odd and  $c = m^2 + n^2 \leq N$  and  $\text{gcd}(m, n) = 1$  then
4:        $\text{EMIT}((m^2 - n^2, 2mn, c))$ 
5:     end if
6:   end for
7: end for
```

17.14 Verification and summary

Every arithmetic claim of this chapter is proved in full in the text: the boundary laws (Theorem 17.2), the batch identities (Theorem 17.4), the affine crossover (Theorem 17.6), the lattice finiteness (Theorem 17.8), the band contraction (Theorem 17.14), and the automaton equivalence (Theorem 17.18) and its Myhill–Nerode refinement (Theorem 17.19). The computational claims, the finite checks that the arithmetic licenses, are discharged by the executable verifiers of Appendix A; the measured tables of this chapter are reproducible from the source distribution.

The method of this chapter: enrich the cost model with the kernel boundary, prove that realization choice is a threshold, and take the minimum of the finite lattice of behavior-preserving rewrites. It is the answer to the question “what is the last optimization?” It is not a bag of tricks; it is the decision procedure that Theorem 17.8 proves exists, applied to the molecules this thesis constructs. The optimizations discovered this way are exactly the ones the measurements confirm: fewer syscalls per request, the correct datapath at the correct payload size, and a zero-boundary loop as the type-correct realization of a hot path.

Table 17.2: Triple generator: measured stages. Baseline (B), Euclid (E), Euclid unrolled ($E + U$).

Metric	B	E	$E + U$
Input	$N = 20,000$	$N = 2,000,000$	$N = 2,000,000$
Triples found	3,186	318,320	318,320
Instructions per triple	2.04 M	2.29 k	2.01 k
Instructions (total)	6.483 G	0.728 G	0.640 G
Cycles (total)	3.333 G	1.214 G	1.159 G
IPC	1.95	0.60	0.55
Wall time, p50 (ms)	1215	23	22
Wall time, p99 (ms)	1223	23	23
Branch misses (%)	0.02	5.40	6.00
L1 data misses (%)	7.42	6.92	6.77
L3 load misses (%)	43.4	35.5	54.0
dTLB misses (%)	0.21	0.36	0.28
Page faults	78	77	77
Peak RSS (KiB)	1936	1916	1916

Appendix A

Proof-Checking Methodology

Every theorem of this thesis carries a complete proof in the text. The proofs were written before finalization and are self-checked during authoring: each step is justified by an axiom, a definition, or an earlier step, and no conjecture is presented as a theorem. This appendix records the verification discipline that complements the pencil-and-paper proofs and the machinery that discharges the computational claims.

A.1 The Verification Discipline

Claims in the thesis fall into two classes, with different verification regimes.

Structural claims (the existence, uniqueness, congruence, finiteness, and soundness statements of the catalog) are proved in full in the text and are the subject of the proofs themselves. Nothing in this class is outsourced to a tool; the tools below never substitute for a proof, they verify specific *computational* consequences of one.

Computational claims (termination of a completion procedure, exhaustiveness of a normal-form enumeration, satisfaction of a numerical bound, existence of a bisimulation witness) assert that a procedure run over finitely many cases terminates with the claimed outcome. These are claims about the execution of a finite computation, and they are verified by executable tools written in Rust, using only the standard library. The constraint is deliberate: the verification tooling must build offline with no third-party dependencies, so that the certification of a computational claim never depends on the availability or behavior of external packages. The tools live in the `verify` subdirectory of the paper sources.

The output of every tool run is a `PASS` or `FAIL` line together with the evidence it produced. All verification is deterministic. A claimed `PASS` must reproduce on a clean build.

A.2 The Verification Tools

Six tools implement the computational checks. Table A.1 lists each tool, the theorems whose computational content it verifies, and precisely what it checks. Each tool is an independent binary; the build script runs all six under its `--verify` flag and fails the paper build on any `FAIL`.

A.3 Reproducibility

The verification pipeline is part of the repository, not an appendix to it. The sources of all six tools, their `Cargo` manifest, and the full `PASS` logs of the runs that certify the claims of the thesis are

Table A.1: The proof-verification tools. Each tool is a Rust binary in `verify/`, std-only, deterministic, and prints `PASS` or `FAIL`.

Tool	Theorems	What it checks
<code>kb_completion</code>	5.3 (supports 5.1–5.2, 5.7)	Runs Knuth–Bendix completion on the ring theory of chapter 4; asserts that completion terminates in a complete system and that every critical pair generated during completion joins.
<code>normal_forms</code>	6.6–6.9	Exhaustive enumeration of normal forms on small finite signatures; asserts finiteness of hom-sets, uniqueness of normal forms, and the completeness equivalence $M \approx M' \iff \text{NF}(M) = \text{NF}(M')$ on all enumerated pairs.
<code>contraction</code>	10.6	Bound arithmetic: for sampled $(\alpha, \varepsilon, d_0)$ with $\alpha < 1$, computes $k(\alpha, \varepsilon, d_0) = \lceil \ln(\varepsilon(1 - \alpha)/d_0) / \ln \alpha \rceil$ and asserts the fixed-point iterate stays within ε after k steps.
<code>bisim</code>	3.23	Exhaustive congruence check: enumerates small finite machines, computes behavioral equivalence, and asserts that the equivalence is preserved by composition $\circ$ and tensor $\otimes$, i.e. that $\approx$ is a congruence.
<code>batch_amort</code>	11.2	On sample batch sizes n , asserts the amortization bound $\text{cost}(\text{batch}_n) \leq n \cdot \text{cost}(\text{single}) + \text{fixed}(n)$ and that $\text{fixed}(n)/n \rightarrow 0$ along the sample.
<code>affine typer</code>	12.6	Linear typing of sample molecules in the affine system: asserts every variable is consumed exactly once, contraction and weakening are rejected, and evaluation preserves the leaf multiset and never increases the node count (no allocation).

committed at `Docs/paper/verify/`, with the logs in `Docs/paper/verify/logs/`. Each log records the tool name, the commit or source hash of the tool, the sampled inputs, and the `PASS` verdict. A reviewer reproduces the verification by building the `verify` crate and running each binary; the `--verify` flag of the build script runs all six in sequence and gates the paper build on their success.

A.4 Scope of the Machine Checks

The checks verify the *computational content* of the cited theorems on the sampled or enumerated finite cases; they do not prove the theorems. The structural argument (why the finite check licenses the general statement) is always the theorem’s proof in the text, and the tool is cited as evidence only for the specific run it performs. The tooling here is the minimum that makes every computational claim of the thesis reproducible, and no more.

Appendix B

Optimization and Testing Catalogs

This appendix consolidates the framework’s engineering content into two reference tables. Table B.1 maps each algebraic mechanism of the theory to the optimization it licenses, with the governing theorem. Table B.2 maps each feature of the framework to the testing strategy it generates, as developed in chapter 14. Both tables are intended to be used directly by implementers: a molecule description is inspected against the left-hand column, and the corresponding row names the optimization that is provably sound and the tests that must discharge it.

B.1 Theorem-to-Optimization Catalog

Each row states an algebraic law of Mol, the optimization it licenses, and the theorem that makes the optimization sound. Each optimization in this table is a rewriting of the molecule that preserves behavior (or provably improves cost) in every context where the governing theorem applies.

B.2 Testing-Strategy Catalog

Each row names a structural feature of the framework, the optimization class it enables, and the testing strategy that certifies the optimized artifact. The strategies are those of chapter 14; the governing results column cites the theorem the tests operationalize.

B.3 How to Read the Catalogs

The two tables are the bridge from the theory to the standard that accompanies this thesis: the standard’s policies cite the governing theorems, the decision matrices select among the optimizations above, and the testing policies instantiate the testing column. Together they close the loop that the thesis opened: a networking dataplane is a Mol diagram, its optimizations are provable rewrites of that diagram, and its certification is the mechanical discharge of the diagram’s laws.

Table B.1: The optimization catalog: algebraic mechanism to licensed optimization.

Algebraic mechanism	Licensed optimization	Governing result
Associativity of $\circ$	Pipeline fusion: reassociate adjacent stages to eliminate intermediate materialization.	3.7, 12.1
Interchange law	Parallelize independent branches: $(f \otimes g); (h \otimes k) = (f; h) \otimes (g; k)$ splits work across cores.	4.7
Trace sliding	Hoist pure atoms out of loops: $\text{Tr}(f; M) = f; \text{Tr}(M)$ moves f across the feedback boundary.	8.2
Trace vanishing	Flatten nested state machines: $\text{Tr}(\text{Tr}(M)) = \text{Tr}(M)$ collapses nested loops.	8.3
Cut elimination	Deforestation, zero allocation: every pure pipeline fuses to a single morphism.	12.1
Monad laws	Combine state updates: Kleisli composition groups state reads and writes into one transition.	3.17
Isomorphisms	Eliminate parse-serialize roundtrips: $\text{parse}; \text{serialize} \approx \text{id}$ on the well-formed domain.	13.2
Optics fusion	Single-offset field access: compose accessors into one offset computation instead of a chain.	<i>none</i>
Parametricity	Data-parallel SIMD: polymorphic pipelines are natural, so vectorization is behavior-preserving.	Wadler [1989]
Automata minimization	Fewer branches: minimal state space gives the minimal dispatch structure.	9.6, 9.7
Linear logic	Zero-copy by construction: linear atoms cannot allocate in a well-typed hot path.	12.2
Graded cost enrichment	Cache-line-aware layout, false-sharing avoidance, branch-predictable ordering: pick the molecule minimizing the graded cost.	7.22
Modular completion	Extensible rewrite engine: new equational theories join without breaking termination or confluence.	5.9
Affine typing	No-allocation guarantee: linear typing enforces zero allocation at compile time.	12.6

Table B.2: The testing catalog: framework feature to derived testing strategy.

Feature	Optimization class	Testing strategy	Governing results
Category laws	Algebraic rewrites, fusion	Equational testing: assert both sides of every law on generated traces.	3.7–3.23
Monoidal structure	Reordering, batching, SIMD	Interchange tests: assert $(f \otimes g); (h \otimes k) = (f; h) \otimes (g; k)$.	3.12, 4.7
Trace	Hoist, flatten	Sliding/vanishing tests: assert $\text{Tr}(f; M) = f; \text{Tr}(M)$ and $\text{Tr}(\text{Tr}(M)) = \text{Tr}(M)$.	8.2–8.4
Lawvere theories	Cancel ring ops	Conformance tests: components must satisfy the theory axioms.	4.3–4.8
Cost enrichment	Shortest-path synthesis	Cost regression: assert additive costs, triangle inequality, fusion bound.	7.3–7.8
Poset refinement	Specialization	Refinement tests: assert $f \sqsubseteq f' \wedge g \sqsubseteq g' \Rightarrow g \circ f \sqsubseteq g' \circ f'$.	7.11–7.13
Metric enrichment	Fixed-iteration loops	Convergence validation: assert $\ x_{k+1} - x^*\ \leq \varepsilon$ within the iteration bound.	10.5–10.8
Coalgebraic minimization	Merge states	Language equivalence: minimized artifact must bisimulate the original.	9.6–9.9
Operads	Batch consolidation	Associativity of batching: assert the flattening identity of 11.1.	11.1–11.2
Parametricity	Vectorization	Naturality tests: polymorphic behavior is uniform across concrete types.	Wadler [1989]
Linear logic	Zero-copy	Ownership/borrow checks: allocation counter and borrow checker enforce linearity.	12.1–12.2

Bibliography

- Jiří Adámek and Jiří Rosický. *Locally Presentable and Accessible Categories*, volume 189 of *London Mathematical Society Lecture Note Series*. Cambridge University Press, 1994.
- Stefan Banach. Sur les opérations dans les ensembles abstraits et leur application aux équations intégrales. *Fundamenta Mathematicae*, 3:133–181, 1922.
- Garrett Birkhoff. On the structure of abstract algebras. *Proceedings of the Cambridge Philosophical Society*, 31(4):433–454, 1935.
- Filippo Bonchi, Elena Di Lalore, and Mario Román. Effectful mealy machines, 2024.
- Srećko Brlek, Pierre Jamet, and Guy Paquin. Syntactic complexity of ideal and closed languages, 2010. arXiv:1010.3263.
- Janusz A. Brzozowski. Derivatives of regular expressions. *Journal of the ACM*, 11(4):481–494, 1964.
- Kenta Cho, Bart Jacobs, Bas Westerbaan, and Abraham Westerbaan. An introduction to effectus theory, 2015.
- Thomas Colcombet and Daniela Petrişan. Automata minimization: a functorial approach, 2017.
- U. Dal Lago and M. Hofmann. Quantitative models and implicit complexity, 2005.
- Roland L. Dobrushin. Central limit theorem for nonstationary Markov chains. *Theory of Probability & Its Applications*, 1(4):329–383, 1956.
- D. B. A. Epstein and Paul J. Sanders. Automatic groups and knuth-bendix with infinitely many rules, 1998.
- Tobias Fritz. A synthetic approach to markov kernels, conditional independence and theorems on sufficient statistics, 2019.
- Jean-Yves Girard. Linear logic. *Theoretical Computer Science*, 50(1):1–101, 1987.
- John E. Hopcroft. An $n \log n$ algorithm for minimizing states in a finite automaton. Technical Report CS-TR-71-190, Stanford University, 1971.
- Nick Hu and Jamie Vicary. Traced monoidal categories as algebraic structures in prof, 2021.
- Jules Jacobs and Thorsten Wißmann. Fast coalgebraic bisimilarity minimization, 2022.
- André Joyal, Ross Street, and Dominic Verity. Traced monoidal categories. *Mathematical Proceedings of the Cambridge Philosophical Society*, 119(3):447–468, 1996.

- G. Max Kelly. On clubs and doctrines. In *Category Seminar*, volume 420 of *Lecture Notes in Mathematics*, pages 181–256. Springer, 1974.
- Donald E. Knuth and Peter B. Bendix. Simple word problems in universal algebras. In *Computational Problems in Abstract Algebra*, pages 263–297. Pergamon Press, 1970.
- Stephen Lack and Jiří Rosický. Notions of lawvere theory, 2008.
- F. William Lawvere. Functorial semantics of algebraic theories. *Proceedings of the National Academy of Sciences*, 50(5):869–872, 1963.
- Tom Leinster. *Higher Operads, Higher Categories*, volume 298 of *London Mathematical Society Lecture Note Series*. Cambridge University Press, 2004.
- Saunders Mac Lane. *Categories for the Working Mathematician*, volume 5 of *Graduate Texts in Mathematics*. Springer, 2nd edition, 1971.
- George H. Mealy. A method for synthesizing sequential circuits. *Bell System Technical Journal*, 34(5):1045–1079, 1955.
- François Métayer. State monads and their algebras, 2004.
- Edward F. Moore. Gedanken-experiments on sequential machines. In *Automata Studies*, volume 34 of *Annals of Mathematics Studies*, pages 129–153. Princeton University Press, 1956.
- Anil Nerode. Linear automaton transformations. *Proceedings of the American Mathematical Society*, 9(4):541–544, 1958.
- M. H. A. Newman. On theories with a combinatorial definition of “equivalence”. *Annals of Mathematics*, 43(2):223–243, 1942.
- David Park. Concurrency and automata on infinite sequences. In *Theoretical Computer Science (ICALP)*, volume 104 of *Lecture Notes in Computer Science*, pages 167–183, 1981.
- Russell Power. Using memory-protection to simplify zero-copy operations, 2013.
- Thomas Seiller. Interaction graphs: Full linear logic, 2015.
- Yoshihito Toyama. On the Church–Rosser property for the direct sum of term rewriting systems. *Journal of the ACM*, 34(1):128–143, 1987.
- Philip Wadler. Theorems for free! In *Proceedings of the Fourth International Conference on Functional Programming Languages and Computer Architecture (FPCA)*, pages 347–359, 1989.
- Mark Weber. Operads as polynomial 2-monads, 2014.